密级：_____

中国科学院大学
University of Chinese Academy of Sciences

博士学位论文

Java 虚拟机性能优化技术研究

作者姓名：傅　杰

指导教师：王剑　研究员级高工

中国科学院计算技术研究所

学位类别：工学博士

学科专业：计算机科学与技术

研 究 所：中国科学院计算技术研究所

二〇一六年五月

Optimization of Java Virtual Machines

A Dissertation Submitted to

The University of Chinese Academy of Sciences

in partial fulfillment of the requirement

for the degree of

Doctor of Philosophy

in

Computer Science and Technology

by

Jie Fu

Dissertation Supervisor: Professor Jian Wang

Institute of Computing Technology

Chinese Academy of Sciences

May, 2016

声　　明

我声明本论文是我本人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，本论文中不包含其他人已经发表或撰写过的研究成果。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作者签名：　　日期：

论文版权使用授权书

本人授权中国科学院计算技术研究所可以保留并向国家有关部门或机构送交本论文的复印件和电子文档，允许本论文被查阅和借阅，可以将本论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编本论文。

（保密论文在解密后适用本授权书。）

作者签名：　　导师签名：　　日期：

摘　要

云计算和大数据的迅猛发展使得 Java 的应用更为广泛，同时也对 Java 的性能提出了更高的要求。传统 Java 虚拟机的设计已不能满足实际系统的需求。如何提高 Java 虚拟机的性能已成为现代信息产业亟需解决的关键问题。因此，研究 Java 虚拟机性能的优化技术具有十分重要的现实意义，已成为现代计算机系统领域研究的热点。

动态编译和解释执行是 Java 虚拟机执行应用程序的两种方式，直接决定了 Java 虚拟机的性能。本文分别从动态编译和解释执行两方面对 Java 虚拟机性能优化技术进行了研究，针对动态编译的程序热点识别、动态编译调度和解释器指令分派提出了一系列性能优化技术，并在龙芯平台上进行了验证。本文的主要贡献和创新点如下：

1. 提出了基于贝叶斯估计的程序热点识别算法。为了提高热点识别的灵活性以及对程序行为的自适应能力，本文设计了以智能阈值为核心的热点识别模型。在此基础上，提出了一种基于贝叶斯估计的程序热点识别算法。其核心思想是：利用方法当前执行次数以及在未来短时间内可以预期的前瞻性执行次数为观测数据，通过贝叶斯估计实现对程序热点的前瞻性识别。该算法可以显著降低热点识别的延迟，从而大幅降低热点方法的编译延迟，有效提升虚拟机的性能。实验表明，该算法使得系统的启动性能平均提升 14.6%，最高提升幅度高达 61.3%；峰值性能平均提升 4.2%，最高提升幅度高达 24.0%。
2. 提出了编译队列监视下的高性能动态编译调度算法。本文深入分析了动态编译行为的主要特征，揭示了动态编译调度的优化目标。在此基础上，提出了编译队列监视下的高性能动态编译调度算法。其核心思想是：通过降低热点方法的排队延迟和运行时开销来提升系统的性能。调度时以程序中方法的重要程度和编译代价为参数计算调度的优先级，既能增强系统的响应能力，又能降低编译队列的平均排队延迟。此外，在调度的同时进行编译队列监视，通过跟踪编译队列中方法的活跃性，以延迟非活跃方法的编译，并且选择性地将部分非活跃方法从编译队列中删除，进一步降低了系统的运行时开销。实验表明，该算法使得系统的启动性能平均提升 8.9%，最高提升幅度高达 41.8%。
3. 提出了一种软硬件协同设计的解释器指令分派优化技术。本文对解释器指令分派的开销进行了量化分析，指出对字节码本地指令序列入口地址的获取构成了指令分派的瓶颈。在此基础上，提出了一种软硬件协同设计的指令分派优化技术。其核心思想是：在软件层面，通过对指令分派表进行优化，消除了代价较高的入口地址加载操作；在硬件层面，通过扩展处理器的访存指令，进一步实现了基于硬件支持的访存加速。软硬件协同设计可以最大限度地降低由指令分派引入的运

行时开销，从而提升解释执行的效率。实验表明，优化后解释器的平均性能提升11.5%，最高提升幅度为 15.4%。

上述部分工作已在龙芯平台的 Java 虚拟机中得到了实际应用。本文提出的一些优化思想和方法对其它虚拟机和应用程序的设计与优化也具有重要的参考意义。

关键词：Java 虚拟机；热点识别；编译调度；指令分派；优化

Abstract

The rapid development of cloud computing and big data makes the application of Java increasingly popular, which results in more challenges on the performance of Java. Traditional design of the Java virtual machine(JVM) can no longer meet the demands of real systems. How to improve the performance of JVMs has become a key issue that needs to be solved in the modern information industry. Therefore, deep research of JVM optimization is of vital importance, and has become a hot research field in computer systems.

Java applications can be executed either by dynamic compilation or interpretation. The efficiency of both execution modes determines the performance of JVMs directly. This dissertation lucubrates dynamic compilation and interpretation, and then proposes a series of optimization technologies regarding hot method detection, dynamic compilation scheduling and instruction dispatch. The main contributions and innovations of this dissertation include:

1. We propose a smart hot method detection algorithm. To improve the flexibility and adaptability of hot method detection, we introduce a novel hot method detection model based on a smart threshold. Then we propose a hot method detection algorithm based on Bayesian estimation. The key to earlier detection of hot methods is to combine the current execution number of a method with its foreseeable execution number in the Bayesian estimation. Our algorithm can significantly reduce the delay of hot method detection and compilation, thus improving the performance. Experimental results show that the startup performance has been improved by 14.6% on average with the highest 61.3% and the peak performance has been improved by 4.2% on average with the highest 24.0%.
2. We propose an efficient dynamic compilation scheduling algorithm. After an in-depth analysis of the features of dynamic compilation, we unveil objectives for the optimization of dynamic compilation scheduling. Based on these objectives, we propose a high performance dynamic compilation scheduling algorithm under compilation queue monitoring. The key to the performance improvement is to reduce the delay of compilation queue and decrease the runtime overhead of dynamic compilation. Both the importance and the compilation cost of a method are used to compute the scheduling priority, which not only improves responsiveness of the system, but also reduces the delay of compilation queue. Moreover, compilation queue monitoring is performed during the scheduling, which decreases the overhead of dynamic compilation by calculating the activity

of methods in the compilation queue and selectively removing the inactive methods. Experimental results show that the startup performance has been improved by 8.9% on average, and the highest performance boost is up to 41.8%.

3. We propose an optimizing instruction dispatch technique.After quantifying the cost of instruction dispatch, we reveal that loading the entry address of native instructions for each bytecode has become the performance bottleneck. Based on this observation, we propose a hardware and software co-design approach to reduce the overhead of instruction dispatch. The main ideas are in the aspect of software, the instruction dispatch table is re-designed to eliminate the expensive operation of entry address loading, and in the aspect of hardware, the processor's instruction set is enhanced to accelerate the speed of memory access. Both of them are coordinated to minimize the runtime overhead of instruction dispatch, thus improving the performance. Experimental results show that our approach can significantly improve the performance of interpreters. The overall performance has been improved by 11.5%, and the highest performance boost is up to 15.4%.

Part of the above work has been applied in the JVMs for Loongson processors. And some of the proposed ideas and techniques can be easily adopted by other virtual machines and applications.

Keywords: Java virtual machine; hot method detection; compilation scheduling; instruction dispatch; optimization

目　　录

摘要 **I**

目录 **V**

图目录 **IX**

表目录 **XI**

第一章　绪论 **1**

1.1　Java的发展与应用 1

1.2　Java技术体系概述 2

1.3　Java虚拟机面临的挑战 4

1.4　本文的研究内容和主要贡献 6

1.5　本文的组织结构 7

第二章　Java虚拟机及其相关优化技术简介 **9**

2.1　引言 9

2.2　Java虚拟机基础 9

2.2.1　运行时数据区 10

2.2.2　执行引擎 12

2.2.3　类加载器 14

2.2.4　垃圾收集器 15

2.2.5　本地方法接口 18

2.3　Java虚拟机性能优化技术 18

2.3.1　解释执行的优化技术 18

2.3.2　动态编译的优化技术 24

2.3.3　虚拟机其它优化技术 31

2.4　本章小结 32

第三章 实验平台介绍 33
3.1 引言 33
3.2 龙芯高性能通用处理器 33
3.2.1 GS464V处理器核 34
3.2.2 龙芯3B1500处理器 35
3.3 HotSpot虚拟机 36
3.3.1 HotSpot虚拟机总体结构 37
3.3.2 龙芯上的HotSpot虚拟机 38
3.4 测试程序 43
3.5 本章小结 45

第四章 基于贝叶斯估计的程序热点识别算法 47
4.1 引言 47
4.2 相关工作 47
4.3 基于智能阈值的热点识别模型 49
4.4 基于贝叶斯估计的程序热点识别算法 51
4.4.1 先验分布的构建 52
4.4.2 先验分布的更新 54
4.4.3 基于后验分布的估计 55
4.4.4 算法的具体实现 57
4.5 实验结果与分析 59
4.5.1 实验设计 59
4.5.2 实验结果 60
4.5.3 实验分析 61
4.6 本章小结 65

第五章 编译队列监视下的高性能动态编译调度算法 67
5.1 引言 67
5.2 相关工作 68
5.3 动态编译的特征与优化目标 69
5.3.1 动态编译的特征 69
5.3.2 动态编译的优化目标 71
5.4 编译队列监视下的高性能动态编译调度算法 72

5.4.1 收益代价分析的调度模型 72
5.4.2 调度模型的实现 74
5.4.3 自适应的编译队列监视机制 75
5.4.4 算法的具体实现 76
5.5 实验结果与分析 78
5.5.1 实验设计 78
5.5.2 实验结果 79
5.5.3 实验分析 80
5.6 本章小结 82

第六章 软硬件协同设计的解释器指令分派优化 85
6.1 引言 85
6.2 相关工作 85
6.3 解释器设计的关键技术 87
6.4 解释器指令分派的瓶颈分析 88
6.5 软硬件协同设计的解释器指令分派优化 90
6.5.1 指令分派表的优化设计 91
6.5.2 软件优化的指令分派表访问技术 93
6.5.3 基于硬件支持的访存加速策略 95
6.6 实验结果与分析 97
6.6.1 实验设计 97
6.6.2 实验结果 98
6.6.3 实验分析 98
6.7 本章小结 100

第七章 总结与展望 103
7.1 本文工作总结 103
7.2 进一步研究工作 104

参考文献 107

致谢 i

作者简历 iii

图 目 录

1.1 Java技术体系 3
1.2 Java虚拟机动态编译的基本过程 4

2.1 Java虚拟机核心模块 10
2.2 运行时数据区 11
2.3 即时编译的基本过程 13
2.4 类加载器的层次结构图 14
2.5 Switch结构的指令分派 19
2.6 线索化结构的指令分派 20
2.7 解释器操作数栈的变化 21
2.8 采用栈顶缓存优化，解释器操作数栈的变化 22
2.9 静态与动态的编译模式 24
2.10 传统的虚方法调用解析过程 27
2.11 高速内联缓存优化的虚方法调用解析过程 28
2.12 类的层次关系分析图 29
2.13 部分编译优化原理示意图[1] 30

3.1 GS464V处理器核微结构[2] 34
3.2 龙芯3B1500芯片结构[3] 36
3.3 HotSpot虚拟机总体结构 37
3.4 基于体系结构描述语言的即时编译器构造原理 38
3.5 指令规则定义的实例 41
3.6 方法调用堆栈 43

4.1 DaCapo测试集上方法执行次数的统计分析 53
4.2 幂率分布模型的拟合结果 60
4.3 虚拟机总体性能 61
4.4 虚拟机在具体测试项目上的性能 62
4.5 热点方法的识别延迟 62
4.6 热点方法的编译延迟 63

4.7 热点方法识别率 64
4.8 热点识别开销 65
4.9 编译请求次数 65

5.1 每10ms内新识别的热点方法数目 69
5.2 热点方法在编译队列中的排队延迟随时间的变化 71
5.3 编译队列长度随时间的变化 71
5.4 虚拟机总体的启动性能 79
5.5 虚拟机总体的峰值性能 79
5.6 虚拟机在具体测试项目上的启动性能 80
5.7 热点方法在编译队列中的平均排队延迟 80
5.8 编译队列监视机制的效果 81
5.9 僵尸方法的比例 82

6.1 解释器指令分派表 88
6.2 字节码iadd的本地指令序列 89
6.3 指令分派在本地指令序列中所占比例 89
6.4 优化后的指令分派表结构 92
6.5 指令分派表访问时栈顶缓存状态的频率分布 93
6.6 软件优化的指令分派表访问方法 94
6.7 纯软件优化的指令分派 95
6.8 扩展访存指令的硬件实现原理 96
6.9 软硬件协同的指令分派 96
6.10 解释器总体性能 98
6.11 解释器在SPECjvm98和DaCapo测试集上的性能 99
6.12 软硬件协同优化前后，指令分派在字节码的本地指令序列中所占比例 . . 99
6.13 软硬件协同优化后，解释器减少执行的机器指令比例与性能提升幅度 . . 100

表 目 录

2.1 主流的垃圾收集器 17

3.1 寄存器使用规约 39
3.2 关键的Call节点 42
3.3 SPECjvm98基准测试程序简介 44
3.4 DaCapo基准测试程序简介 44

5.1 僵尸方法的比例 70
5.2 本文调度的运行时开销 83

6.1 解释器栈顶缓存状态 87
6.2 解释执行的运行时数据 91
6.3 扩展的访存指令 97
6.4 软硬件协同优化后减少执行的机器指令 101

第一章　绪论

在云计算和大数据时代，高度异构的硬件设备和复杂的网络环境对应用程序的可移植性和安全性提出了更高的要求[4][5]。基于虚拟机技术实现的Java语言，使得程序具备“一次编译，到处运行”的高度可移植性[6]。同时虚拟机内部提供的多种运行时检查机制，如类型转换检查、空指针检查和数组越界检查等，极大增强了程序运行的安全性。此外，Java有着极为丰富的类库，对各领域尤其是网络和数据库等应用编程提供了非常强大的支持，大幅提升了应用软件的开发效率。故Java在云计算和大数据时代得到了广泛的应用。

Java虚拟机是整个Java技术体系的核心，是实现Java平台无关性和安全性的基石，同时也决定了Java的性能。近年来，云计算、大数据和移动互联网的迅猛发展，一方面使得Java的应用更为广泛，另一方面也对Java的性能提出了更高的要求。Java虚拟机的性能往往不能满足实际应用系统的需求。因此，对Java虚拟机性能优化技术的研究已成为近年计算机科学研究的热点。

本章首先概述了Java技术的发展与应用，并介绍了Java技术体系。随后揭示了Java虚拟机面临的性能问题。最后阐述了本文的研究内容和主要贡献，并展示了论文组织结构。

1.1　Java的发展与应用

1995年，Sun公司首次推出了Java程序设计语言及其运行时环境。Java语言是一种面向对象的高级程序设计语言，可用于编写跨平台的应用程序，目前已广泛应用于网络服务器、桌面系统和移动终端等领域，并在全世界拥有最大的开发群体。最初的Java主要面向电视、电话和洗衣机等消费类家电产品进行设计，旨在解决家电产品的互联通讯和智能控制等问题。后来，随着互联网技术的快速发展，互联网市场迅速超过了家电市场，成为新的经济增长点。与此同时，互联网中异构的设备和复杂的网络环境使得网络应用程序的开发面临极大的挑战。由于Java具备平台无关性、安全性和网络移动性等特点，Java迅速转变为面向网络应用的主流编程语言。经过20多年的发展，Java针对不同的应用领域进一步衍生出三大分支，即Java精简版(Java Micro Edition，Java ME)、Java标准版(Java Standard Edition，Java SE) 和Java企业版(Java Enterprise Edition，Java EE)，分别面向嵌入式/移动终端系统、桌面系统和企业级应用系统。

2009年，Oracle公司收购了Sun公司，全面整合了Sun的Java业务，为Java的发展提供了强大的动力。随后，以Oracle公司为主导，产业界迅速制定了包括Java 8、Java

9和Java 10在内的Java十年发展规划路线图。其中，备受关注的是Java为进一步适应云计算和大数据需求而定制的功能升级，重点分为性能提升、模块化部署、网络的互操作性以及对异构计算模型的支持等方面。2014年发布的Java 8升级了Java的核心类库、同时增强了Java的模块化构建、安装部署和维护等功能，并首次实现了支持强大并发处理的Lambda表达式，开启了Java全面进军云计算和大数据领域的序幕。

据Oracle的官方统计数据，全球有超过97%的企业基于Java进行应用开发。全世界范围内大约有30亿台设备运行Java程序。而在整个IT产业中，从事Java应用开发的人员已超过了900万。此外，世界著名的编程语言排行榜TIOBE在2016年1月的统计数据[7]表明，Java的流行度再次位居榜首，人气值高达21.465%，而排名第二的C语言仅为16.036%。2015年，Java的人气值继续保持最高的增长幅度，一举夺得TIOBE排行榜2015年度的编程语言奖。因此，Java技术的应用和发展达到了前所未有的高度。凭借其优秀的平台无关性、安全性和网络移动性等优势，Java必将在云计算、大数据、移动互联网和物联网等新兴应用中大显身手。与此同时，上述新兴的应用模式将进一步巩固和增强Java的优势，使Java焕发出新的活力，并且预示着Java巨大的发展空间和应用前景。

1.2 Java技术体系概述

Java的主要目标是实现应用程序的跨平台兼容性，并向上提供一个安全的运行时环境。经过20多年的不断创新和发展，Java应用已全面渗透到服务器、桌面和移动终端等领域，形成了一套高度完善的技术体系。Java技术体系主要包括Java语言、字节码与类文件、Java虚拟机和Java应用程序编程接口等[6]，它们之间的关系如图1.1所示。Java语言是Java应用程序的编程语言，采用面向对象的程序设计理念。Java源程序经过Java编译器编译后被转换为存储在标准类文件中的字节码，且最终以字节码的形式对外发布。Java虚拟机屏蔽了底层操作系统和硬件的差异，为Java程序提供了统一的运行时环境。而丰富的Java应用程序编程接口则为应用程序的开发带来了极大的便利。因此，Java不仅是一种编程语言，更是一种开发平台和运行环境，其应用早已渗透到人类生产和生活的方方面面，是现代信息产业中应用最广泛和最具影响力的基础平台。

Java虚拟机是Java技术体系的核心，负责对Java程序进行模拟执行，是影响系统性能的关键因素。Java虚拟机是Java实现平台无关性、安全性和许多高级语言特性的基础。对上述特性的支持使得Java虚拟机的设计所面临许多性能问题。首先，为了实现平台无关性，Java程序以字节码的形式对外发布。Java虚拟机在执行Java程序时通常需要将Java字节码先等价转换为本地机器指令，再由处理器直接执行转换后的机器指令。上述转换过程存在巨大的运行时开销，是制约Java虚拟机性能的主要因素。其次，为了提供更加安全可靠的运行时环境，Java虚拟机需要实现多种安全性检查机制，如类型转换检查、空指针检查和数组越界检查等。额外的安全性检查操作会引入大量的运

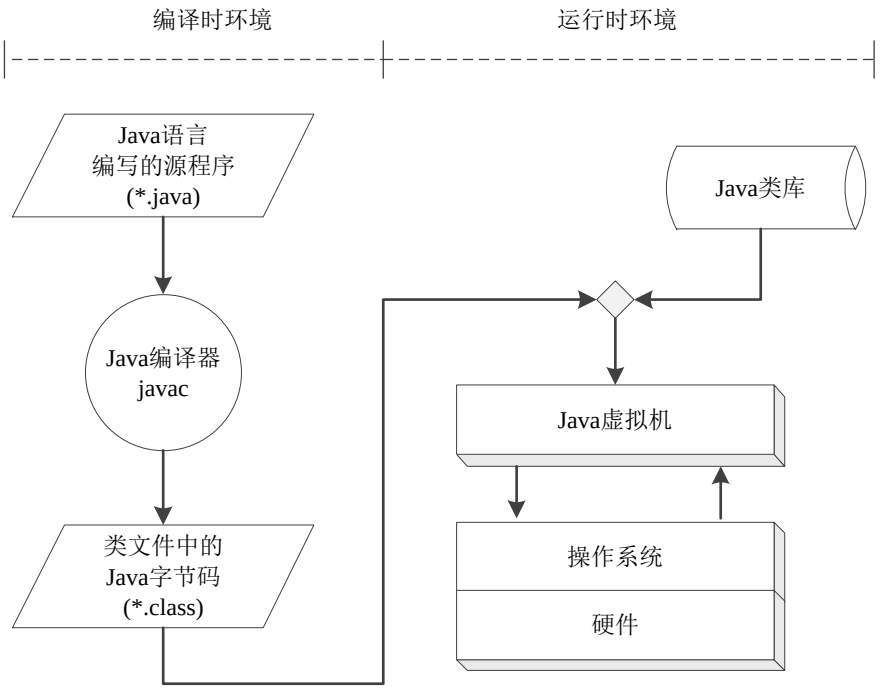


图 1.1: Java技术体系

行时开销，进一步限制了Java虚拟机的性能。最后，Java语言的动态性、多态性和自动内存管理等许多特性也增加了Java虚拟机设计的难度。

为了解决上述问题对Java性能的挑战，现代Java虚拟机系统中设计了先进的执行引擎。Java虚拟机的执行引擎通常包括解释器(interpreter)[8–10]和即时编译器(just-in-time compiler, JIT)[11–13]两类。解释器结构简单，易于实现和维护，故Java虚拟机最早的执行引擎仅由解释器来实现。但解释器存在运行效率过低的缺陷，为了提升虚拟机的性能，人们引入了JIT编译器。由于JIT编译器的编译行为发生在程序运行期间，故又被称为动态编译。研究表明，编译后生成的本地机器代码的执行效率通常比解释执行高出10倍以上。因此，动态编译是决定虚拟机性能最为关键的因素。

和传统C/C++等静态编译不同，动态编译行为发生在程序运行期间，会引入额外的编译开销。研究表明，程序运行的时间分布具有明显的局部性，即程序执行的大部分时间集中在一小部分代码上。为了尽可能降低由动态编译产生的额外开销，现代虚拟机的动态编译系统通常依据“选择性编译”(selective compilation)理论[12]进行设计和实现。其核心思想是：仅编译程序中频繁执行的部分(称为程序的“热点”)。图1.2以HotSpot虚拟机为例展示了Java虚拟机动态编译的基本过程。HotSpot虚拟机[14][15]是当前实际应用最为广泛的Java虚拟机，最初由Sun公司研发，并于2006年开放了全部源代码。HotSpot的执行引擎包括解释器和即时编译器，并且以方法为基本

单位进行动态编译。根据选择性编译的原理，HotSpot仅翻译执行频度较高、对程序性能影响较大的方法，即热点方法。程序中所有的方法均由解释器开始执行。在解释执行期间，虚拟机为每个方法分别维护一个方法调用计数器和内部循环计数器。当上述两个计数器的和超过预设的阈值时，对应的方法便被识别为热点方法并被加入到编译队列中等待编译。随后，编译线程会按照一定的编译调度策略从编译队列中选择方法进行编译。方法编译完成后，后续的调用均直接执行编译后生成的本地代码。

为了充分发挥多核处理器的性能，现代高性能虚拟机的动态编译系统普遍采用了多线程并发编译和后台编译的技术[16]。虚拟机会根据具体的软硬件平台信息自适应地启动多个编译线程对程序热点进行并发编译，可加快动态编译的速度，大幅提升动态编译的吞吐量。同时，在多核处理器的硬件平台上，编译线程和应用线程可以同时执行，两者互不干扰。此时，相对于应用线程，编译线程相当于在后台运行，故又被称为后台编译技术。后台编译技术有效掩盖了动态编译的编译延迟，对于提升虚拟机系统的性能非常重要。

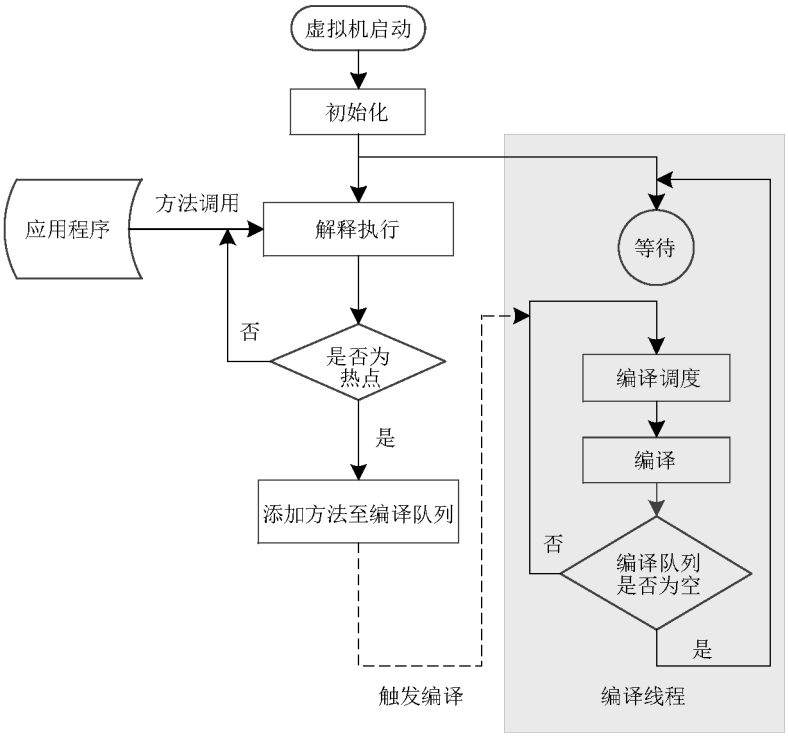


图 1.2: Java虚拟机动态编译的基本过程

1.3 Java虚拟机面临的挑战

除了对传统的平台无关性、安全性和高级语言特性[17]的支持外，新兴的应用模式

使得Java虚拟机的设计面临更为严峻的挑战，其性能往往不能满足实际系统的需求。当前，Java虚拟机的性能面临以下亟需解决的问题。

- 服务器端的海量数据处理问题。在服务器应用领域，如大规模ERP系统、电子商务网站和大型数据中心等，Java一直雄踞霸主的地位。随着云基础设施和大型数据中心服务的不断完善，服务器的访问量急剧增加，每天都有海量的数据需要处理[5]。服务器处理客户端请求的吞吐量和响应延迟直接决定了服务的质量，并最终影响服务提供商的利润。而传统的Java虚拟机主要基于中小规模的数据进行设计，其处理能力已接近饱和，不能适应服务器端日益增长的数据处理需求。因此，Java虚拟机需要不断提升性能，以提高服务器的吞吐量，并且缩短服务器的响应延迟。

- 移动终端的交互式体验问题。在移动终端领域，Java是仅次于Android和iOS的第三大开发平台，并且其市场占有率处于不断增长的态势。交互能力是目前移动终端最需要关注的问题，包括系统的启动性能和响应速度两个方面。对于目前广泛使用的智能手机、平板电脑等手持终端设备，启动性能和响应速度直接影响用户的使用体验[18][19]，在很大程度上决定了产品的竞争能力。Java虚拟机由于存在动态编译、动态加载与链接以及垃圾收集等运行时开销，启动性能和响应速度通常较差。故Java虚拟机在移动终端的交互能力亟需提升。

- 新兴架构处理器的适应性问题。近年来，除了经典的X86、ARM和Power等处理器外，新架构的处理器不断涌现，仅在我国就诞生了中科院计算所的龙芯处理器、江南56所的申威处理器、国防科大的飞腾处理器和华为的海思处理器等[20]。虽然Java是平台无关的，但Java虚拟机却是和平台密切相关的。然而，现有的Java虚拟机通常只针对X86等少数平台的处理器进行设计，对于新架构处理器的优化不足，导致Java虚拟机在新架构的处理器上并不能充分发挥硬件的处理能力。因此，为了充分发挥处理器的性能，Java虚拟机需要密切结合处理器的特性进行设计和优化，必要时可采取软硬件协同的方案进行定制。

- 多语言运行平台的效率问题。随着软件设计规模和复杂度的不断攀升，多语言联合编程逐渐成为主流[21]。例如，在实际项目中，应用的展示层可能使用JRuby实现，而中间件则采用Java语言编写，并且JRuby和Java均同时运行在Java虚拟机上。为了支持上述需求，Java虚拟机已经成为支持多种语言程序的通用运行时环境[22]，目前已经实现了对诸如Scala、JRuby、Clojure和Groovy等新生语言的支持。上述新生语言被越来越多地应用到客户端图形界面显示、人工智能处理、信息检索、数据库处理和控制台应用等交互式环境中，对系统的响应速度有着较高的要求。因此，为了更好地支持多语言的运行，Java虚拟机有必要进一步提升其对外界的快速响应能力。

综上所述，在服务器端，Java虚拟机需要提升性能以适应海量数据处理的需求，加快对服务请求的响应速度，以提高服务的吞吐量，并改善服务质量。而在移动终端领域，Java虚拟机亟需提升其启动性能和对外界的响应速度，以增强系统的交互能力。需要特别注意的是，Java虚拟机交互能力的提高对于服务器端的应用也是十分有益的。例如，较快的启动速度，可以缩短服务器从故障中自我恢复的周期，最大限度降低损失，增强服务器系统的可靠性。此外，将Java虚拟机的设计优化和龙芯等国产处理器结合起来，不仅可以更充分地发挥国产处理器的性能，还可以有效解决国产处理器普遍面临的软件缺乏问题，促进我国自主信息产业的发展壮大。因此，研究Java虚拟机的性能优化技术具有十分重要的理论意义和实际应用价值。

1.4 本文的研究内容和主要贡献

Java虚拟机性能问题是制约现代信息产业进一步发展的重要因素。本文以当前实际应用最广泛的HotSpot高性能Java虚拟机为基本框架，对Java虚拟机性能优化的关键技术进行了研究。针对动态编译的程序热点识别、动态编译调度和解释器指令分派等关键环节提出了一套系统的、通用的高性能Java虚拟机性能优化方案。本文的研究内容和主要贡献包括：

1. 基于贝叶斯估计的程序热点识别算法。

   热点识别是影响虚拟机性能的关键因素。本文首先设计了基于智能阈值的热点识别模型。智能阈值由方法的静态特征和系统当前的运行状态共同确定。智能阈值的引入旨在提高热点识别的灵活性以及对程序行为的自适应能力。在此基础上，本文进一步提出了一种基于贝叶斯估计的程序热点识别算法。通过贝叶斯估计，该算法实现了对程序热点的前瞻性识别。本文提出的热点识别算法可以显著降低热点识别的延迟，从而大幅降低热点方法的编译延迟，最终有效提升系统的性能。

2. 编译队列监视下的高性能动态编译调度算法。

   动态编译调度对虚拟机的性能有着十分重要的影响。本文首先研究了动态编译行为的主要特征，揭示了动态编译调度的优化目标。在此基础上，本文提出了一种编译队列监视下的高性能动态编译调度算法。该算法主要由收益代价分析的调度模型和自适应的编译队列监视机制组成。该算法在调度时综合考虑了方法的重要性和编译代价，可以加快系统的响应速度并降低编译队列的平均排队延迟。此外，在调度的同时进行编译队列监视，可以延迟编译队列中非活跃方法的编译，并且有选择性地将部分非活跃方法从编译队列中删除，从而有效降低系统的运行时开销。

3. 软硬件协同设计的解释器指令分派优化。

解释器指令分派一直是制约解释器性能最关键的因素。本文对解释器指令分派的开销进行了量化分析，指出对字节码本地指令序列入口地址的获取构成了指令分派的瓶颈。在此基础上，提出了一种软硬件协同设计的指令分派优化方案。该方案具体包括优化的指令分派表设计、软件优化的指令分派表访问技术和基于硬件支持的访存加速策略。在软件层面，通过将优化后指令分派表的起始地址缓存到一个可以高速访问的部件中，消除了优化前代价较高的入口地址加载操作。在硬件层面，通过对原有访存指令进行扩展，进一步提高了访存效率。本文的方案大幅降低了指令分派的开销，有效提升了解释器的性能。

上述关键技术对Java虚拟机的多个方面进行了优化，共同构成了本文提出的Java虚拟机性能优化的技术体系。上述工作已在龙芯平台的Java虚拟机上进行了验证，基于真实系统的性能评测数据表明，优化后Java虚拟机的性能获得了明显提升。本文所提出的一些优化思想和方法不仅适用于Java虚拟机，对其它虚拟机和应用程序的设计和优化也具有参考意义。

1.5　本文的组织结构

本文第一章首先介绍了Java的发展与应用，指出由于Java具有平台无关性和安全性等特征，Java在云计算和大数据时代获得了更广泛的应用。随后介绍了Java技术体系，重点展示了Java虚拟机执行引擎设计的基本原理。其次，揭示了当前Java虚拟机面临的诸多性能问题，并点明了本文研究的意义和重要性。最后，阐述了本文的研究内容、主要贡献和组织结构。

第二章首先介绍了Java虚拟机基本概念、结构和功能。其中，着重介绍了运行时数据区、执行引擎、类加载器、垃圾收集器和本地方法接口等Java虚拟机的核心部件。最后总结了Java虚拟机的性能优化技术，并重点展示了解释执行和动态编译的优化。

第三章主要介绍了本文的软硬件研究平台和测试程序。其中，硬件平台为龙芯3B1500处理器。软件平台主要为OpenJDK的HotSpot虚拟机。测试程序包括主流的SPECjvm98和DaCapo测试集。

第四章研究了基于贝叶斯估计的程序热点识别算法。首先设计了一个基于智能阈值的热点识别模型，并在此基础上提出了基于贝叶斯估计的程序热点识别算法。

第五章研究了编译队列监视下的高性能动态编译调度算法。首先分析了动态编译行为的特征，揭示了动态编译调度的优化目标。在此基础上提出了一种编译队列监视下的高性能动态编译调度算法。

第六章研究了软硬件协同设计的解释器指令分派方法。首先对解释器指令分派的开销进行了量化分析，指出了指令分派的瓶颈。在此基础上提出了一种软硬件协同设计的解释器指令分派优化技术。

第七章对全文进行了总结，并指出了未来Java虚拟机的研究方向。

第二章　Java虚拟机及其相关优化技术简介

2.1　引言

自20世纪中期虚拟机技术诞生以来，人们对虚拟机系统性能的优化从未停止。20世纪60年代的Lisp语言运行环境通常被认为是最早的虚拟机系统[23]。但早期的Lisp程序完全依赖解释器执行，并没有设计动态编译系统。20世纪70年代，卡内基梅隆大学的Hansen等人为Fortran语言设计并实现了著名的自适应优化系统，该系统通常被认为是动态编译的鼻祖[24]。20世纪80年代的Smalltalk虚拟机[25]和20世纪90年代初期的Self虚拟机[26]是虚拟机技术发展史上两个重要的里程碑。在Smalltalk系统中，动态编译技术变得更加成熟，提出了高速内联缓存技术(inline cache)和代码缓冲区(code cache)等动态编译系统设计的核心概念[25]。Self虚拟机继承了Smalltalk的优点，并进一步提出了一系列诸如多态的高速内联缓存技术(polymorphic inline cache)[27]，栈上替换技术(on-stack replacement，OSR)[28]和逆优化(deoptimization)[29]等现代虚拟机系统广泛使用的经典技术。1995年诞生的Java语言将虚拟机技术推向了巅峰。Java虚拟机不仅全面继承和发展了Smalltalk和Self等虚拟机设计的精髓，更重要的是将基于虚拟机技术的编程语言首次成功推向市场。

本章首先介绍Java虚拟机的基本概念、结构和功能。其中，着重介绍了运行时数据区、执行引擎、类加载器、垃圾收集器和本地方法接口等Java虚拟机的核心部件。随后，对经典的Java虚拟机性能优化技术进行了总结，重点展示了Java虚拟机中解释执行和动态编译相关的优化技术。

2.2　Java虚拟机基础

图2.1展示了Java虚拟机的核心模块，主要包括运行时数据区、类加载器、执行引擎以及垃圾收集器等。各个模块的主要功能为：

(1) 运行时数据区：用于存放虚拟机本身的重要数据结构、被加载的应用程序、运行时的堆栈以及编译生成的本地代码等。

(2) 类加载器：从本地或网络中加载应用程序到虚拟机的运行时数据区，并在加载的过程中负责对程序进行校验、链接和初始化等。

(3) 执行引擎：对被加载的应用程序进行模拟执行。

(4) 垃圾收集器：管理虚拟机的运行时数据区，负责内存分配和垃圾回收等。

(5) 本地方法接口：为虚拟机调用本地方法提供了统一的接口。

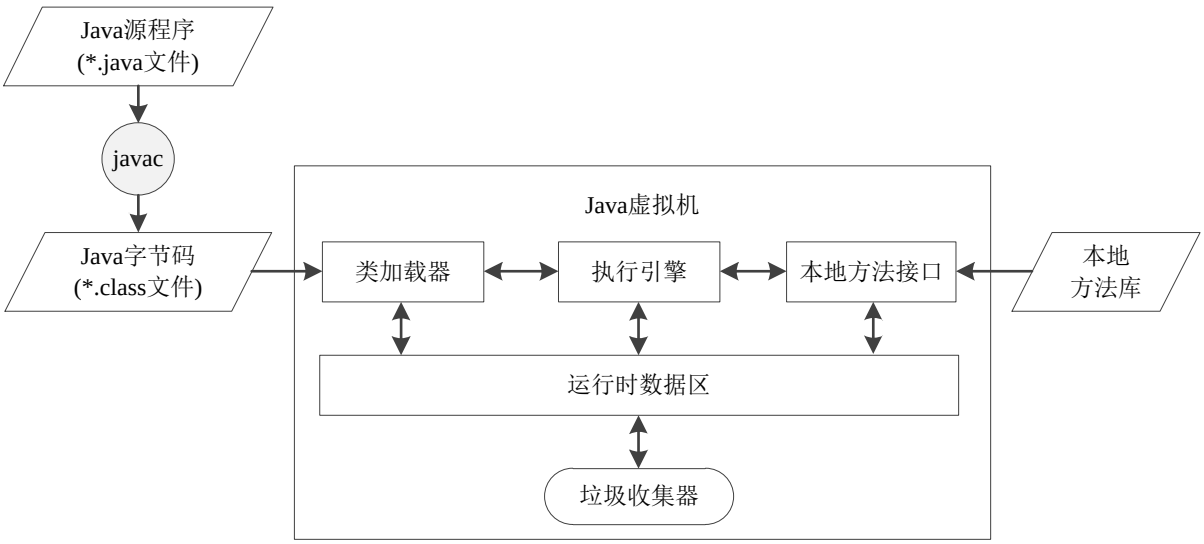


图 2.1: Java虚拟机核心模块

本节主要对上述虚拟机的核心部件进行介绍，以加深对Java虚拟机系统的认识和了解，为本文的研究工作奠定基础。

2.2.1 运行时数据区

运行时数据区是在程序运行时操作系统分配给Java虚拟机进行管理的内存空间。图2.2展示了Java虚拟机运行时数据区的结构。从图中可以看出，运行时数据区分为程序计数器、虚拟机栈、本地方法栈、堆、方法区和运行时常量池等6个组成部分。其中，程序计数器、虚拟机栈和本地方法栈为线程私有的数据结构，而堆、方法区和运行时常量池则被所有线程共享。运行时数据区各个组成部分的用途分别为：

- 程序计数器。主要用于指向执行线程将要执行的下一条字节码。当虚拟机执行Java方法时，程序计数器的值为指向下一条待执行的字节码的地址；而当虚拟机执行本地方法时，程序计数器的值是未定义的。由于Java虚拟机支持多线程并发执行，故程序计数器是线程私有的，生命周期和执行线程相同。

- 虚拟机栈。每个执行线程均对应一个虚拟机栈。虚拟机栈随线程的启动而创建，随线程的结束而消亡。虚拟机栈由栈帧组成。Java方法调用时，新的栈帧会被创建并压入执行线程对应的虚拟机栈中。当Java方法调用返回时，对应的栈帧从虚拟机栈中弹出。每个栈帧主要用于存储被调用方法的局部变量、操作数栈和常量池引用等相关信息。

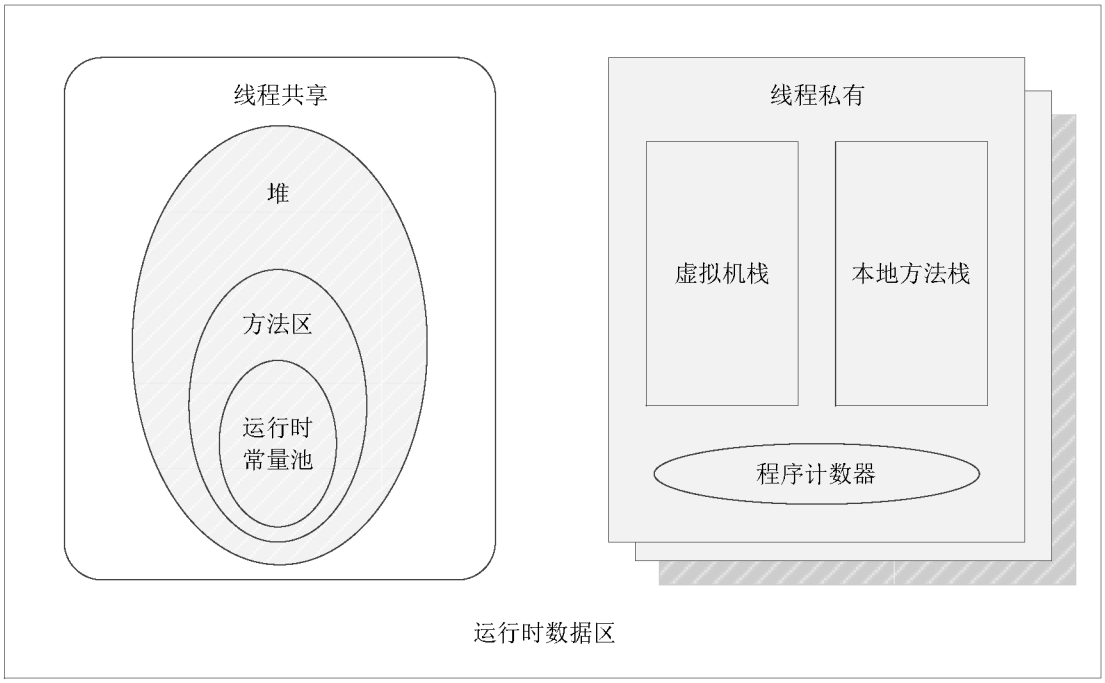


图 2.2: 运行时数据区

- 本地方法栈。本地方法栈是当虚拟机调用本地方法时创建的调用堆栈，其结构和用途与虚拟机栈类似。本地方法指除Java方法之外的其它方法，最常见的是用C和C++语言编写的方法。本地方法通过Java虚拟机的JNI接口进行调用。虚拟机会根据调用的语言类型，自动创建C或者C++的调用堆栈。

- 堆。堆是所有类的实例和数组对象分配的区域。在程序运行期间，所有线程共享一个堆空间。堆空间在虚拟机启动时创建，并当虚拟机销毁时才进行释放。堆空间中对象的分配和释放是由虚拟机的垃圾收集器自动管理的。堆在物理内存中不需要占用连续的内存空间。一般地，堆空间的大小可以根据程序运行的需求进行动态扩充或者紧缩。

- 方法区。方法区用于存储被加载的类信息，包括运行时常量池信息、属性和方法的元数据信息、类的静态变量、方法的字节码和即时编译器编译后的本地代码等数据。方法区类似于传统的C/C++等编译型语言在Unix系统进程中的正文段(text segment)数据区。虚拟机中所有的线程共享一个方法区，该区域在虚拟机启动时创建。虽然方法区在逻辑属于堆的一部分，但Java虚拟机规范中并未规定方法区在运行时数据区中的位置，也未对该区域的垃圾收集机制和编译后代码管理等作强制规定。因此，虚拟机厂商可以根据需要灵活实现。

- 运行时常量池。运行时常量池存储每个类或接口的class文件中constant_pool表中的数据信息，是程序运行时constant_pool表在虚拟机内部的表示。运行时常量池属于方法区的一部分，每个类对应的常量池信息在该类被虚拟机加载后创建并存放到方法区的运行时常量池中。运行时常量池的作用类似于传统编译型语言的符号表，被所有线程共享，在虚拟机的运行过程中扮演着十分重要的角色。常量池中存储了所有类或者接口中定义的常量信息以及相关属性和方法的地址。在对某个属性或者方法进行访问前，虚拟机需要查找并解析常量池中的信息，以获取所需的目标地址。

2.2.2 执行引擎

字节码通常不能直接在处理器上执行，Java程序必须借助Java虚拟机的执行引擎才能运行。Java虚拟机的执行引擎可分为解释器和即时编译器两类[12][30]。

解释器结构简单，易于构建，很早就用于各种高级语言的实现[31][32]。解释器解释执行本质上是用软件的方法模拟一个抽象CPU的执行过程，主要包括取指令、指令分派、取操作数和执行等四个步骤。由于存在取指令和指令分派等额外的运行时开销，解释器存在执行效率较低的缺点。但即便如此，解释器依然广泛应用于各种高级语言和虚拟机系统中，其原因在于：

(1) 解释器易于构建、移植和维护。由于字节码的语义都比较简明，故对其解释执行非常容易。同时，解释器可以完全使用C/C++等高级语言来实现，因此具备优良的可移植性。此外，从逻辑上看，解释器仅需实现由取指令、指令分派、取操作数和执行等步骤组成的核心循环，结构非常简单，很容易维护。

(2) 解释执行的速度不一定比动态编译后执行慢。研究表明，程序的执行时间分布符合局部性规律，即只有少量的方法被多次执行，大量方法被执行的次数很少。对于执行次数较少的方法，当动态编译的时间大于解释执行的时间时，解释执行会更快。

(3) 解释器常用于构建剖面分析器和调试器。现代高级语言虚拟机为了不断提升系统性能，通过剖面分析收集大量的运行时信息，以指导后期的编译优化[33]。所收集的运行时信息包括：分支指令跳转的频率、方法调用的频率和对象的类型分布等。上述信息的收集均要求以字节码为单位进行。由于解释器逐条对字节码进行解释，故很容易在解释器中实现对程序的剖面分析。同理，相对于即时编译器，在解释器中也更容易实现断点设置、单步调试以及查看和更改变量的值等程序的调试功能。

(4) 解释器可作为动态编译激进优化的逃生门。为了尽可能提升虚拟机的性能，即时编译器通常会基于收集到的剖面分析信息[34][35]进行假设，实施激进的优化。但

是，剖面分析过程中收集到的运行时信息只代表程序过去的行为。由于程序行为的易变性，激进优化依赖的假设条件可能不再成立。此时，虚拟机需要禁止对基于错误假设的编译后代码的执行，采用解释执行或者重新编译的方式继续运行。考虑到重新编译会产生较大的延迟，故激进优化假设失败后多以解释器执行为主。因此，解释器充当了激进优化的逃生门。

(5) 解释器消耗的内存资源少。由于不涉及动态编译，解释器所需的内存空间非常少。故解释器尤其适用于存储空间较为稀缺的单片机和嵌入式系统等领域。

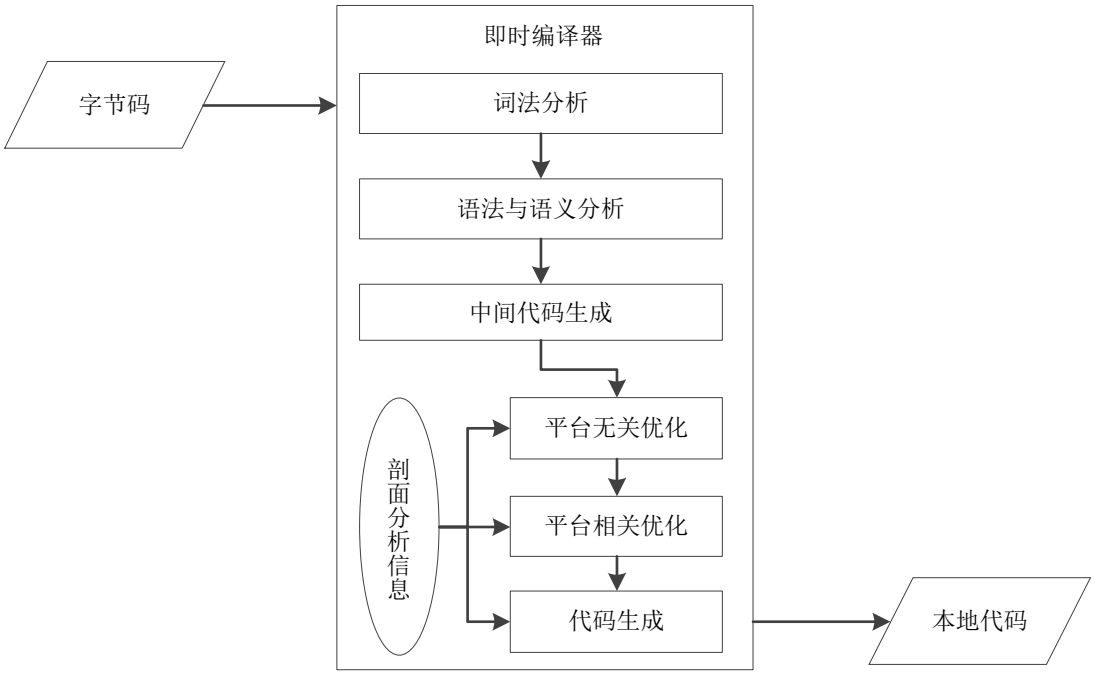


图 2.3: 即时编译的基本过程

对于频繁执行的方法，动态编译后执行性能更高。即时编译器将方法编译成本地处理器直接支持的机器指令序列，消除了解释执行中取指令和指令分派等软件模拟的开销。同时，编译期间还可以进行许多卓有成效的编译优化，故能生成高质量的本地代码。因此，动态编译后执行的速度通常高出解释执行10倍以上。

图2.3展示了即时编译的基本过程，包括：词法分析、语法与语义分析、中间代码生成、平台无关优化、平台相关优化和代码生成等步骤。从图中还可以看出，即时编译器的输入除了被编译的方法本身外，通常还包括该方法的剖面分析信息。剖面分析信息用于指导即时编译器对程序施加更有针对性的优化。

2.2.3 类加载器

Java虚拟机将来自本地磁盘或者网络的Class文件加载到内存，并且对被加载的信息进行校验、解析和初始化，最终在虚拟机的运行时数据区中构造出可以直接被Java虚拟机使用的Java类型，这个过程称为虚拟机的类加载机制[6]。Java语言可以根据需要在运行时动态加载相应的模块，并且经过动态链接后执行，因此具备很强的动态扩展性的特征。同时，可供加载的模块也不仅限于本地磁盘，还可以来自网络传输、集群中的数据库以及运行时动态生成等。Java语言主要面向网络应用，由于经常需要从网络服务器中获取Java应用模块并执行，Java虚拟机很早就提供了对网络移动性需求的支持。Java应用的动态扩展性和网络移动性均以类加载器为基础实现。类加载器最早针对Java Applet应用而设计。经过近20年的发展，从早期的Applet、JSP到现代的Web容器和更复杂的OSGi技术，类加载器的应用更为广泛，其基础性的地位也越来越凸显。

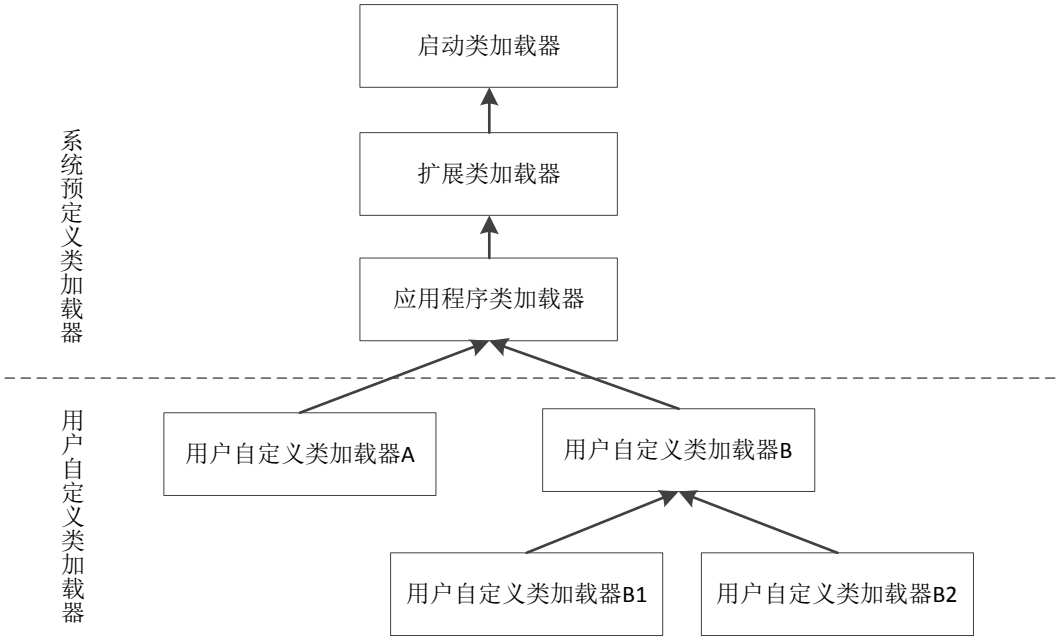


图 2.4: 类加载器的层次结构图

类加载器的一个重要特征是采用了层次化的组织方式。图2.4中展示了虚拟机中类加载器的层次化组织模式图。从图中可以看出，类加载器主要分为以下四大类：

(1) 启动类加载器(bootstrap class loader): 用于加载Java类库中最基础、最核心的类。启动类加载器由本地的C/C++代码实现，属于Java虚拟机的一部分，在虚拟机启动时创建。

(2) 扩展类加载器(extension class loader): 用于加载Java类库中的扩展类。扩展类加载器由名为sun.misc.Launcher$ExtClassLoader的Java类实现。扩展类加载器还负责大量安全类相关模块的加载。

(3) 应用程序类加载器(application class loader): 用于加载第三方类库或者用户自定义的类。应用程序类加载器由名为sun.misc.Launcher$AppClassLoader的Java类实现。扩展类加载器是应用程序中默认的类加载器，又被称为系统类加载器。

(4) 用户自定义加载器(user-defined class loader): 用于实现具有特殊加载需求的类加载器。例如，从网络中指定位置加载、动态生成代码的加载以及加密后代码的加载等都需要用户自定义类加载器。

上述类加载器除了启动类加载器外，其余均使用Java代码实现，并且均要求继承java.lang.ClassLoader这个抽象类。

基于上述层次化的组织方式，类加载器以代理模式进行类的加载。所谓类加载的代理模式，是指当前的类加载器在进行类加载前，先会请求其父类加载器进行类加载。只有当父类加载器不能完成加载操作时，当前的类加载器才亲自实施类加载操作。所有的类加载器按上述方式进行类加载操作。为了支持上述代理模式，要求为每个类加载器指定一个父类加载器。如图2.4所示，用户自定义加载器的父类加载器通常为应用程序类加载器，而应用程序类加载器的父类加载器则为扩展类加载器，扩展类加载器的父类加载器为启动类加载器。启动类加载器比较特殊，由于它处于层次结构的最顶层，其父类加载器为空。类加载的代理模式对于Java类库中核心类的安全非常重要。例如，对于Java的基础类java.lang.Object，该类是所有Java类的基类，任何Java应用都必须加载该类。通过类加载的代理模式，java.lang.Object类只能由启动类加载器从Java核心类库中加载，从而避免了该类被用户恶意定义的同名类所覆盖。因此，类加载的代理模式对于虚拟机安全非常重要。

2.2.4　垃圾收集器

自动内存管理是Java的一大特色。C/C++等传统编程语言要求程序员负责内存的分配和释放。当系统的内存未被及时、彻底地释放时，就会引发内存泄露问题，严重时会导致系统瘫痪或者崩溃。Java虚拟机的自动内存管理机制使得程序员编写应用程序时，不再需要考虑内存的申请释放问题，极大减轻了编程人员的负担，既提高了工作效率，又保证了程序的可靠性。为了实现自动内存管理机制，Java虚拟机提供了强大的垃圾收集功能。

垃圾收集[36]，即识别内存中不再使用的对象，并将其清除以释放内存空间的过程。垃圾收集需要解决的两大关键问题是：1)识别不再使用的垃圾对象；2)清除垃圾对

象。虚拟机研究人员提出了多种垃圾收集算法来解决上述问题[37–40]。根据实现原理的不同，主流的垃圾收集算法可分为：

(1) 引用计数算法(reference counting collection)[41][42]：对对象的引用进行计数，并在垃圾收集时清除引用计数为0的对象。引用计数法的设计思路简单，实现也非常容易，是最古老的垃圾收集算法。在微软的COM组件技术和Adobe的ActionScript3中便使用了引用计数的垃圾收集算法。但由于引用计数法难以识别存在循环引用关系的垃圾对象，故该算法很少在Java虚拟机中使用。

(2) 标记-清除算法(mark-sweep collection)[43][44]：分为标记和清除两个阶段。标记阶段标记所有正在使用的对象，清扫阶段清理所有未被标记的对象，即垃圾对象。标记阶段，通常以根集中的对象作为起始顶点，并以对象间的引用关系作为边进行遍历。根集中的对象通常包括：1)虚拟机栈中引用的对象；2)方法区中静态属性引用的对象；3)方法区中常量引用的对象；4)本地方法栈中引用的对象。标记-清除算法在垃圾回收后，所有活跃对象在内存中并不连续分布。故标记-清除算法很容易导致内存碎片化的问题。此外，标记-清除算法还存在标记和清除阶段效率低下的问题。

(3) 复制算法(copying collection)[45][46]：复制算法将内存空间分为两半，每次仅使用一半的存储空间，另一半备用。垃圾收集时，将当前使用区域中活跃的对象拷贝到另一半区域，随后释放掉当前使用区域的内存空间，之后交换两个存储区域的角色，从而完成一次垃圾收集的过程。复制算法可以解决内存碎片化问题，其缺点是可用的内存空间仅有实际空间的一半。当系统中垃圾对象的比例较高时，复制算法十分高效。反之，则代价较高。

(4) 标记-压缩算法(mark-compact collection)[47][48]：在标记-清除算法基础上提出的一种垃圾收集算法。和标记-清除类似，其基本过程依然可分为标记和清除两个阶段。但在标记阶段，标记-紧缩算法不仅负责标记活跃的对象，同时还将活跃的对象压缩到内存的一端。标记完成后，清除所有被压缩对象之外的存储空间。标记-压缩算法避免了标记-清除算法的内存碎片化问题。同时，该算法不需要将内存分为两半，并降低了对象复制的开销。故标记-压缩算法是一种性价比较高的垃圾收集算法。

(5) 分代收集算法(generational collection)[49][50]：研究表明，程序中不同的对象往往具有不同的生命周期。因此，可以将内存区域划分为若干代，分别用于存放具有不同生命周期的对象。对每个代分别采用不同的垃圾收集算法，以提高垃圾收集的效率。现代的商业虚拟机中普遍采用分代收集算法。通常将虚拟机的内存区域分为新生代和老年代。初次分配的对象通常位于新生代中。新生代中经历过多次

垃圾收集后依然存活的对象会被移动到老年代中。由于新生代对象的生命周期大多都很短，垃圾收集时仅有少量活跃的对象，故通常对新生代采用复制的垃圾收集算法。而老年代中对象的存活率高，故通常在老年代中使用标记-压缩的垃圾收集算法。

上述垃圾收集算法，可用于实现各具特色的垃圾收集器。按垃圾收集的线程数分类，垃圾收集器可分为串行垃圾收集器和并行垃圾收集器。串行垃圾收集器每次仅使用一个线程进行垃圾回收，而并行垃圾收集器可以同时使用多个线程进行垃圾收集。在现代多核处理器中，并行垃圾收集器可以充分发挥系统的并发能力，有效缩短垃圾收集的停顿时间。而按工作模式分，垃圾收集器又可分为并发式垃圾收集器和独占式垃圾收集器。并发式垃圾收集器在进行垃圾回收时应用程序无需暂停，故能有效缩短程序的停顿时间。独占式垃圾收集器在工作期间必须停止应用程序的执行，直到垃圾收集完全结束。故独占式垃圾收集器很容易导致较大的系统停顿时间。现代商业的垃圾收集器及其主要特性如表2.1所示。

表 2.1: 主流的垃圾收集器

名称	工作区域	主要特性
Serial收集器	新生代	标记-压缩算法 串行、独占式收集器
ParNew收集器	新生代	标记-压缩算法 并行、独占式收集器
Parallel Scavenge收集器	新生代	复制算法 并行、独占式收集器
Serial Old收集器	老年代	标记-压缩算法 串行、独占式收集器
Parallel Old收集器	老年代	标记-压缩算法 并行、独占式收集器
CMS收集器	老年代	标记-清除算法 并行、并发式收集器
G1收集器	新生代和老年代	G1收集算法 并行、并发式收集器

2.2.5 本地方法接口

Java是一种比C/C++抽象程度更高的高级编程语言。更高的抽象层次给程序员的编程带来了更多的便利，但同时也使得Java在性能和与底层交互等方面无法与C/C++进行媲美。为了弥补上述缺陷，Java虚拟机引入了本地方法接口技术[51][6]，其主要设计意图是：

- 改善Java应用的性能。通过直接调用本地C/C++的库和程序，可以进一步提升Java的性能。
- 增强Java对底层的处理能力。一些和底层硬件及操作系统密切相关的功能必须通过本地方法调用才能实现。
- 复用本地既有的系统。通过本地方法接口，可以兼容本地长期稳定运行的系统，降低开发成本和风险。

本地方法接口提供一个标准的方式使得Java程序可以和C/C++以及汇编等语言实现的本地库和应用程序进行交互。目前，本地方法接口已形成了一个统一的规范，用户只需编写一次本地库和应用程序，就能够在所有支持本地方法调用的虚拟机上运行。

2.3 Java虚拟机性能优化技术

本节主要介绍Java虚拟机的性能优化技术。执行引擎是影响Java虚拟机性能最关键的部件，故本节重点介绍了Java虚拟机执行引擎的优化，包括解释执行和动态编译的优化。同时，本节对虚拟机其它经典的优化技术也做了简介。

2.3.1 解释执行的优化技术

2.3.1.1 指令分派优化技术

字节码是解释执行的基本单位。每条字节码由操作码和操作数构成。解释执行包括：取指令、指令分派、取操作数和执行等四个基本步骤，其具体含义如下：

- 取指令：获取下一个即将执行的字节码。
- 指令分派：根据该字节码的操作码，跳转到实现其语义的处理例程的起始地址。
- 取操作数：获取解释执行该字节码所需的操作数。
- 执行：执行该字节码的处理例程。

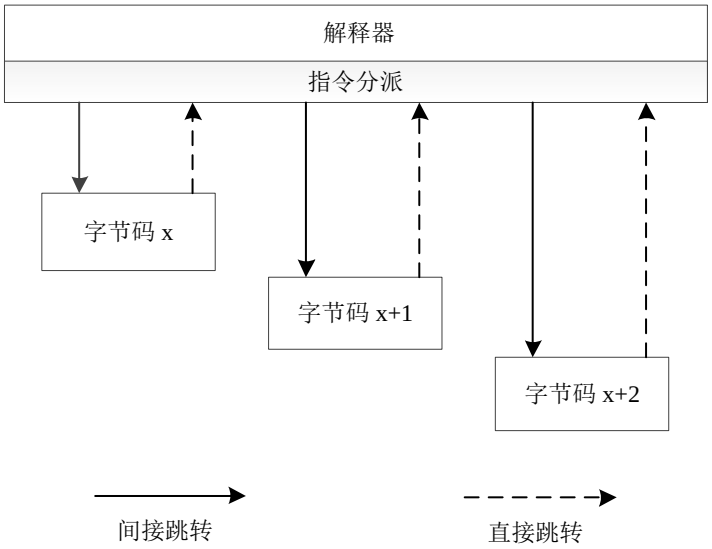


图 2.5: Switch结构的指令分派

指令分派[52][53]是当前字节码执行完毕后过渡到下一个字节码继续执行的桥梁，通常构成解释执行的性能瓶颈。对于大多数的字节码，其语义功能都非常简单，如加减法操作、类型转换和操作数栈访问等，仅需1∼2条机器指令即可实现上述操作。而指令分派功能相对较为复杂，通常需要十多条机器指令才能实现。此外，指令分派中还至少包括一条执行开销较大的间接转移指令。因此，指令分派在解释执行期间消耗了较大比例的程序执行时间[54][55]。

Switch结构是最简单的指令分派的实现模式[56]。Switch结构的指令分派由一个主循环和位于主循环内的switch语句构成。switch语句的case分支实现了所有字节码的语义功能。图2.5展示了Switch结构的解释器进行指令分派的示意图。

当一个字节码对应的case分支执行完毕后，程序的控制流会通过一个直接跳转指令返回到核心循环。然后，在switch语句中根据即将执行的字节码间接跳转到相应的case分支继续解释执行。由于所有的字节码均需要通过switch语句的间接转移指令跳转至对应的case分支，故Switch结构的指令分派属于集中式的指令分派模式。Switch结构在指令分派时至少进行两次跳转(直接跳转和间接跳转各一次)，故指令分派的效率较低。

为了提高解释器指令分派的效率，Bell提出了线索化的指令分派技术[8]。其核心思想是在每个字节码的处理例程的末尾均添加独立实现指令分派功能的模块。当前字节码执行完毕后，立即获取下一个字节码，并根据所获取的字节码，从当前字节码的处理例程中直接跳转至下一个字节码的处理例程。线索化指令分派的具体执行过程如图2.6所示。

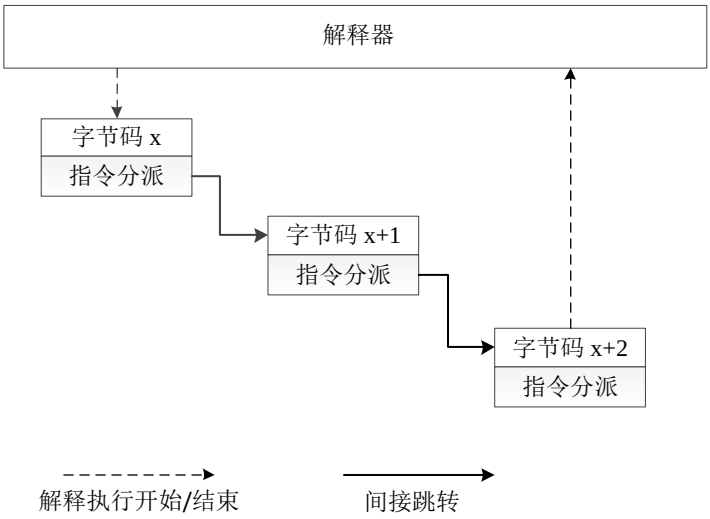


图 2.6: 线索化结构的指令分派

从图2.6中可以看出，每个字节码执行完毕后，可以立即跳转至下一个字节码的处理例程继续执行。所有字节码的解释执行如同被一条细线索引起来，故称为线索化的指令分派。线索化指令分派在所有字节码处理例程的末尾进行，故属于分布式的指令分派方法。相对于Switch结构的指令分派，线索化指令分派的优势为：

(1) 消除了冗余的跳转操作。Switch结构的指令分派需要执行两次跳转，而线索化的指令分派仅需一次跳转即可。

(2) 消除了冗余的边界检查操作。switch语句在编译后通常会进行边界检查，即对待分派的字节码的操作码进行边界检查。但虚拟机在类加载阶段已经对方法中所有字节码进行了校验，可保证字节码的安全性，故switch语句的边界检查是冗余的。而线索化指令分派则消除了上述冗余的检查操作。

(3) 有助于提升处理器对分支指令的执行效率。Switch结构指令分派采用了集中式的指令分派方法，其中所包含的间接跳转指令的跳转目标为所有可能的case分支，跳转目标很多。对于采用BTB(branch target buffer, BTB)表进行间接转移猜测的处理器而言，过多的历史跳转目标很可能降低处理器对间接转移指令跳转猜测的正确率，从而影响跳转指令的执行效率。而线索化指令分派采用分布式的指令分派技术，每个字节码的处理例程单独进行指令分派，其中的跳转指令可能的跳转目标大幅减少。对于跳转目标较少的跳转指令，基于BTB表的分支预测器的命中率往往更高，故能有效提升处理器对分支指令的执行效率。

2.3.1.2　栈顶缓存优化技术

除了指令分派的瓶颈外，解释执行过程中频繁的访存操作也是导致解释器性能低下的重要原因。Ertl和Gregg对解释器进行深入分析和研究，发现在解释执行的过程中，访存指令占了相当大的比例[56]。其中，load类指令约占30%，而store类指令约占15%。进一步分析知，上述访存指令大多数来源于对解释器操作数栈的访问。为使解释器能够适应不同架构的计算机系统，通常基于栈式体系结构来设计解释器[10]。对于基于栈式体系结构设计的解释器，运算时所需的操作数均从操作数栈中获取，计算的结果也需存入操作数栈。因此，解释执行时，解释器的操作数栈会被频繁地访问，导致解释执行中的访存操作十分频繁。

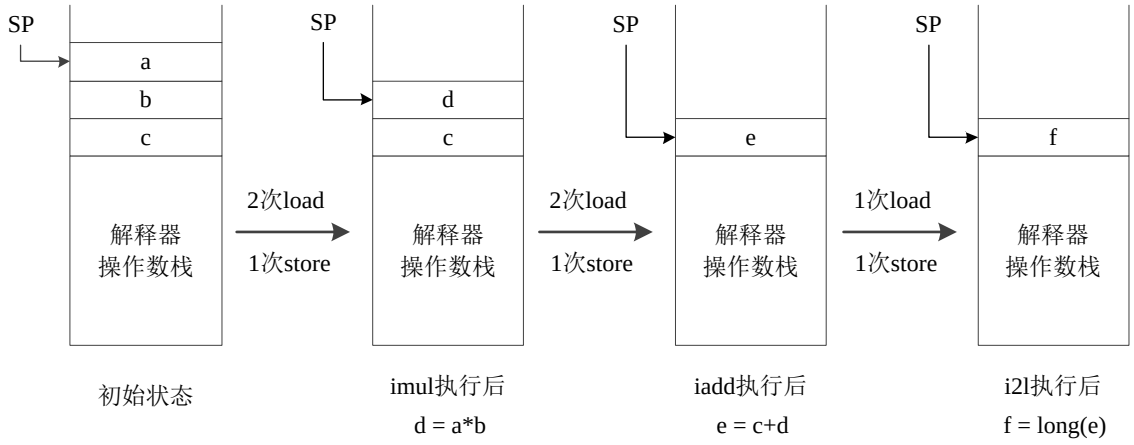


图 2.7: 解释器操作数栈的变化

图2.7展示了字节码序列imul, iadd和i2l的解释执行过程。其中，SP表示指向当前操作数栈栈顶的指针。imul和iadd为分别表示整数乘法和加法的字节码，两者均需要通过执行两条load指令从操作数栈中获取两个操作数，并将计算的结果通过执行一条store指令保存到操作数栈中。i2l为表示将一个整数转换为一个长整型的字节码，需要通过执行一条load指令从操作数栈中获取一个操作数，并将转换的结果通过执行一条store指令保存到操作数栈中。从图2.7中可以看出，上述三个字节码解释执行时，解释器总共需要执行5条load和3条store指令。频繁的访存操作严重影响解释执行的效率。

为了减少解释器的访存次数，提高解释执行的效率，Etrl提出了著名的栈顶缓存技术[57]。其核心思想是：将解释器操作数栈顶的一个或多个元素缓存到通用寄存器中，通过对寄存器的访问代替(或部分代替)原来对操作数栈的访问。图2.8展示了基于栈顶缓存技术，解释器对字节码序列imul, iadd和i2l的执行过程。在图2.8中，解释器操作数栈的栈顶最顶端的元素被缓存到了一个通用寄存器中。从图2.8中可以看出，上述三个

字节码在执行期间，解释器仅需要执行2条load和0条store指令。相对于优化前(图2.7)，栈顶缓存技术减少了3条load和3条store指令的执行。访存次数的减少也降低了应用程序对系统访存带宽的要求。故栈顶缓存技术能够有效地提升解释执行的性能。

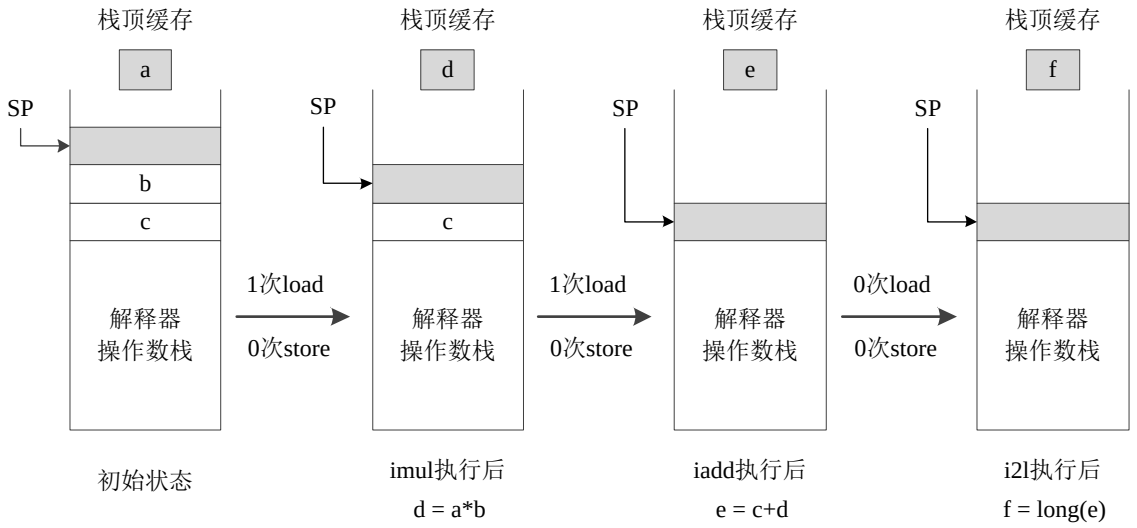


图 2.8: 采用栈顶缓存优化，解释器操作数栈的变化

2.3.1.3 指令集增强的优化技术

处理器以机器指令为基本单位执行应用程序。虽然一条乘法指令和一条加法指令的组合与单独一条乘加指令语义相同，但对于处理器而言，后者的执行效率显然更高。因此，指令系统的设计对于处理器执行效率的优化具有重要的意义。同理，解释器以字节码为单位对应用程序进行解释执行，字节码的设计也对解释执行的效率有着重大的影响。为了进一步发掘解释执行潜在的性能，研究人员从指令集扩充的角度提出了多种优化方法，主要包括：

- 特化指令技术[9]。其基本原理是根据字节码的语义及频繁出现的操作数定义新的功能等价的字节码。新定义的字节码的语义为已有字节码语义的特例，故称为指令特化技术。例如，字节码iload表示从解释器的局部变量数组中加载一个整型变量的值到解释器的操作数栈顶。字节码iload在编码时，除了iload的操作码外，其后包括一个整型的常量操作数，记为i，用于对局部变量数组进行索引。iload在解释执行时，需要单独进行一次内存加载操作才能获得常量操作数i。同时，在class文件中，iload字节码也需要两个字节的存储空间。然而，研究发现，iload字节码中常量操作数i的取值为0、1、2和3的频率非常高。为了提

高解释执行的效率，虚拟机规范中特别定义了仅由操作码组成的字节码iload_0，iload_1，iload_2和iload_3，其语义功能分别等价于原有iload字节码中常量操作数分别为0、1、2和3的情形。相对于原有的iload字节码，虚拟机在执行iload_X(X = 0,1,2,3)时，无需加载常量操作数。同时，每个特化的iload_X(X = 0,1,2,3)字节码在存储时仅需要一个字节的存储空间，使得应用程序更加紧凑。此外，特化的字节码还可以促进后期的编译优化，从而更有效地提升虚拟机性能。

- 超级指令技术[58]。其基本原理是从连续的多条字节码中定义新的、功能等价的字节码。新定义字节码的语义和原有字节码的语义没有重复，并且是原有多个字节码的叠加，因此被称为超级指令。例如，在应用程序中可能会观察到大量imul和iadd字节码连续出现的情形。据此可以定义一条新的字节码imul-iadd，用于实现imul和iadd字节码两条字节码的功能。超级指令技术最大的好处是可以彻底消除原有字节码之间指令分派的开销。同时，超级指令技术还可以消除原有字节码之间对中间结果的存取操作。此外，超级指令技术也使得应用程序更加紧凑，以减小存储和传输的开销。

- 冗余指令技术[9]。其基本原理是对部分字节码定义语义功能相同的、冗余的字节码，以提升处理器对指令分派中间接转移猜测的正确率，从而加快解释执行的速度。现有的高性能解释器通常采用线索化的指令分派技术，在每个字节码处理例程的末尾均包含用于指令分派的模块。由于每个字节码后面可能出现许多不同的字节码，故指令分派的间接转移指令存在多个跳转目标，客观上增加了处理器进行转移猜测的难度。冗余指令技术可以减少指令分派中间接转移指令的跳转目标数量，从而降低处理器转移猜测的错误率。例如，对于由imul、iadd、imul和isub组成的字节码序列，imul进行指令分派时，间接转移指令存在多个可能的跳转目标(imul执行后，第一次跳转至iadd，第二次则跳转到了isub)。如果定义一个和imul语义功能完全一样的冗余指令imul_rep，并将原来的字节码序列替换为imul、iadd、imul_rep和isub，则指令分派时所有间接转移指令的跳转目标都是唯一的。此时，处理器中的分支预测算法更容易正确预测指令分派中间接转移指令的跳转方向。因此，冗余指令技术可以在一定程度上加快指令分派的过程，从而提升解释器的性能。

然而，上述优化技术一个共同的缺陷在于其可供优化的空间受到字节码中操作码编码域长度的限制。现有Java虚拟机规范中定义的标准字节码已经超过了200个，故实际能够提供的特化指令、超级指令和冗余指令的数目十分有限。此外，上述优化和具体应用程序的字节码组成密切相关，并且对输入也较为敏感，故其优化效果受到限制。

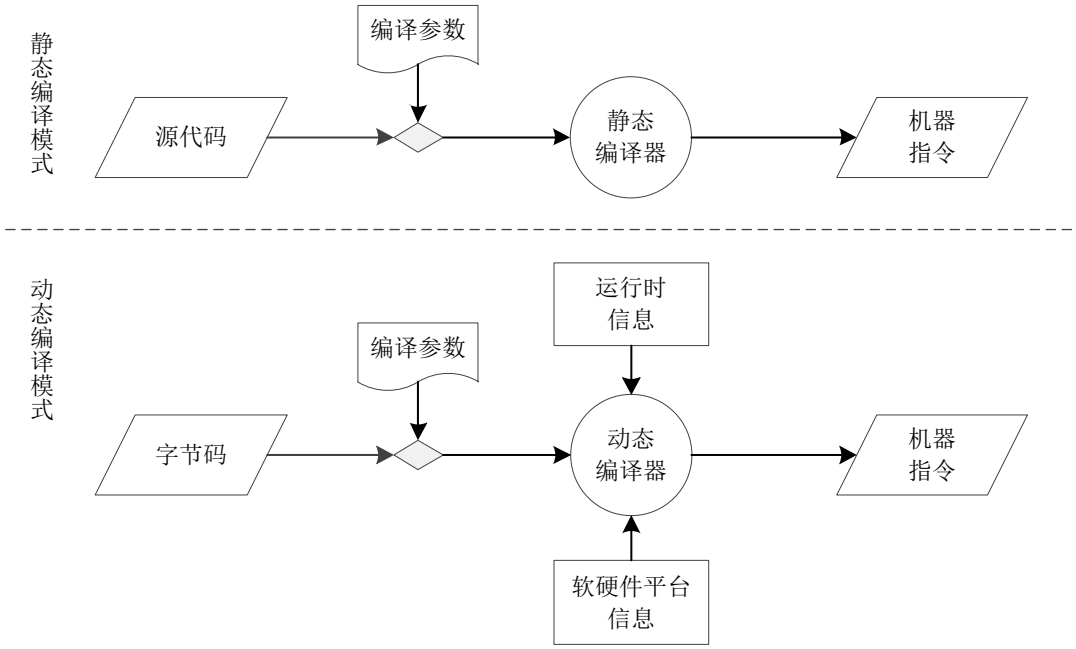


图 2.9: 静态与动态的编译模式

2.3.2 动态编译的优化技术

动态编译是决定虚拟机性能的关键。图2.9对比了C/C++等静态编译语言和Java等动态编译语言的编译模式。从图中可以看出，对于传统的静态编译模式，编译后生成的本地机器代码仅由源代码和编译参数决定。而动态编译的结果不仅取决于源代码和编译参数，同时还受到程序的行为和软硬件平台等因素的影响。和静态编译相比，动态编译特有的优势为：

- 对程序的行为更加适应[12]。动态编译可以根据程序的行为，自适应地决定编译哪些方法以及如何进行编译。动态编译通常只编译程序中频繁执行的方法。并且在编译期间，会根据所收集到的诸如对象类型分布、分支指令的跳转次数和方法调用次数等运行时信息进行深度优化，以更好地适应程序的行为。

- 对软硬件平台更加亲和[15]。动态编译会根据当前的操作系统和处理器等软硬件平台信息施加更有针对性地优化。例如，虚拟机会根据当前的内存大小来决定代码缓冲区和堆空间的大小，以及是否启用对象的压缩存储机制等。同时，即时编译器还会自动根据当前处理器的架构特征来决定是否生成向量指令、多媒体指令和其它特殊指令等。动态编译的上述亲和性可以最大限度地发挥本地平台的性能。

- 对程序的优化更加激进[59][60]。动态编译可以根据所收集到的运行时信息进行假设，对程序施加更为激进的优化。例如，可以假设程序中某个虚方法调用的目标固定为某个方法，进而实施激进的方法内联优化。由于该假设存在失效的风险，虚拟机中设计了逆优化[29]等补救措施以保证程序执行的正确性。激进优化是动态编译区别于静态编译最大的特征，可以将动态编译的性能发挥到极致。

本节主要介绍动态编译的性能优化技术，包括：程序热点识别技术、动态编译调度技术、高速内联缓存技术、激进的方法内联技术和部分编译技术等。

2.3.2.1　程序热点识别技术

虚拟机诞生初期，执行引擎主要由解释器构成，运行效率很低。为了提高虚拟机的性能，人们引入了即时编译器。由于编译后程序执行的效率通常比解释执行高出10倍以上，故程序中被编译的部分决定了系统的性能。为了尽可能多地执行编译后生成的本地代码，IBM公司的Arnold等人[61]和Intel公司的Cierniak等人[62]在2000年提出了Compile-only的策略，即程序中所有编译单元(函数、方法和踪迹等)在首次执行时就会被编译，以避免效率低下的解释执行。但是，该方案会导致动态编译的开销急剧增大，严重降低系统的性能。此外，过度的编译也使得系统中本地代码存储的压力明显增加。

为了降低动态编译的开销，现代高性能虚拟机通常仅编译程序中频繁执行的、对性能影响较大的部分，称为程序热点。程序热点识别决定了应用程序中的编译对象。因此程序热点识别是影响系统性能的关键。目前，主流的虚拟机均采用阈值法来识别程序中的热点。所谓阈值法，指当程序中编译单元总的执行次数超过某个阈值时，则被判定为程序热点。具体分为计数[15]和采样[61]两种形式。对于基于计数的热点识别，虚拟机对程序中编译单元的执行次数进行统计，当该编译单元的执行次数超过阈值时，则被识别为程序热点。而对于基于采样的热点识别，虚拟机周期性地对线程的调用堆栈进行采样，当发现某个编译单元的出现频率超过阈值时，则判定为程序热点。由于执行次数或采样频率需要一个不断累积的过程才能超过热点识别阈值，故上述方案存在热点识别延迟较大的问题。

为了降低热点识别的延迟，加州大学圣地亚哥分校的Krintz和Calder在2001年提出了基于注解信息(annotations)的程序热点识别技术[63]。该技术需要在线下收集大量有关程序运行期间的数据，并根据所获得的信息对源程序进行标注。虚拟机通过识别程序中预先标注的信息，能够提前发现并优化程序中的热点。该方案的缺陷在于必须提前运行应用程序以收集大量的运行时信息，并且还要求对程序代码本身进行注解。同时，相应的注解信息也仅适用于特定的应用程序，并且对程序的输入十分敏感，因而不具备通用性。

近年来，随着Android虚拟机在智能手机等移动终端设备上的广泛应用，热点识别延迟对系统性能的影响也越来越突出。2012年，LG公司的Lim和Parambil等人提出了利用静态编译技术对应用程序进行预编译的优化思想，以彻底消除热点识别延迟和动态编译开销对系统性能的影响[64]。该方案虽然能够避免热点识别导致的编译延迟和动态编译产生的运行时开销，却丧失了根据系统的运行时信息对程序进行深度优化的机会，故最终生成的本地代码质量往往低于动态编译的结果。因此，该方案在现代高性能Java虚拟机中并没有被采纳。

2.3.2.2 动态编译调度技术

动态编译调度主要解决程序热点的编译时机问题，即对于识别出的程序热点，如何确定它们的编译次序。编译顺序直接决定了程序热点在编译队列中的排队延迟，对于系统性能，尤其是系统的启动性能和对外界的响应速度具有十分重要的影响。故降低程序热点在编译队列中的排队延迟是动态编译调度优化的重要目标。

早期，程序热点主要按照被识别的先后顺序进行编译。程序热点在编译队列中排队延迟的降低主要依靠提高动态编译的吞吐量来实现。后台编译技术[65]是一种提高编译吞吐量的有效方法，该技术最早由美国华盛顿大学的Plezbert和Cytron于1997年正式提出。其核心思想是将编译功能从应用程序的执行线程中分离，构成独立的编译线程。应用程序执行线程触发编译请求，编译线程接收到编译请求后独立地在后台进行编译，因此编译任务不会阻塞执行线程的运行。随着多核处理器技术的发展，后台编译通常被设计为多个编译线程并行编译的模式，从而大幅提升动态编译的吞吐量。上述技术已广泛应用于HotSpot等高性能Java虚拟机中。在此基础上，APM公司的Harris提出了一种朴素的优化技术，其主要思想是提升编译线程的优先级，使得编译线程优先被操作系统调度[66]。2007年，IBM公司的Kulkarni等人还规定了在多线程环境下编译线程对CPU占用率的下限，优先满足编译线程对系统资源的需求[30]。上述编译线程调度的优化方案可以加快动态编译的速率，从而提升动态编译的吞吐量。但是，由于系统资源总是被优先分配给编译线程，当编译负荷较大(如系统启动或程序行为突变)时，应用程序本身的执行将受到抑制。此外，该方法还要求操作系统内核提供特定的线程调度支持，缺乏通用性，因而其适用范围受到限制。

近年来，研究发现程序热点识别的顺序并不一定是最佳的编译顺序。在动态编译调度过程中合理地安排编译次序，可以有效提升虚拟机的性能。Kulkarni等人的研究成果表明，对性能影响较大的方法在编译队列中排队延迟过大是导致虚拟机性能低下的重要原因，并且进一步指出通过动态编译调度合理优化编译的先后顺序是提高虚拟机性能的有效途径[13][67]。至此，动态编译调度的优化目标重点转向降低对性能影响较大的方法在编译队列中的排队延迟。根据上述思想，Stadler等人提出了基于方法运行速度的编译调度算法。该算法在调度期间实时计算方法的运行速度，优先编译运行速度较快的方法[68]。然而，上述方案仅关注了对性能影响较大的方法的排队延迟，却

忽略了对编译队列平均排队延迟的优化，故不能充分发挥动态编译调度的优化潜能。2014年，美国威廉玛丽学院的Ding等人首次从理论层面证明动态编译调度是一个NP-完全问题[69]，并通过实验证明了动态编译调度对虚拟机性能提升拥有巨大的潜力和空间。目前，动态编译调度已成为虚拟机优化关注的焦点之一。然而，由于缺乏对动态编译行为关键特征的深入剖析，动态编译调度依然缺乏一个较为全面系统的优化目标，因此该方向的研究仍需不断深入。

2.3.2.3　高速内联缓存技术

Java方法分为类方法和实例方法两类。实例方法调用时需要有一个实例对象，而类方法调用无需实例化，仅需指定类名即可。类方法使用静态绑定技术确定最终调用的目标方法，因此也被称为静态方法。而实例方法采用动态绑定技术，在运行时根据实例对象的具体类型确定最终调用的目标方法，故也被称为虚方法。相对于面向过程的语言，在Java等面向对象语言中，虚方法调用尤其频繁。与此同时，虚方法调用目标动态解析的开销很大，为虚拟机即时编译器的设计带来了巨大的挑战。

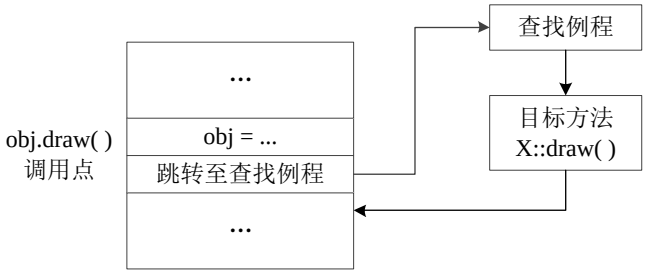


图 2.10: 传统的虚方法调用解析过程

图2.10展示了对虚方法调用进行动态解析的基本原理。假设程序中有一个虚方法调用点obj.draw()，其中obj对象的类型可能是Point、Circle和Rectangle中的一个。如图2.10所示，即时编译器在编译时会生成跳转到一个通用的虚方法调用查找例程的跳转指令。在虚方法调用查找例程中，虚拟机会根据对象obj的具体类型查找正确的目标方法，并跳转到该目标方法继续执行。由于虚方法调用查找例程的查找开销很大，故虚方法调用的开销很大，严重影响了虚拟机的性能。

为了加速系统对虚方法调用的动态解析速度，研究人员提出了虚方法调用的高速内联缓存技术[70]。该技术最早诞生于20世纪80年代设计的Smalltalk虚拟机[25]，其基本原理如图2.11所示。图2.11中，假定虚拟机已经观察到obj对象的类型大多为Point。则即时编译器对该调用点进行编译的策略为：1)为Point.draw()方法生成一段用于对象类型检查的先导指令序列；2)在调用点生成一个跳转到1)中先导指令序列起始地址的跳转指令。

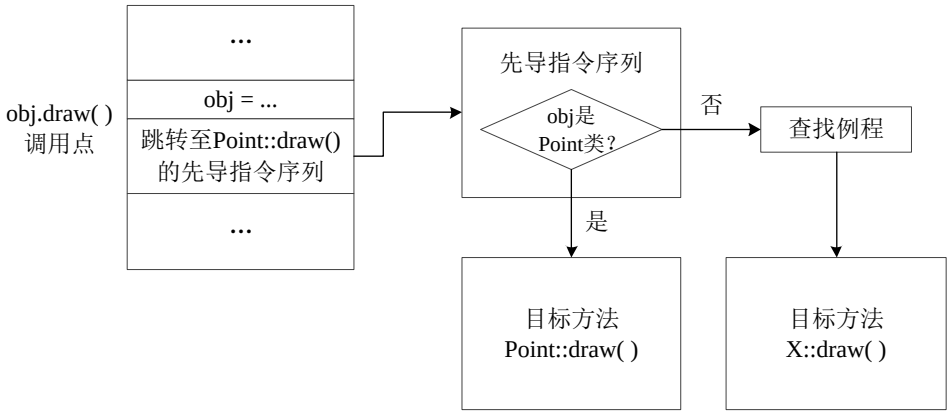


图 2.11: 高速内联缓存优化的虚方法调用解析过程

如图2.11所示，高速内联缓存优化后，程序在该调用点会直接跳转到Point.draw()方法的先导指令序列。在Point.draw()方法的先导指令序列中，会对obj对象的类型进行检查。若发现obj对象的类型恰好为Point，则内联缓存命中，动态解析成功，虚拟机继续执行Point.draw()方法。否则，内联缓存未命中，需跳转到慢速的虚方法调用查找例程以完成虚方法调用的解析。由于即时编译器在编译时将先导指令序列的起始地址直接缓存到了所生成的跳转指令中，故称为内联缓存。显然，内联缓存命中时，虚方法调用解析的开销大幅降低。一般地，同一个调用点上对象的类型通常不会频繁变化。研究表明，对于同一个虚方法调用点，约有90%以上的调用者类型保持不变[71]。故在实际系统中，内联缓存的命中率通常很高。因此，高速内联缓存技术能够大幅加快虚方法调用的速度，显著提升系统的性能。在Smalltalk的内联缓存技术基础之上，研究人员又进一步提出了多态内联缓存和众态内联缓存等扩展技术[27]。多态内联缓存和众态内联缓存技术在原有基础上进一步增加了缓存的对象类型的数量。

2.3.2.4 激进的方法内联技术

虽然高速内联缓存技术解决了虚方法动态解析开销大的问题，但却不能减少程序中虚方法调用的次数。由于需要对现场信息进行保存和恢复，方法调用的开销通常较大。而在Java等面向对象编程语言中，虚方法调用十分频繁，并且相当比例的调用均为对小方法的调用，例如获取或设置对象某个属性的值等。由此导致的另一个严峻的挑战是，虚拟机对小方法调用的开销通常远大于被调用方法本身的执行开销。为了提高虚拟机的性能，虚拟机有必要减少方法(尤其是对小方法)调用的次数。

方法内联是减少方法调用次数普遍采用的优化方案[72]。方法内联不仅可以消除方法调用的开销，同时还能为即时编译器提供更多的优化空间。但是，由于虚方法调用的目标在运行时由对象的类型确定，编译时尚无法获知应当被内联的目标方法。为了

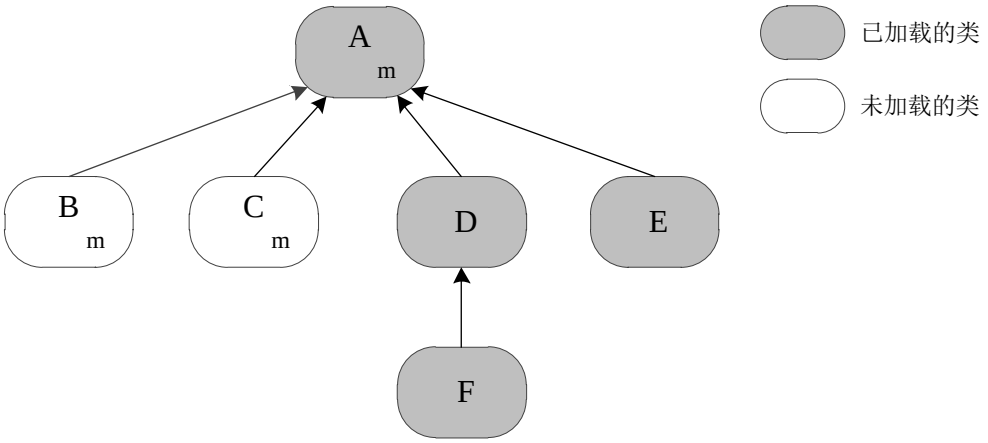


图 2.12: 类的层次关系分析图

实现对虚方法的内联优化，虚拟机研究人员提出了基于类层次关系分析的内联优化技术[73][74]。其基本思想是，分析当前已经加载的类的继承关系，对于当前能够确定调用目标的且适合进行内联的虚方法实施激进的内联优化。

例如，对于程序中的某个虚方法调用点obj.m()，程序中声明了方法m()的类型为A、B和C类，并且B和C均为A的子类。当即时编译器尝试对该调用点进行内联优化时，虚拟机已经加载的类的层次关系如图2.12所示。显然，由于B和C类此时尚未被加载，故obj对象的类型不可能是它们。虽然D、E和F类已被加载，并且均继承了A类，但由于它们都没有重写A类中的方法m()，故obj对象的类型也不可能是它们。因此，从图2.12的类层次关系分析可知，目前可以确定虚方法obj.m()调用的目标为A类的方法m()。于是，及时编译器假设对象obj的类型一直是A类，并将A类的方法m()作为被内联的目标。上述假设是较为激进的，因为后续运行时，B和C类均可能被加载，从而打破该假设。因此，虚拟机需要对假设的条件进行监控。当发现假设条件不成立时，虚拟机必须及时禁用基于错误假设的编译结果，以保证程序执行的正确性。在实际的应用程序中，内联假设失效的概率很低，故激进的方法内联技术可以大幅提升系统的性能[71]。

2.3.2.5　部分编译技术

部分编译[1]是动态编译特有的一项优化技术。方法内部通常包含很少执行甚至一次都不执行的代码分支。例如，对于程序中处理极端数据输入的分支，由于极端输入出现的概率极低，故相应的代码分支执行的频率也极低。而对于程序中处理异常和错误的代码，当异常和错误没有发生时，用于异常和错误处理的代码就不会被执行。显然，编译方法中执行频率较低的部分对程序性能的提升效果微乎其微。而对不执行的

代码片段进行编译，非但不能带来任何性能提升效果，反而白白浪费了系统的编译资源。此外，低频执行的代码片段被编译后产生的本地机器指令还会进一步增大系统的存储压力。

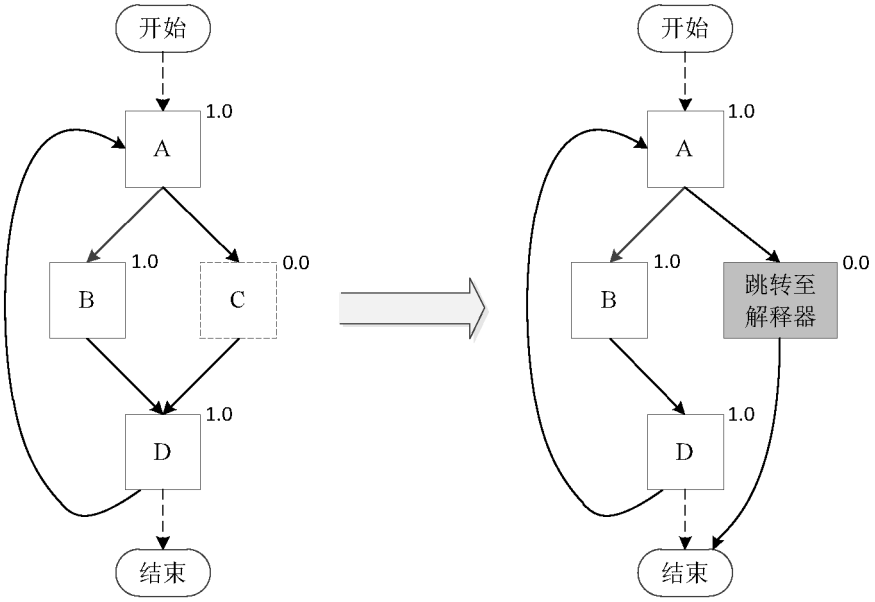


图 2.13: 部分编译优化原理示意图[1]

针对上述问题，虚拟机研究人员提出了部分编译技术[1][75]。通过该技术，即时编译器仅编译方法中频繁执行的分支，而忽略对较少执行的代码分支的编译。该技术实现的两个关键点是：1)如何识别方法中执行次数较少的分支；2)如何确保未编译分支在真正需要时能被执行。图2.13展示了对上述两个问题的解决思路。

- 通过剖面分析识别方法中执行次数较少的分支。虚拟机可以通过解释器中的剖面分析器识别方法中执行次数较少的分支。一种普遍采用的方案是对方法中所有分支指令的跳转方向进行计数，随后计算出方法中所有基本块的执行频率，并将执行频率较低的基本块识别为执行次数较少的分支。图2.13中每个方框表示方法的一个基本块，方框右上角标注了该基本块的执行频率。由于基本块C的执行频率为0，故基本块C被识别为执行次数较少的分支，编译器将不再编译基本块C。

- 通过解释器保证未编译分支在真正需要时被执行。虽然基本块C在过去被执行的次数为0，但并不能保证基本块C在未来永远也不会执行。为了使得未被编译的代码分支在需要的时候能被执行，即时编译器在基本块C的位置会生成跳转到解释器的指令序列，即通过解释执行的方式来执行基本块C的代码。

部分编译技术不仅提高了动态编译的效率，同时也降低了动态编译对系统存储资源的消耗。此外，部分编译技术还能削减编译器进行数据流分析、全局代码移动和寄存器分配的复杂度，以更好地优化方法中频繁执行的分支。

2.3.3 虚拟机其它优化技术

随着应用程序本身和输入数据规模不断增大，虚拟机的内存管理机制面临前所未有的挑战，垃圾收集的瓶颈效应逐渐开始显现。一方面，设计更加先进的垃圾收集算法是提高垃圾收集效率的关键；另一方面，研究如何减少垃圾对象的数量则是解决垃圾收集性能问题的根本途径。逃逸分析可以有效减少系统中垃圾对象的数量。逃逸分析[76–78]用于在编译时计算对象的生命周期及其可见范围。对象的生命周期及其可见范围分为：1)方法内部、2)单一线程内部和3)多线程可见三类。其中，若某个对象的生命周期及其可见范围属于前两类，则称该对象未逃逸，反之则为一个逃逸的对象。显然，对于属于第1)类的未逃逸对象，它也属于第2)类，反之则不成立。对于生命周期和可见范围均局限在某个方法内部的对象，可以将该对象直接分配在堆栈中[79][80]，从而避免了对其进行垃圾收集。同时，对于生命周期和可见范围均局限在某个线程内部的对象，针对该对象的加解锁操作是冗余的，在编译时均可被消除[81][79]。此外，对于未逃逸的对象，编译器还可以进行标量替换的优化，即将对象属性的访问直接转换为对一个普通变量的访问[82]。但是，由于逃逸分析的代价较高，仅在HotSpot等少数虚拟机中予以实现。

当前，多处理器架构的计算机系统已成为网络服务、科学计算和事务处理的主流。SMP(symmetric multi-processor，SMP)[83]和NUMA(non-uniform memory access，NUMA)[84]系统是两种最常见的多处理器计算机系统。相对于SMP系统，NUMA系统更易于实现大规模并行和分布式系统的构建，具有很高的可扩展能力，故NUMA系统的应用更为广泛。但是，NUMA系统中往往存在由于处理器跨节点访存所导致的性能低下的问题。为了充分发挥NUMA机器上虚拟机的性能，研究人员提出了针对NUMA架构的虚拟机优化算法[85–87]。其核心思想是，根据一定的策略确定对象分配的位置，以尽可能减少程序运行期间跨片访存的现象。该思想具体包含两层含义：1)减少应用线程的跨片访存次数；2)减少垃圾收集的跨片访存次数。对象分配时，将对象存储到最优的NUMA节点中，使得应用线程对该对象的访问尽可能多地位于同一个节点。垃圾收集时，合理划分每个垃圾收集线程负责的内存区域，使得垃圾收集线程和被回收的对象尽可能多地位于同一个节点。由于对象的分配和释放均由垃圾收集器负责，故现有的NUMA优化技术主要在垃圾收集器中实现。

近年来，有人尝试使用机器学习的方法对虚拟机性能进行优化。2012年，美国特拉华大学的Kulkarni等人，以马尔科夫随机过程为核心控制模型研究了动态编译的优化顺序问题，并在实验中观察到了性能提升的效果[88]。2013年，美国堪萨斯大学的Jantz等人又通过机器学习对动态编译优化组合的选择进行了研究，实验中发现虚拟

机的启动性能和峰值性能均获得了一定程度的提升[89]。同年，Kulkarni等人，通过机器学习的方法实现了对虚拟机中方法内联优化判别准则的自动构建[90]。实验结果表明，与传统的启发式规则相比，通过机器学习方法构建的内联规则具有更好的适应能力。上述机器学习的优化方法在很大程度上依赖于训练特征的正确选取，其训练过程复杂耗时，并且部分结果仅适用于特定的应用程序。因此，基于机器学习的优化技术仍处于试验和探索阶段，该方向的研究仍需不断深入。

2.4 本章小结

本章主要介绍了Java虚拟机及其相关的优化技术。Java虚拟机主要包括：运行时数据区、执行引擎、类加载器、垃圾收集器和本地方法接口等五大模块。其中，执行引擎是决定虚拟机性能的关键部件。虚拟机的执行引擎通常由解释器和即时编译器组成。本章对Java虚拟机主要模块的结构和功能进行了介绍，并重点阐述了解释执行和动态编译的优化技术。对Java虚拟机结构和功能的了解，是本文研究工作开展的基本前提。而对前人优化工作的总结和梳理，其目的是借鉴并吸收经典优化思想的精髓，并在此基础上进一步对虚拟机性能优化技术展开研究。

第三章　实验平台介绍

3.1　引言

本文主要以龙芯处理器为硬件研究平台。选择上述硬件平台最主要的原因是，本文的研究成果可以有效提升Java虚拟机在龙芯平台上的性能，以满足龙芯处理器在实际应用和推广中的需求，并进一步完善国产处理器的软硬件生态环境，具有重要的实际应用价值和战略意义。此外，龙芯处理器作为我国第一款自主设计的高性能通用处理器，具有完备的设计流程和仿真测试环境，便于本文从软件和硬件等多个维度对Java虚拟机的性能进行优化。

本章主要对本文的软硬件研究平台及其相关背景进行介绍。本章第二部分首先介绍了龙芯3B1500处理器。龙芯3B1500处理器是一款MIPS兼容的8核通用处理器，具有高性能、低功耗和低成本等特点。本文所有的研究工作均在龙芯3B1500处理器上进行。第三部分介绍了HotSpot虚拟机及其在龙芯平台上的实现。HotSpot是世界领先的高性能Java虚拟机，在实际应用中最为广泛。但在本文工作开展时，官方最新发布的HotSpot虚拟机中并没有针对MIPS体系结构的实现，导致HotSpot虚拟机在龙芯平台上无法运行。本文首先将HotSpot虚拟机移植到了龙芯平台，使得最新发布的Java虚拟机能够在龙芯平台上正确运行，并将其作为本文研究工作的软件平台。第四部分介绍了本文采用的测试程序。最后对本章进行了小结。

3.2　龙芯高性能通用处理器

龙芯是国产高性能处理器的杰出代表，经过10多年的发展，目前已经形成了以龙芯3号为主的面向PC和服务器领域、以龙芯2号为主的面向智能移动终端领域和以龙芯1号为主的面向嵌入式领域的三大产品系列。由于同X86和ARM等主流架构处理器的指令集不兼容，许多重要的软件无法直接在龙芯处理器上运行。随着龙芯等国产处理器市场化工作的不断推进，软件生态问题成为限制龙芯处理器推广应用的瓶颈[91]。Java具有平台无关的优良特性，并且在服务器、桌面和嵌入式等领域均有丰富的软件资源，可以很好地解决龙芯平台软件缺乏的问题。此外，由于Hadoop和Spark等Java软件在产业中已经实际构成了云计算和大数据应用的基础，未来龙芯平台还需要运行更多的Java程序。然而，由于现有的Java虚拟机主要面向X86和ARM等更为主流的处理器进行设计，导致Java虚拟机的性能在龙芯平台上面临严峻的问题，严重制约了以龙芯为代表的国产处理器的应用和推广。因此，对Java的高效支持是解决龙芯等国产处理器软件匮乏的有效途径，对于龙芯等国产处理器的进一步推广具有十分重要的战略意义。

3.2.1 GS464V处理器核

GS464V是一款兼容MIPS64指令集的通用RISC处理器核，最早应用于龙芯3B系列的高性能通用处理器中，其微结构如图3.1所示[2]。GS464V为四发射的超标量结构，每个时钟周期可对四条指令进行取指和译码，并且将译码后的指令动态地发射到五个全流水的功能部件中执行。在GS464V的指令流水线中，虽然指令在保证依赖关系的前提下乱序执行，但指令的提交还是按照程序原来的顺序进行，以保证精确例外和访存的顺序执行。

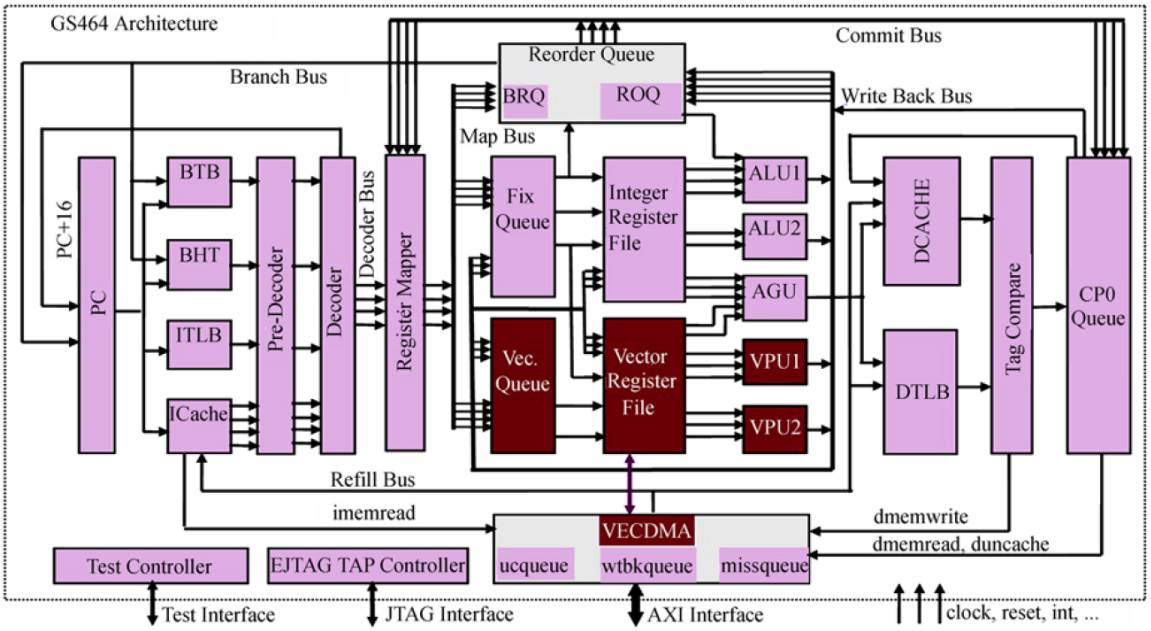


图 3.1: GS464V处理器核微结构[2]

为了减少流水线中由指令和数据的相关性导致的流水线阻塞问题，GS464V采用了乱序执行技术来提高流水线的性能。乱序执行主要包括寄存器重命名、动态调度和转移猜测等技术。寄存器重命名技术主要用于解决WAR(读后写)和WAW(写后写)相关，并用于例外和转移猜测失败引起的精确现场恢复。GS464V通过64项物理寄存器堆进行定点的重命名。动态调度技术主要用于减少由RAW(写后读)相关引起的阻塞，译码后指令的发射顺序为指令操作数就绪的次序，而并非指令在程序中出现的次序。GS464V中用于乱序发射的定点保留站和浮点保留站大小均为16项，并通过一个大小为64项的Reorder缓冲队列实现乱序执行的指令按照程序规定的次序提交。转移猜测通过预测转移指令是否跳转来减少由于控制相关引起的流水线阻塞。GS464V使用16项的转移目标地址缓冲器，2K项的转移历史表，9位的全局历史寄存器和4项的返回地址栈进行转移猜测。

为了进一步提高流水线的效率，GS464V中设计了先进的存储系统。GS464V的一级Cache采用指令和数据分离的哈佛结构，其指令Cache的大小为64KB，采用四路组相

联；数据Cache的大小为32KB的，采用二路组相联。GS464V中包含64项的TLB，采用全相联结构，每项可以映射一个奇页和一个偶页，支持4KB到16MB之间的页大小。与此同时，GS464V通过16项的访存队列以及8项的访存失效队列来动态地解决地址依赖问题，实现访存操作的乱序执行、非阻塞Cache、取数指令猜测执行等访存优化技术。此外，GS464V还支持128位的访存操作，其虚地址和物理地址均为48位。

相对于GS464等早期的龙芯处理器核，GS464V重点增强了对向量的处理能力。GS464V有两个向量功能部件，每个向量部件都可以全流水地执行4个64位的或者8个32位的浮点/定点乘加操作。同时，针对媒体应用、科学计算、信号处理和加解密等应用的特点，GS464V新增了大量256位的向量指令。此外，为了增强GS464V中两个向量部件读写数据的能力，GS464V采用了128项256位的向量寄存器堆，并且设计了专门的向量DMA模块在向量寄存器堆和二级Cache/内存之间搬运数据。

GS464V支持MIPS公司的EJTAG调试规范，采用标准的AXI接口，其指令Cache实现了奇偶校验，数据Cache实现了ECC校验。GS464V中还为支持二进制翻译和虚拟化等应用场景设计了许多专门的指令。上述特点可以增加龙芯3号处理器的适用范围。

3.2.2　龙芯3B1500处理器

本文的研究工作主要在龙芯3B1500处理器[92][93]上进行。龙芯3B1500处理器是一款高性能、低功耗和低成本的通用处理器，片内总共集成了8个处理器核，采用32nm工艺制造，最高工作主频为1.5GHz，其主要技术特征为：

- 片内集成8个GS464V高性能处理器核；
- 每个处理器核的工作频率可单独设置；
- 共享二级Cache，大小为8MB；
- 通过目录协议维护多核及I/O DMA访问的Cache一致性；
- 片内集成2个64位667MHz的DDR2/3控制器，支持DDR3-1333；
- 片内集成2个16位800MHz的HyperTransport控制器，最高支持1600MHz总线；
- 每个16位的HT端口可以拆分成两个8路的HT端口使用；
- 片内集成32位33MHz PCI(非标准电压)；
- 片内集成1个LPC(非标准电压)、2个UART、1个SPI、16路GPIO接口。

龙芯3B1500处理器包含两个对称节点的8核处理器，每个节点由4个GS464V处理器核组成，芯片的整体架构基于两级互连实现，结构如图3.2所示。第一级互连采用两

个相连的6*6交叉开关，分别用于连接四个GS464V处理器核(作为主设备)、四个共享二级Cache模块(作为从设备)、一个HT端口(每个端口使用一个Master和一个Slave)以及相邻的节点。HT控制器和一级互联开关相连，其中的DMA控制器负责HT总线IO请求的DMA控制并负责片间一致性的维护。第二级互连采用两个独立的5*4交叉开关，分别连接4个共享二级Cache模块(作为主设备)，DDR2内存控制器、高低速I/O以及芯片内部的控制寄存器模块。上述两级互连开关都采用读写分离的、宽度为128bit的数据通道，其工作频率与处理器核的频率相同，以实现高速的片上数据传输。

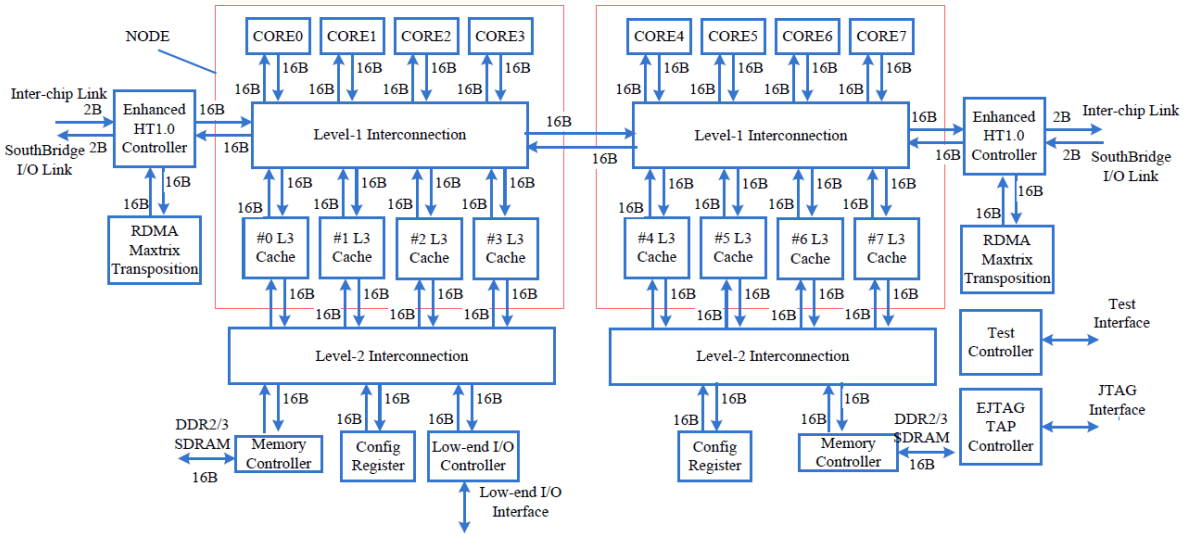


图 3.2: 龙芯3B1500芯片结构[3]

3.3 HotSpot虚拟机

HotSpot最初是由Sun公司开发并维护的Sun JDK中的Java虚拟机，经过十多年的发展，现已成为Java虚拟机在工业界的事实标准。2006年，Sun公司宣布对外提供Sun JDK的开源版本OpenJDK。HotSpot虚拟机严格遵循Java语言规范，是Java虚拟机的标准实现。同时，HotSpot虚拟机的设计和性能处于世界领先的地位，故实际应用最广泛。因此，本文选择HotSpot虚拟机作为本文研究的软件平台。

本文的研究基于最新发布的OpenJDK8进行。相对于OpenJDK6和OpenJDK7，OpenJDK8进一步增强了虚拟机在并发处理、垃圾回收和向量运算等方面的能力，同时新增了对Lambda表达式等新特性的支持。但在本文工作开展时，OpenJDK8中的HotSpot虚拟机并没有针对MIPS体系结构的实现，导致最新的HotSpot虚拟机无法在龙芯上运行。本文首先将HotSpot虚拟机移植到了龙芯平台，一方面可以提升龙芯平台上Java虚拟机的性能，另一方面也使得本文的后续研究工作得以展开。

3.3.1 HotSpot虚拟机总体结构

相对于C/C++等传统编程语言，Java语言具有跨平台和高安全性等优势。这些优势主要通过Java字节码和Java虚拟机来实现。Java字节码本质上是一套平台无关且面向Java虚拟机的指令集。所有的Java程序均以标准类文件的形式对外发布。类文件是Java程序的字节码表示形式，可以被Java虚拟机加载、链接和执行。如图3.3所示，HotSpot虚拟机主要由解释器、即时编译器和虚拟机运行时服务等三大部分构成。

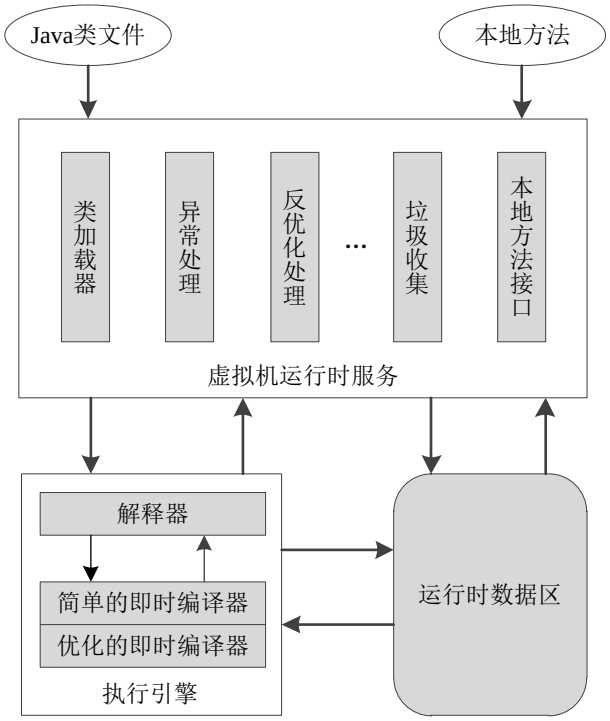


图 3.3: HotSpot虚拟机总体结构

解释器用于对字节码进行解释执行，即时编译器负责将字节码翻译为功能等价的机器指令。HotSpot虚拟机包含简单的和优化的两个功能等价的即时编译器。优化的即时编译器投入了更多的优化算法，因而能够获得更高的峰值性能。虚拟机运行时服务为虚拟机提供了诸如类加载、异常处理、反优化、垃圾收集、本地方法接口等重要功能。

Java字节码在HotSpot虚拟机中以混合模式执行，即字节码既可以由解释器解释执行，也可以被即时编译器翻译为本地机器指令后执行。最初的Java虚拟机仅由解释器实现，为了提高Java程序的性能，不久便在虚拟机中引入了即时编译器。在HotSpot虚拟机中，Java方法最初都是由解释器以解释方式执行的。在解释执行的过程中，HotSpot虚拟机会识别出Java程序中执行频度较高、对性能影响较大的方法，即

所谓的“热点”方法。随后虚拟机调用即时编译器将这些方法编译为本地机器指令，此后对这些方法的调用将直接执行编译后生成的本地机器指令。

3.3.2 龙芯上的HotSpot虚拟机

本文研究工作开始时，HotSpot的解释器已经初步移植到龙芯平台上。而决定虚拟机性能的即时编译器尚未开始移植。由于优化的即时编译器具备更高的性能，为了减小移植的工作量，本文仅对HotSpot中优化的即时编译器进行了移植，其基本工作流程包括语义分析与中间代码生成、机器无关优化、指令选择、全局代码移动与调度、寄存器分配、窥孔优化和代码生成等几个主要阶段。上述阶段可分为平台无关和平台相关两大部分。平台无关部分包括语义分析与中间代码生成以及机器无关优化等，和处理器的架构无关。平台相关部分包括指令选择、全局代码移动与调度、寄存器分配、窥孔优化和代码生成等，与处理器的指令系统、流水线结构以及存储系统等密切相关。因此，龙芯平台上即时编译器移植的主要工作是针对龙芯处理器的平台相关部分的设计与实现。

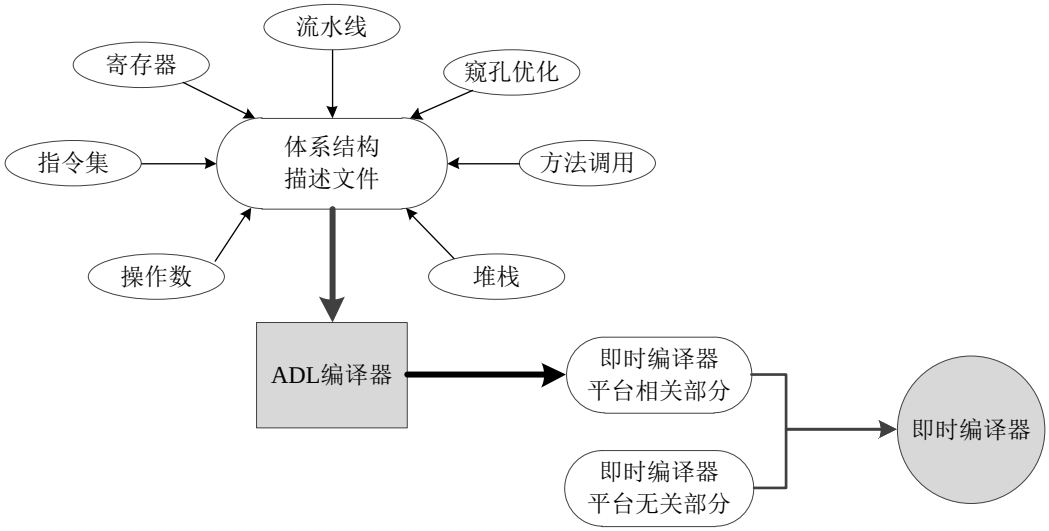


图 3.4: 基于体系结构描述语言的即时编译器构造原理

为了方便在不同架构的处理器上构建即时编译器，HotSpot提供了一种称为体系结构描述语言(architecture description language，ADL)的工具。该语言由Sun公司定义，可用于在各种处理器平台上对HotSpot中优化的即时编译器进行半自动化构建。图3.4展示了在龙芯处理器平台上，HotSpot中优化的即时编译器设计和实现的基本原理。其关键在于高效地设计出适用于龙芯处理器的体系结构描述文件。该文件采用Sun公司定义的体系结构描述语言编写，主要用于规定即时编译器平台相关部分的设计，重点包括

指令集、操作数、寄存器分配、堆栈、流水线、方法调用和窥孔优化等内容。体系结构描述文件经过ADL编译器编译即可自动生成编译器平台相关部分的C++源代码，再结合即时编译器平台无关部分的代码，最终生成完整的即时编译器。因此，龙芯平台上即时编译器移植的核心是体系结构描述文件的设计。下面将对龙芯平台的体系结构描述文件设计中的关键部分进行叙述。

3.3.2.1　寄存器分配

龙芯处理器中包含32个通用寄存器，只有高效地利用这些寄存器，才能充分发挥虚拟机在龙芯上的性能。在Java方法的翻译过程中，寄存器的分配和使用是由即时编译器的设计者决定的，具有较高的灵活性。为了和其它MIPS处理器保持兼容，本文在设计即时编译器中寄存器的使用规则时参考了MIPS体系结构基本的ABI规范。具体的寄存器分配和使用规则如表3.1所示。

表 3.1: 寄存器使用规约

类型	名称	用途
自由使用寄存器	AT、T8、T9	在具体指令规则中暂存中间结果， 不参与编译时的变量分配
临时寄存器	T1 ∼ T3 S0 ∼ S7	编译器可自由分配
传参寄存器	T0 A0 ∼ A7	Java方法调用时用于参数传递 T0为消息接收对象，A0 ∼ A7为调用参数
返回值寄存器	V0、V1	保存Java方法调用的返回值
返回地址寄存器	RA	保存Java方法调用的返回地址
栈顶指针寄存器	SP	保存当前的栈顶指针
帧指针寄存器	FP	保存当前的栈基地址
系统保留寄存器	K0、K1、GP	系统保留寄存器

3.3.2.2 操作数定义

操作数定义用于对目标平台上可能用到的操作数进行声明，通常可分为寄存器操作数、常量操作数和存储器操作数等三类。

(1) 寄存器操作数：定义指令规则中可以使用的寄存器操作数。在龙芯平台上，分别针对int型、long型、float型和double型定义了相应的寄存器操作数类型。由于在指令规则定义中声明的寄存器操作数类型直接限定了该条指令规则可以使用的寄存器，因而寄存器操作数的定义对即时编译器的寄存器分配有较大影响。

(2) 常量操作数：定义程序中可能出现的常量。内存基地址、常指针、访存偏移量和部分运算分量等可能以常量的形式出现。通常将使用频率高的常量单独定义成体系结构描述文件中的操作数类型，这不仅可以使得相关指令规则的描述更加简洁，还能精简该指令规则所需生成的本地机器指令的数目。

(3) 存储器操作数：对内存中某个存储单元地址的描述，实质是对目标处理器中寻址方式的表达。X86体系结构的寻址方式复杂，故在X86的体系结构描述文件中可以找到很多有关存储器类的操作数定义。而MIPS架构的寻址方式相对简单，故在龙芯平台的体系结构描述文件中定义的存储器类型操作数更为简洁明了。

3.3.2.3 指令规则定义

指令规则定义用于描述可供即时编译器在指令选择阶段进行选择的机器指令，是对目标处理器指令集的系统表述。根据ADL语法的规定，每条指令规则的定义需要描述该条指令的操作数、IR(intermediate representation，IR)子树匹配模式、汇编输出样式、指令代价、发射的本地机器指令序列以及所属的流水线类型等信息。图3.5展示了龙芯处理器上int型寄存器操作数的加法指令规则的一个实例。

指令规则定义需要注意以下几点：

(1) 明确有无标志寄存器：目标处理器标识寄存器的有无，对于指令规则的定义影响很大。在X86中需时刻考虑到标志寄存器的变化，而在龙芯平台用于整型运算的部件中并无标志寄存器，故无需考虑标志寄存器的变化。

(2) 合理估计指令规则代价：由于即时编译器利用每条指令规则的代价估值进行指令选择，故需要根据机器指令在目标处理器上的延迟合理估计其代价。

(3) 正确描述指令规则的流水线类型：指令所属的流水线类型将用于在基本块内进行局部指令调度。

```
instruct addI_Reg_Reg(mRegI dst, mRegI src1, mRegI src2) %{
  match(Set dst (AddI src1 src2));              // 匹配模式
  format %{ add $dst,$src1,$src2 %}             // 汇编输出样式
  ins_encode %{
    Register  dst = $dst$$Register;             // 目的寄存器
    Register src1 = $src1$$Register;            // 源寄存器1
    Register src2 = $src2$$Register;            // 源寄存器2
    __ addu(dst, src1, src2);                   // 发射指令
  %}
  ins_pipe( ialu_reg);                          // 流水线类型
%}
```

图 3.5: 指令规则定义的实例

3.3.2.4　流水线描述

现代处理器为了加速指令的执行，采用了乱序执行和多发射等先进的流水线技术。为了尽可能发挥目标处理器的流水线优势，体系结构描述文件中设计了处理器流水线特征的描述接口。流水线的信息将被用于对基本块内的代码实施更有效的局部调度，以减少由于数据相关导致的流水线阻塞。体系结构描述文件中对目标处理器流水线的描述分为宏观流水线描述和指令相关流水线类型描述两个层次：

- **宏观流水线描述：**主要包括对处理器流水级划分和流水线中资源部件如译码器、运算器等的描述。这部分内容只需在体系结构描述文件中单独描述即可。
- **指令相关流水线类型描述：**表征机器指令在处理器流水线中实际执行所经历的流水过程，要求指明各个阶段的处理延迟。与宏观流水线描述不同，这部分内容除了单独进行描述之外，还需要和体系结构描述文件中定义的具体指令规则相关联。按照实际指令的执行过程，通常可将指令相关流水线类型划分为整型数据访存类型、浮点型数据访存类型、整型数据运算类型、浮点数据运算类型和分支转移类型等。

3.3.2.5　方法调用机制

Java程序运行期间方法调用非常频繁。根据被调用对象的不同，方法调用可细分为静态方法调用、动态方法调用和虚拟机内部运行时服务调用等不同的类型。方法调用时，需要将控制流由调用者转移给被调者。为了实现正确的跳转过程，需要针对不

表 3.2: 关键的Call节点

节点名称	功能描述
CallStaticJava	静态Java方法调用
CallDynamicJava	动态Java方法调用
CallRuntime	从Java方法中调用运行时服务

同的方法调用类型进行方法调用的解析。龙芯平台上的即时编译器在其体系结构描述文件中为方法调用的解析实现了若干Call节点作为方法调用的接口，表3.2列举了其中最关键的Call节点。此外，体系结构描述文件中还设计了另外两个关键的节点用于实现对方法调用的支持，即MachPrologNode节点和MachEpilogNode节点。在方法调用跳转之后、被调方法执行之前，首先会执行由MachPrologNode节点发射的指令序列；而在执行完被调方法的语义动作之后、方法调用返回之前，又会执行由MachEpilogNode节点发射的指令序列。通常，在MachPrologNode中需要实现堆栈空间分配、返回地址(RA)和帧指针(FP)寄存器的保存等方法调用的语义动作；而在MachEpilogNode中需要实现堆栈空间释放、返回地址(RA)和帧指针(FP)寄存器的恢复等方法调用的语义动作。

3.3.2.6 堆栈布局

合理的堆栈设计是实现方法调用机制的关键。龙芯平台上方法调用堆栈的布局如图3.6所示。方法调用堆栈从上到下依次分为传入参数区、返回地址、帧指针、加锁变量区、局部变量区和输出参数区等六个部分。

(1) 传入参数区：存储方法调用的传入参数；

(2) 返回地址：存储方法调用的返回地址(RA寄存器的值)；

(3) 帧指针：存储方法调用者栈帧寄存器(FP)的值；

(4) 加锁变量区：存储线程间同步时用于加锁的变量；

(5) 局部变量区：存储被调方法中使用的局部变量；

(6) 输出参数区：当被调方法调用新的方法时用于传递所需的调用参数。

方法调用堆栈向下增长，即由内存高地址向低地址方向生长。被调用者栈帧由被调用者在内存中动态分配，并在方法调用返回时释放。

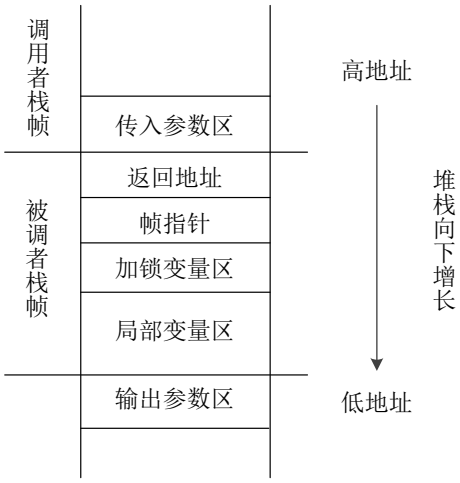


图 3.6: 方法调用堆栈

3.3.2.7　窥孔优化

窥孔优化是编译器常用的一种简单有效的优化手段。窥孔优化设计时主要使用了龙芯特有的指令，以尽可能发挥龙芯平台的硬件优势。

3.4　测试程序

本文使用了学术界和工业界广泛使用的SPECjvm98[94]和DaCapo-9.12-bach[95]测试集。SPECjvm98和DaCapo专门针对Java虚拟机性能测试而设计，是国际上公认的Java虚拟机性能测试程序。SPECjvm98测试集由标准性能评估公司开发，包含7个独立的测试项目，是进行Java虚拟机性能评估的经典测试集。DaCapo测试集由美国普渡大学和IBM沃森实验室等世界著名计算机研究团队联合开发，包含14个独立的测试项目，是现代主流的Java虚拟机测试集。上述两个测试集均取自真实的应用程序，以保证测试结果的有效性和普适性。SPECjvm98和DaCapo测试集模拟了现实世界中包括数值计算、事务处理、访存和I/O等操作，可以全面评估系统的定点/浮点计算能力、访存能力和I/O的性能。此外，DaCapo测试集还进一步强化了对系统内存管理和并发能力的评测，更加符合现代云计算和大数据应用的需求。由于最新发布的DaCapo-9.12-bach中，*batik*和*eclipse*引用了与OpenJDK8不兼容的低版本类库，因此本文将它们排除在实验结果之外。两个测试集中所包含的具体测试程序如表3.3和表3.4所示。将上述两个测试集联合起来，可以更加全面地覆盖现代Java虚拟机应用的各个领域，从而使得对本文研究成果的评估更为系统和全面。

表 3.3: SPECjvm98基准测试程序简介

测试名称	功能描述
compress	LZW压缩算法
jess	交互式脚本系统
db	数据库访问操作
javac	Java编译器
mpegaudio	音频文件解码
mtrt	光线跟踪算法
jack	Java语法分析器的自动生成器

表 3.4: DaCapo基准测试程序简介

测试名称	功能描述
avrora	AVR微控制器网格应用模拟
batik	SVG图像生成
eclipse	Eclipse性能测试
fop	XSL-FO格式转PDF
h2	数据库事务处理
jython	Python解释器
luindex	文本索引
lusearch	文本搜索
pmd	Java类分析器
sunflow	图像渲染
tomcat	网络服务请求处理
tradebeans	基于Java Beans的daytrader测试
tradesoap	基于SOAP的daytrader测试
xalan	XML到HTML转换器

3.5　本章小结

本章主要对本文研究工作的软硬件研究平台和测试程序进行了介绍。硬件平台方面，主要介绍了龙芯GS464V处理器核和龙芯3B1500通用处理器芯片。在软件平台方面，主要介绍了HotSpot高性能Java虚拟机系统，并展示了其在龙芯平台上的移植过程。本文的研究工作基于真实的处理器平台展开，并且使用权威的测试程序进行性能评估。因此，本文的研究成果具有重要的理论意义和实际应用价值。

第四章　基于贝叶斯估计的程序热点识别算法

4.1　引言

近年来，云计算、大数据和移动互联网等新兴应用使得Java的应用更为广泛，同时也使得Java虚拟机的设计面临更为严峻的挑战，其性能往往不能完全满足实际应用的需求[4][5]。动态编译是提升虚拟机性能最有效的手段。即时编译器将方法中的字节码编译成本地机器代码，并通过硬件直接执行生成的机器代码来实现加速。然而，动态编译会引入额外的运行时开销，从而对程序的性能产生负面影响。为了尽可能减少动态编译的开销，虚拟机通常只编译程序中频繁执行的方法，即热点方法。热点识别要求尽可能早地识别程序中的热点方法，以充分发挥本地机器代码执行的优势。由于对程序未来的行为不可预知，程序热点提前侦测的难度非常大。现有的虚拟机通常采用基于固定阈值的启发式规则来识别程序中的热点。固定的阈值使得热点识别缺乏灵活性，很容易导致热点识别过于保守或者过于激进。同时，固定的阈值使得热点识别对程序行为的适应能力较差。此外，现有的热点识别技术通常仅使用过去的信息预测程序将来的行为，无法实现对程序热点的前瞻性预测，导致热点识别的延迟较大，严重影响虚拟机的性能[67][30]。

为了克服上述热点识别技术的缺陷，本章首先设计了一种基于智能阈值的热点识别模型，解决了现有技术中热点识别阈值的设置缺乏灵活性和适应性的问题。随后，以该模型为基础，本章进一步提出了一种基于贝叶斯估计的程序热点识别算法，实现了对程序热点的前瞻性识别，有效降低了热点识别的延迟。

本章其余部分组织如下：4.2节对本章研究内容的背景和相关工作进行了介绍；4.3节设计了基于智能阈值的热点识别模型；4.4节提出了基于贝叶斯估计的程序热点识别算法；4.5节对本章算法进行了实验，从热点方法的识别延迟、编译延迟和识别率等方面进行了分析，展示了本章算法对提升虚拟机性能的有效性；最后对本章进行了小结。

4.2　相关工作

由于直接统计方法的运行时间会引入巨大的运行时开销，实际的虚拟机系统通常采用轻量的剖面分析技术来识别程序中的热点方法。采样和计数是虚拟机中两种最为常用的热点识别策略[12]。对于基于采样的热点识别系统(如Jikes)，虚拟机在程序运行期间周期性地对线程的调用堆栈进行采样，并对方法在采样中出现的次数进行计数。当监测到某个方法出现的频率超过热点识别的阈值时，该方法被识别为热点方法。而对于基于计数的热点识别系统(如HotSpot)，虚拟机在程序运行期间对所有方法的调用

次数进行计数[15]。与此同时，为了加快对含有循环的热点方法的识别，高性能虚拟机还对方法内部循环迭代的次数进行计数。设$R(m,t)$表示对方法m未来执行次数的估计，$ic(m,t)$和$lc(m,t)$分别表示方法m在t时刻被调用的次数和内部循环的次数，则$R(m,t)$可表示为

$$R(m,t) = ic(m,t) + lc(m,t) \tag{4.1}$$

当$R(m,t)$超过热点识别的阈值时，方法m即被识别为热点方法。采样和计数的核心思想都是直接使用程序过去的行为估计程序将来的运行趋势。即若某个方法在过去频繁执行，则认为它在将来也会频繁执行。由于基于计数的热点识别系统更易于实现和调试，故基于计数的热点识别在实际的虚拟机中更为常见。

除了利用方法在过去的运行时信息，研究人员还引入了方法本身的静态特征来辅助热点方法的预测。Schilling[96]提出了一种基于方法调用次数和方法大小的线性热点识别模型。该模型对$R(m,t)$的估计表示为

$$R(m,t) = \alpha \cdot ic(m,t) + \beta \cdot size(m) \tag{4.2}$$

其中，$size(m)$表示方法m的大小，α和β为线性判别模型的待定参数。该模型引入了方法本身的静态特征来构建热点识别模型，对于此后热点识别算法的改进具有参考意义。但该模型忽略了方法内部的循环事件，无法快速识别含有循环的热点方法。此外，实验结果表明，该模型的性能全面落后于“Always-JIT”(即无条件编译所有方法)的性能，故其实际应用价值不大。

受Schilling的启发，Liu等人[97]提出了一种多级阈值的热点识别技术。其基本思想是采用4.1式来估计$R(m,t)$的值，并根据方法的大小设计多个不同的热点识别阈值。Liu等人经过分析后认为不同大小的方法应对应不同的热点识别阈值。一般地，方法越小，对其进行热点判别的阈值也越低。为了控制热点识别阈值的数量，他们根据方法的大小对方法进行了归类，每一类对应一个阈值。实验结果表明，采用两级的阈值即可获得较好的性能提升效果。然而，该方案的热点识别阈值仅由方法的大小确定，并未充分发掘方法内部的结构特征和程序的运行时信息，热点识别缺乏灵活性和对程序行为的适应能力。此外，该方案无法对程序中的热点方法进行前瞻性识别，热点识别的延迟依然较大。

综上所述，自动态编译技术诞生以来，人们已经提出了多种热点识别的方案。但上述方案至今存在许多问题尚未解决。其中，对性能影响最大的两个问题是：1）热点识别缺乏灵活性和自适应性；2）热点识别延迟过大。在热点识别的阈值设置方面，现有技术通常采用一个(或若干个)固定的热点识别阈值。固定的阈值忽略了方法自身的结构特征和系统运行状态等因素对热点识别的影响，使得热点识别缺乏灵活性，很容易导致热点识别过于保守或者过于激进。同时，固定的阈值使得热点识别无法自适应地调节系统的编译负荷，对程序行为的适应能力较差。而在热点识别方面，现有的热点

识别技术通常仅基于过去的运行信息预测程序将来的行为，不能实现对程序热点的前瞻性预测，导致热点识别的延迟较大，严重影响虚拟机的性能。针对现有技术的不足，本章提出了一种基于智能阈值的热点识别模型，解决了现有技术中阈值设置缺乏灵活性和自适应性的问题。并在此基础上设计了基于贝叶斯估计的程序热点识别算法解决了现有技术中热点识别延迟过大的问题。

4.3　基于智能阈值的热点识别模型

为了解决现有技术中热点识别阈值缺乏灵活性和适应性的问题，本节提出了基于智能阈值的热点识别模型。其基本原理是若程序中某个方法总的执行次数不低于智能阈值，则该方法被识别为热点方法。其中，智能阈值是整个热点识别模型的核心。该阈值由方法的静态特征和当前系统的运行状态实时计算得到。本节主要对热点识别模型及智能阈值的计算进行阐述。

设在时刻t，将方法m判定为热点方法的收益为$Benefit(m,t)$，开销为$Cost(m,t)$。令$Gain(m,t)$表示将方法m判定为热点方法后系统性能的净收益，则

$$Gain(m,t) = \alpha \cdot Benefit(m,t) - (1-\alpha) \cdot Cost(m,t) \tag{4.3}$$

其中，α为可调参数，且$\alpha \in [0,1]$。动态编译不仅会引入额外的运行时开销，同时也会阻碍应用程序本身的执行。为了降低动态编译对程序性能的负面影响，本文规定当净收益足够大时才将方法识别为程序热点。于是，引入了一个模型参数G，满足$G \geq 0$，当且仅当净收益$Gain(m,t)$不小于G时，方法m才被识别为热点方法。故t时刻，方法m是否为热点方法的判别函数$Hot(m,t)$可表示为

$$\begin{aligned} Hot(m,t) &= Gain(m,t) - G \\ &= \alpha \cdot Benefit(m,t) - (1-\alpha) \cdot Cost(m,t) - G \end{aligned} \tag{4.4}$$

相应的判别准则为：当且仅当$Hot(m,t) \geq 0$时，方法m被识别为热点方法。上述模型实现的关键是合理地计算出收益$Benefit(m,t)$和开销$Cost(m,t)$的值。

首先对模型中收益$Benefit(m,t)$的值进行估计。热点识别的收益$Benefit(m,t)$的大小主要由t时刻后方法m的执行次数和单次执行性能的提升幅度共同决定。设方法m在t时刻后执行的次数为$N(m,t)$，并且t时刻后平均每次执行的性能提升幅度为$B(m)$，则方法m被判定为热点方法的收益$Benefit(m,t)$可表示为

$$Benefit(m,t) = N(m,t) \cdot B(m) \tag{4.5}$$

其中，合理地对$N(m,t)$的值进行估计是本模型实现的难点。本文将$N(m,t)$转换为方法m总的执行次数进行估计，具体的估计方法详见4.4节。方法m单次执行的性能提升主要取决于编译后执行相对于解释执行的性能提升幅度。由于在运行期间直接获取方

法m编译后执行的性能提升幅度非常困难，本文通过对方法m的结构进行静态分析，从而间接估算出编译后单次执行的平均性能提升幅度。一般地，方法中每个字节码对应一段实现其语义功能的本地指令序列。字节码的本地指令序列中包含的机器指令越多，解释执行的开销越大，编译后执行的性能提升空间也越大。因此，$B(m)$的值可以通过方法m中所有字节码的解释执行开销来进行估算，即$B(m)$可表示为

$$B(m) = \sum_{i \in BC(m)} w(i) \tag{4.6}$$

其中$BC(m)$表示方法m所包含的字节码的集合，$w(i)$表示字节码i的执行开销。$w(i)$可以由字节码i在具体平台上实际执行的代价确定，也可以根据字节码i的本地指令序列所包含的机器指令进行估算。至此，收益$Benefit(m,t)$可表示为

$$Benefit(m,t) = N(m,t) \cdot \sum_{i \in BC(m)} w(i) \tag{4.7}$$

接下来估计模型中开销$Cost(m,t)$的值。热点识别的开销$Cost(m,t)$主要为方法m的动态编译开销。方法的编译时间越长，其编译开销也越大。而决定编译时间的因素主要包括方法的大小、方法中循环的数量以及当前编译队列的长度等。文献[96]的研究表明方法的编译时间与方法的大小具有正相关的关系。一般地，方法越大，所需的编译时间也越长。同时，由于现代编译器对方法中循环结构的深度优化也会占用大量系统资源，其内部循环越多，编译耗时也越大。此外，若当前编译队列中积压的待编译方法的数量较多，表明系统面临较重的编译负荷，编译线程对系统资源的竞争非常激烈，也会导致编译时间的延长。因此，本文综合了方法的大小、方法中循环的数量以及当前编译队列的长度等因素对$Cost(m,t)$的值进行估计。令$loop_num(m)$表示方法m中循环结构的数量，$size(m)$为方法m的大小，$queue_len(t)$为时刻t编译队列的长度，则热点识别的开销$Cost(m,t)$可表示为：

$$Cost(m,t) = [loop_num(m) + 1] \cdot size(m) \cdot [queue_len(t) + 1] \tag{4.8}$$

最后，将4.7式和4.8式代入4.4式，热点方法判别函数$Hot(m,t)$可表示为

$$\begin{aligned} Hot(m,t) = \alpha \cdot [N(m,t) \cdot \sum_{i \in BC(m)} w(i)] \\ - (1-\alpha) \cdot \{[loop_num(m) + 1] \cdot size(m) \cdot [queue_len(t) + 1]\} - G \end{aligned} \tag{4.9}$$

在t时刻，4.9式中除变量$N(m,t)$的值未知外，其余变量的值均可在运行时获取。记N_0为等式 $Hot(m,t) = 0$ 成立时$N(m,t)$的临界值，则

$$N_0 = \frac{(1-\alpha) \cdot [loop_num(m) + 1] \cdot size(m) \cdot [queue_len(t) + 1] + G}{\alpha \cdot \sum_{i \in BC(m)} w(i)} \tag{4.10}$$

其中，α为可调参数，并且要求$\alpha \in (0,1]$。又由4.9式知，$Hot(m,t)$随$N(m,t)$单调递增。故当$N(m,t) \geq N_0$时，$Hot(m,t) \geq 0$ 成立，此时方法m应当被判定为热点方法，否则判为非热点方法。于是，t时刻对方法m的热点识别问题可以转换为判定不等式4.11是否成立的数学问题。

$$N(m,t) \geq N_0 \tag{4.11}$$

更进一步地，设t时刻方法m已执行的次数为$Execute(m,t)$，不等式4.11的左右两边同时加上$Execute(m,t)$，得

$$N(m,t) + Execute(m,t) \geq N_0 + Execute(m,t) \tag{4.12}$$

记方法m总共需要执行的次数为$Total_execute(m)$，则$Total_execute(m) = N(m,t) + Execute(m,t)$，为一个与时刻$t$无关的变量。令$T(m,t) = N_0 + Execute(m,t)$，则4.12式可表示为：

$$Total_execute(m) \geq T(m,t) \tag{4.13}$$

则4.13式即为本文提出的热点识别模型。其中，$T(m,t)$被称为该模型的热点识别阈值，其值由若干静态和动态属性特征综合计算得出。静态属性特征主要为方法本身的结构信息，包括方法的字节码、方法的大小和方法中循环的数目等。而动态属性特征可分为方法和系统状态两个方面，包括方法的执行次数和编译队列的长度等。因此，$T(m,t)$的值不仅和方法m有关，也和当前的系统状态密切相关。和传统的固定阈值相比，根据本模型计算出的阈值具有高度的灵活性，并且对程序行为的适应能力更强，因而被称为智能阈值。

4.4　基于贝叶斯估计的程序热点识别算法

由4.3节的分析知，热点识别的关键是判断不等式4.13是否成立。式中的智能阈值$T(m,t)$可以根据热点识别模型中的相关公式在程序运行过程中计算得到。然而方法总的执行次数$Total_execute(m)$在程序结束前却无法准确计算。故在程序运行期间，想要通过分别获取$Total_execute(m)$和$T(m,t)$的值来判断不等式4.13是否成立是不现实的。经典统计和贝叶斯统计是对未知变量进行估计的两种重要途径。经典统计主要依靠样本信息进行估计，其最大的缺陷是忽视了已有的经验和认识等先验信息，因而估计的精度主要取决于样本的大小，对于小样本的估计十分困难。而贝叶斯统计除了基于样本信息外，还充分利用了被估计量的先验信息，对于小样本也可以获得较好的估计结果。由于综合利用了样本信息和先验信息，贝叶斯估计通常能够获得更加合理的结果。因此，本节采用贝叶斯估计[98]的策略对方法执行次数不低于热点识别阈值的概率进行预测，进而提出了基于贝叶斯估计的程序热点识别算法，解决了不等式4.13的判定问题。

贝叶斯估计将被估计的变量视为一个随机变量。贝叶斯估计前，首先需要将被估计变量的先验信息表示为数学形式的概率分布，即先验分布。贝叶斯估计时，利用观测到的样本信息对先验分布进行更新，最终获得后验分布。最后通过后验分布估计随机变量的值。基于上述分析，采用贝叶斯估计的策略对方法的执行次数不低于热点识别阈值的概率进行估计的步骤为：

- **步骤1：先验分布的构建**。通过对应用程序的观察和分析，合理地构造出方法执行次数的先验分布；
- **步骤2：先验分布的更新**。根据观察到的运行时数据对步骤 1 中的先验分布进行更新，并得到方法执行次数的后验分布；
- **步骤3：基于后验分布的估计**。根据步骤 2 中的后验分布，计算出方法的执行次数不低于热点识别阈值的概率。

当且仅当步骤3中计算出的概率值足够大时，才判定不等式4.13成立，相应的方法才被识别为热点方法；否则判为不成立，相应的方法被视为非热点方法。

4.4.1 先验分布的构建

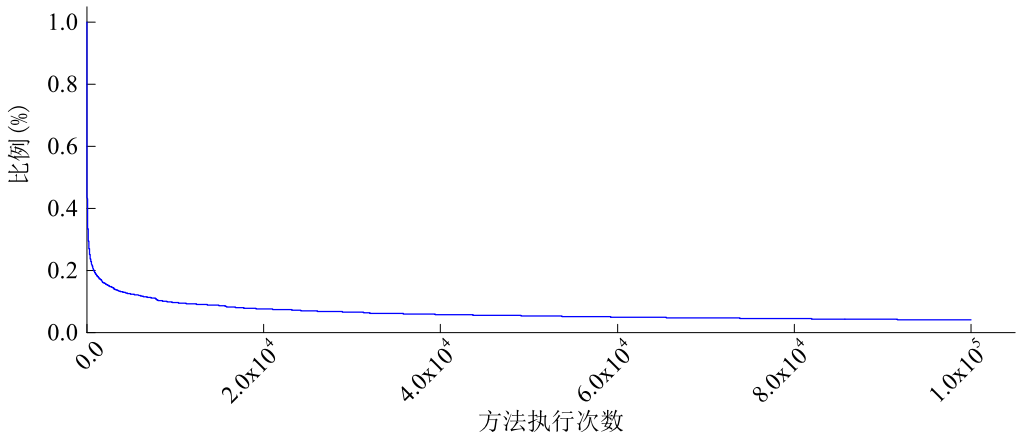
贝叶斯估计的第一步是先验分布的构建。幂率分布广泛存在于各种自然和社会现象中，是科学研究中常用的概率模型[99][100]。其数学表达式为

$$y = cx^{-r} \tag{4.14}$$

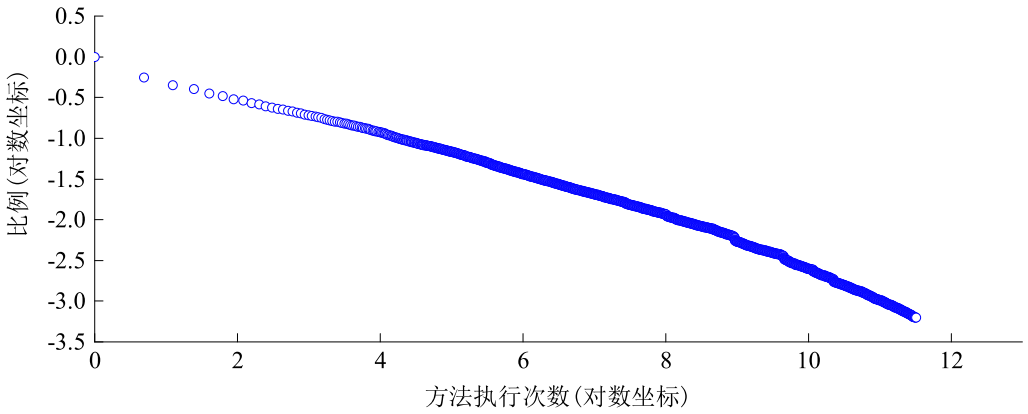
其中c和r均为大于零的常量。对4.14式的左右两边同时取对数，得

$$lny = -rlnx + lnc \tag{4.15}$$

由4.15式可知，lnx和lny满足简单的线性关系，其斜率为$-r$，截距为lnc。故在双对数坐标下，幂率分布表现为一条斜率为负数的直线。上述线性关系是判断给定的两个随机变量是否满足幂率分布的重要依据。图4.1展示了对DaCapo测试集中方法执行次数的统计分析结果。图4.1(a)的横坐标表示方法执行的次数，纵坐标表示执行次数不低于x的方法占被调方法总数的比例，图4.1(b)为图4.1(a)在双对数坐标下的结果。从图4.1(b)中不难看出，方法执行次数的对数与所占比例的对数满足斜率为负的线性关系，因此方法执行次数符合幂率分布规律的特征。虽然DaCapo测试集并不能代表所有的应用程序，但作为经典的测试集，上述分析结果对于实际的应用系统具有代表性。大量研究表明[99–101]，应用程序中方法的执行时间和调用次数等遵循幂率分布的统计规律，并且上述现象在Java等面向对象的应用软件中更为显著[102][103]。最新的研究成果从复杂网络理论层面进一步揭示了幂率分布广泛存在于软件系统中的原因[104]。



(a) 笛卡尔坐标



(b) 双对数坐标

图 4.1: DaCapo测试集上方法执行次数的统计分析

因此，综合应用程序的行为特征以及已有的研究成果，本章使用幂率分布模型构建方法执行次数的先验分布。

记随机变量X表示程序中被调的任意方法m总的执行次数，则$X \geq 0$。令$P(x)$表示方法m总的执行次数不小于x的概率，即

$$P(x) = P(X \geq x) \tag{4.16}$$

由于随机变量X和概率$P(x)$之间遵循幂率分布的规律，故

$$P(x) = c_0 x^{-r_0} \tag{4.17}$$

其中，c_0和r_0均为幂率分布模型中的待定参数。程序运行时，当某个方法的执行次数超过热点识别阈值时，该方法会被判定为热点方法，虚拟机对该方法的热点识别过程也随之结束。因此，热点识别仅需关注随机变量X不大于热点识别阈值的情形。理论研究和实践表明，热点识别阈值通常不超过20000。例如，对于现有的HotSpot虚拟

机，热点识别阈值默认的最大值仅为10000。此外，由幂率分布的长尾分布特性可知，当X的取值大到一定程度时，相应的概率值趋于0，几乎可以忽略。因此，本文对随机变量X设置了感兴趣的上限，记为X_{upper}，并且主要关注X的取值不超过X_{upper}时的概率。本文设置$X_{upper} = 100000$。

根据幂率分布的特性，通常$P(x)$和$P(x+1)$的值相差非常小，为了降低先验分布更新的运算代价，减少统计误差，本文进一步对随机变量X的取值范围进行了分段划分。设分段的长度为$S \in [1, X_{upper})$，划分点为x_0，x_1，x_2，x_3，...，x_{n-1}，x_n，共$n+1$个分段点。其中，$x_i = iS, i \in [0, n]$，并且$n = \lfloor X_{upper}/S \rfloor$。令

$$p(x_i) = \begin{cases} P(x_i) - P(x_{i+1}) & i \in [0, n) \\ P(x_i) & i = n \end{cases} \tag{4.18}$$

其具体含义为

$$\begin{cases} p(x_0) = P(x_0) - P(x_1) = P(x_0 \le X < x_1) & \text{表示}X \in [0, S)\text{的概率} \\ p(x_1) = P(x_1) - P(x_2) = P(x_1 \le X < x_2) & \text{表示}X \in [S, 2S)\text{的概率} \\ p(x_2) = P(x_2) - P(x_3) = P(x_2 \le X < x_3) & \text{表示}X \in [2S, 3S)\text{的概率} \\ ... & \\ p(x_{n-1}) = P(x_{n-1}) - P(x_n) = P(x_{n-1} \le X < x_n) & \text{表示}X \in [(n-1)S, nS)\text{的概率} \\ p(x_n) = P(x_n) = P(X \ge x_n) & \text{表示}X \in [nS, +\infty)\text{的概率} \end{cases}$$

上述$p(x_i)$构成了对随机变量X取值空间的划分，故$\sum_{i=0}^{n} p(x_i) = 1$。又由于对任意$i \in [0, n]$满足$p(x_i) \ge 0$，令集合$D$表示所求的先验概率分布，则

$$D = \{p(x_i) | i \in [0, n]\}$$

其中

$$\begin{cases} p(x_i) = P(x_i) - P(x_{i+1}) = c_0(x_i^{-r_0} - x_{i+1}^{-r_0}) & i \in [0, n)\text{，表示}X \in [iS, (i+1)S)\text{的概率} \\ p(x_n) = P(x_n) = c_0 x_n^{-r_0} & \text{表示}X \in [nS, +\infty)\text{的概率} \end{cases}$$

4.4.2 先验分布的更新

先验分布的更新是贝叶斯估计的关键。令更新后的概率分布为D'，则对先验分布D进行更新的一般过程可表示为

$$D' \leftarrow \varphi(D, e)$$

其中，e为观测数据，φ为先验分布的更新算子。由此可见，观察数据e的寻找以及更新算子φ的设计是先验分布更新的关键。

显然，方法已经执行的次数是更新先验分布最重要的观测数据。在任意时刻t，很容易获取方法m当前的执行次数$Execute(m,t)$。此时，对先验分布D的更新操作可记为

$$D' \leftarrow \varphi(D, Execute(m,t)) \tag{4.19}$$

4.19式的更新操作完全基于方法m在过去的时间段内已经执行的次数，不能全面反映方法m在未来的变化趋势。为了实现对程序热点的前瞻性预测，本章借鉴了文献[68]中提出的方法运行速度的概念，引入了一个前瞻性的观测数据$Foresee(m,t)$，其定义如下：

$$Foresee(m,t) = Execute(m,t) + k \cdot Speed(m,t) \tag{4.20}$$

其中，$k > 0$表示前瞻性观测的步长。$Speed(m,t)$表示方法m在时刻t的运行速度，其计算方法为

$$Speed(m,t) = \frac{\Delta Execute(m,t)}{\Delta t} \tag{4.21}$$

其中，Δt为t时刻前的一个较短的时间段，$\Delta Execute(m,t)$为方法m在Δt时间段内的执行次数。

本文对更新算子φ的设计为：1)首先将分布D中执行次数小于e的元素的概率清零；2)随后将D中所有元素的概率归一化。更新算子φ的具体实现如算法1所示。算法中第3 $\sim$ 8行寻找分布D中执行次数小于e的所有元素。第9 $\sim$ 14行将分布D中执行次数小于e的元素的概率清零，并计算概率归一化因子$nomalizer$的值。第15 $\sim$ 17行将D中所有元素的概率进行归一化。最后，返回更新后的概率分布D'。算法1的时间复杂度为$O(n)$，其中n为先验分布D中元素的个数。

至此，两个重要的观测数据和更新算子均已得到。本文依次使用方法当前已执行的次数$Execute(m,t)$和方法在未来短时间内可以预期的前瞻性执行次数$Foresee(m,t)$连续两次对先验分布D进行更新，最终得到后验分布D'如4.22式所示：

$$D' \leftarrow \varphi(\varphi(D, Execute(m,t)), Foresee(m,t)) \tag{4.22}$$

前瞻性观测数据$Foresee(m,t)$的引入加快了对程序中热点方法的识别速度，有效解决了现有热点识别技术中识别延迟较大的问题。

4.4.3　基于后验分布的估计

基于贝叶斯策略的热点识别的最后一步是估计方法的执行次数不小于热点识别阈值的概率，其核心是后验概率的计算。令$\mu(D',x)$表示方法执行次数的随机变量X在后验分布D'上取值不小于x的概率，则对D'中执行次数不小于x的所有元素的概率求和的结果即为$\mu(D',x)$的值。具体的计算方法如算法2所示。算法中第2 $\sim$ 7行寻找后验分布D'中执行次数不小于x的所有元素。第8 $\sim$ 12行对后验分布D'中执行次数不小于x的

Algorithm 1 概率分布更新算法

Input:

D, 更新前的概率分布

e, 观测数据

Output:

D', 更新后的概率分布

1: $D' \leftarrow D$;
2: $nomalizer \leftarrow 1.0$;
3: $j \leftarrow 0$;
4: **if** $e > X_{upper}$ **then**
5: $\quad j \leftarrow \lfloor X_{upper}/S \rfloor$;
6: **else**
7: $\quad j \leftarrow \lfloor e/S \rfloor$;
8: **end if**
9: **for** each $p(x_i) \in D'$ **do**
10: $\quad$ **if** $i < j$ **then**
11: $\quad\quad nomalizer \leftarrow nomalizer - p(x_i)$;
12: $\quad\quad p(x_i) \leftarrow 0$;
13: $\quad$ **end if**
14: **end for**
15: **for** each $p(x_i) \in D'$ **do**
16: $\quad p(x_i) \leftarrow p(x_i)/nomalizer$;
17: **end for**
18: **return** D';

元素的概率求和。算法2的时间复杂度也为$O(n)$，其中n为后验分布集合D'中元素的个数，亦即先验分布集合D中元素的个数。

由4.3节的分析知，热点识别要求判断不等式4.13是否成立，即判断方法m的执行次数不小于$T(m,t)$是否成立。故热点判别时应计算$X \geq T(m,t)$在后验分布D'上的概率，即$\mu(D', T(m,t))$的值。当$\mu(D', T(m,t))$足够大时才判定不等式4.13成立，相应的方法才被识别为热点方法；否则判为不成立，相应的方法被视为非热点方法。因此，贝叶斯估计的判别准则为：

$$\begin{cases} m\text{为热点方法} & \text{若}\mu(D', T(m,t)) \geq \lambda \\ m\text{为非热点方法} & \text{若}\mu(D', T(m,t)) < \lambda \end{cases}$$

Algorithm 2 后验概率计算方法

Input:

D', 后验概率分布

x, 随机变量X取值的下限

Output:

$\mu(D', x)$, 随机变量X在后验分布D'上取值不小于x的概率

```
1: μ(D', x) ← 0.0;
2: j ← 0;
3: if x > X_upper then
4:     j ← ⌊X_upper/S⌋;
5: else
6:     j ← ⌊x/S⌋;
7: end if
8: for each p(x_i) ∈ D' do
9:     if i ≥ j then
10:        μ(D', x) ← μ(D', x) + p(x_i);
11:    end if
12: end for
13: return μ(D', x);
```

其中，λ为贝叶斯估计中使用的阈值，为了保证热点识别的准确性并尽可能减小动态编译的开销，本文设置$\lambda = 0.9$。

4.4.4 算法的具体实现

基于贝叶斯估计的程序热点识别的主要步骤为：

- **步骤1**：获取先验概率分布更新所需的两个观测数据；
- **步骤2**：根据4.22式更新先验概率分布，并得到后验概率分布；
- **步骤3**：计算热点识别所需的智能阈值；
- **步骤4**：估计方法的执行次数不小于智能阈值的概率，并据此判定是否为热点方法。

上述步骤的具体实现如算法3所示。热点识别期间，虚拟机需要收集程序运行的剖面分析信息，以指导即时编译器进行更有效的编译优化。故算法3中引入了剖面分析阈值$ProfileThreshold$，规定方法被识别为热点方法前至少应执行$ProfileThreshold$次，

以确保虚拟机收集到足够的剖面分析信息。和大多数虚拟机的设计一样，本章定义方法的执行次数为方法调用次数与方法内部循环迭代次数的和，以加速对含有循环的热点方法的识别速度。算法中第1 $\sim$ 4行判断对方法m的剖面分析是否充分。第5 $\sim$ 9行将先验概率分布D更新为后验概率分布D'，并计算方法m的执行次数不小于智能阈值$N_0 + e$的概率p。第10 $\sim$ 14行判断方法m是否为热点方法。由于算法1和2的时间复杂度均为$O(n)$，其中n为先验分布D中元素的个数，故算法3总的时间复杂度也为$O(n)$。

Algorithm 3 基于贝叶斯估计的程序热点识别算法

Input:

D, 先验概率分布

m, 当前待识别的方法

e, 方法m已执行的次数

Output:

H, 布尔值(热点方法$H = True$, 非热点方法$H = False$)

1: **if** $e < ProfileThreshold$ **then**
2: $H \leftarrow False$;
3: **return** H;
4: **end if**
5: 根据4.21式，计算方法m的运行速度为$speed$;
6: 根据4.20式，计算方法m的前瞻性观测数据为$foresee$;
7: 将先验分布D更新为后验分布D': $D' \leftarrow \varphi(\varphi(D, e), foresee)$;
8: 根据4.10式，计算N_0;
9: 计算后验概率p: $p \leftarrow \mu(D', N_0 + e)$;
10: **if** $p \geq \lambda$ **then**
11: $H \leftarrow True$;
12: **else**
13: $H \leftarrow False$;
14: **end if**
15: **return** H;

热点识别算法3的触发时机包括两种情形，即方法调用时触发和循环迭代时触发。现有虚拟机系统中，方法调用的触发时机通常为单次触发，即每次方法调用都有可能触发热点识别。而循环迭代的触发时机通常为周期性触发，以避免由于循环被频繁中断而导致系统性能急剧下降。为了减小热点识别算法的运行时开销，本文对上述两种触发情形均采用了周期性的触发机制。其中，方法调用的触发周期为256，即当且仅当在方法调用时侦测到方法的执行次数为256的倍数时才触发热点识别算法。循环迭代的触发周期为1024(与HotSpot的周期相同)，即当且仅当在循环迭代时侦测到方法的执行

次数为1024的倍数时才触发热点识别算法。根据热点识别的触发周期，本文在先验分布构建时取分段划分的长度$S = 256$，即与最小的触发周期相同。

4.5 实验结果与分析

4.5.1 实验设计

4.5.1.1 实验设置

本文使用业界广泛使用的SPECjvm98[94]和DaCapo-9.12-bach[95]测试集来度量本章提出的基于贝叶斯估计的程序热点识别算法的效果。为了降低垃圾收集等对性能测试的影响，实验通过设置虚拟机参数“-Xms1G -Xmx1G”，统一使用1G大小的堆空间。实验中使用了MIPS兼容的龙芯3B1500处理器，主频900MHz，内存8G，操作系统为64位的CentOS 6.4。

本文进行了以下3组实验：

1. 传统识别法，即HotSpot虚拟机默认使用的热点识别技术；
2. 多级阈值法，即文献[97]提出热点识别技术，采用最优的两级阈值；
3. 贝叶斯估计法，即本章提出的热点识别技术。

为了尽可能减小实验误差，在度量启动性能时，每组实验均以相同的参数单独运行10次，取其算术均值作为启动性能最终的实验结果。而在度量峰值性能时，每组实验均以相同的参数连续迭代运行20次，并取最后10次结果的算术均值作为峰值性能最终的实验结果。由于不同测试项的运行时间存在较大差异，故在对不同测试项的结果求均值运算时，采用了几何平均的计算方法。所有测试项目的结果均以传统识别法的结果为基准进行了归一化处理。

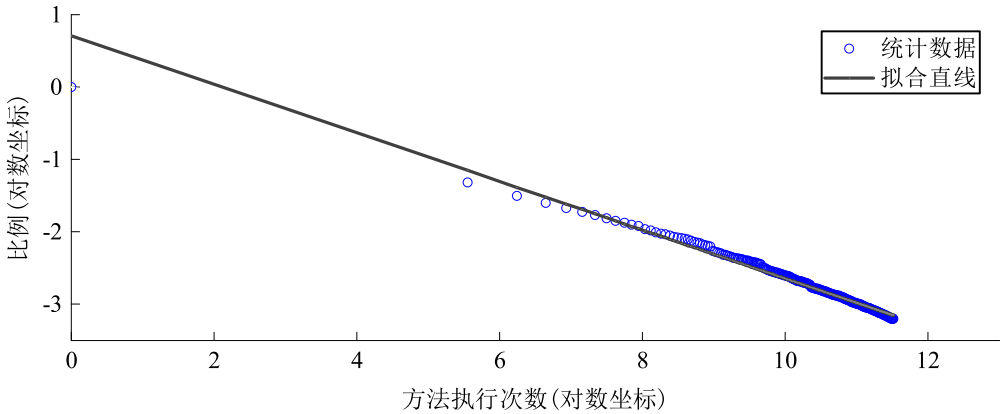
4.5.1.2 参数确定

本章提出的算法含有多个待定参数，包括幂率分布模型4.17式中的c_0和r_0、4.10式中的α和G、4.20式中的k，以及算法3中的$ProfileThreshold$。本文首先通过线性拟合对c_0和r_0进行求解。而对于其余参数则采用遗传算法搜索得到最优的参数组合。

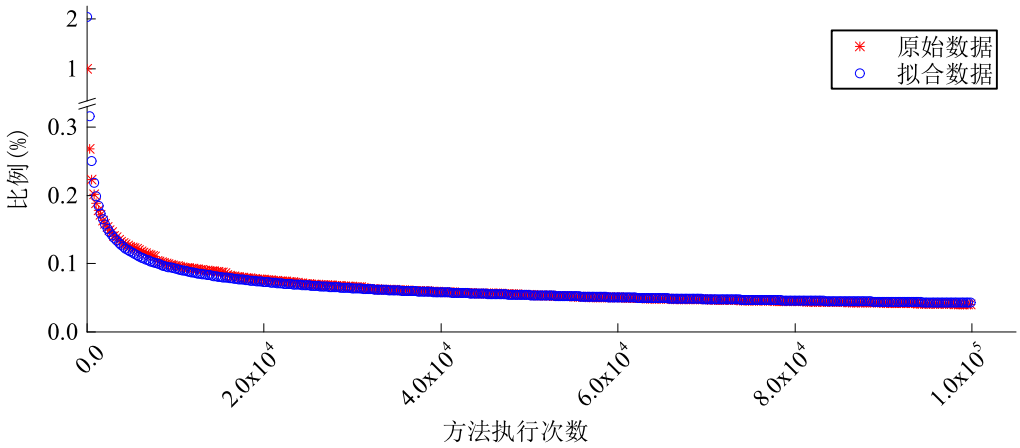
根据4.4.1节中的分析，c_0和r_0可以通过对图4.1(b)进行线性拟合得到。拟合的结果如图4.2所示，拟合所得直线的斜率为-0.3353，截距为0.7085。结合4.15式可知，$r_0 = 0.3353$，$c_0 = e^{0.7085}$。

由于α、G、k和$ProfileThreshold$四个参数的组合构成了庞大的搜索空间，通过人工试验很难找到一组较优的值。因此，本文使用遗传算法[105][106]搜索上述四个参数的最优组合。实验中使用了SPECjvm98测试集的$jess$、$javac$、$mtrt$和$jack$，以

及DaCapo测试集的fop、$jython$、$lusearch$、pmd和$tomcat$来进行遗传算法的迭代试验。选择上述测试项目的主要依据是：一方面这些项目较为全面地覆盖了数值计算、事务处理、访存和I/O等操作，对于评估系统的定点/浮点计算、访存和I/O等能力具有代表性；另一方面它们的运行时间相对较短，可以加速遗传算法迭代的收敛过程。遗传算法的优化目标为求解上述9个项目总体性能(性能的几何均值)的最大值。遗传算法重要参数的设置为：种群规模为32个，染色体的基因个数为7个，交叉率为0.8，变异率为0.1，最大进化代数为1000代。遗传算法收敛的条件为连续5代均收敛至相同的最优值或者达到最大进化代数。经过117代进化，遗传算法最终收敛。此时上述四个待定参数的值分别为$\alpha = 0.33$、$G = 10500$、$k = 40$和$ProfileThreshold = 2500$。



(a) 双对数坐标下的拟合结果



(b) 原始数据的拟合结果

图 4.2: 幂率分布模型的拟合结果

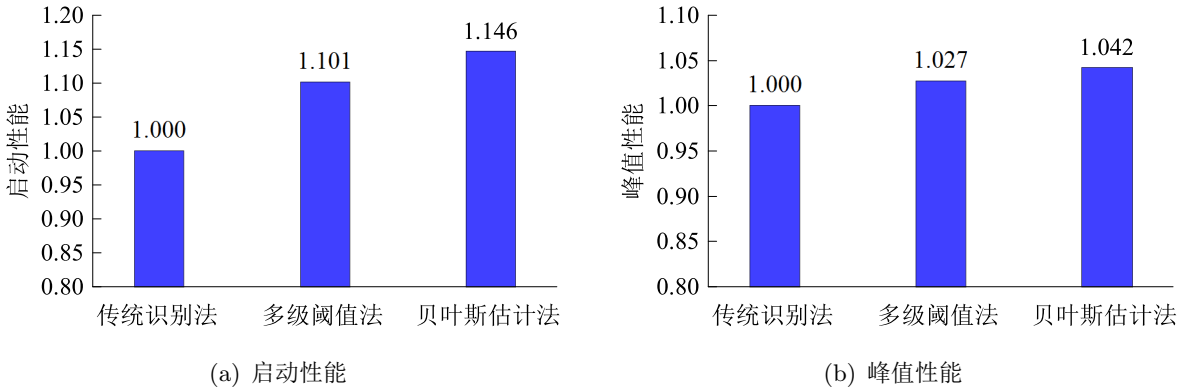
4.5.2 实验结果

图4.3展示了虚拟机在三种热点识别算法下的总体性能，包括总体的启动性能与峰值性能。从图中可以看出，贝叶斯估计法明显优于其它两种算法，在虚拟机的启动性

能和峰值性能上均获得了最高幅度的提升，提升幅度分别为14.6%和4.2%。

图4.4进一步展示了SPECjvm98和DaCapo各个测试项目在三种算法下的启动性能和峰值性能。从图4.4(a)中可以看出，贝叶斯估计法对所有项目的启动性能都有提升效果。除了*luindex*略低于多级阈值法外，贝叶斯估计法在其余项目上均获得了最高幅度的性能提升，并且在项目*jack*、*javac*、*fop*和*mtrt*上的启动性能优化效果非常明显，其提升幅度分别可达61.3%、42.5%、34.8%和19.9%。从图4.4(b)可以看出，贝叶斯估计法在所有项目上的峰值性能均有不同程度的提升。除了项目*compress*和*h2*的提升幅度略低于多级阈值法外，贝叶斯估计法在其余项目上均获得了最大的提升效果。个别项目如*javac*和*fop*的峰值性能提升幅度分别高达24.0%和12.3%。由此可见，贝叶斯估计法对虚拟机的启动性能和峰值性能均有明显的优化效果。

上述实验结果表明，相对于其它两种热点识别算法，贝叶斯估计法具有明显的优势。传统识别法和多级阈值法仅利用了单个方法的历史信息来预测程序的热点，可信度一般，热点预测效率较低，并且缺乏灵活性。而贝叶斯估计法除了利用单个方法的历史信息进行判断外，更多地是基于先验概率分布进行热点预测。先验概率分布基于对大量方法实际执行行为的统计分析而构建，能够很好地反映应用程序执行的统计规律。故基于先验信息的热点预测具有更高的可信度。此外，贝叶斯估计法还利用了更多的静态信息和运行时动态信息来辅助热点预测，其灵活性和自适应能力更好，因而能够获得更高的预测效率。

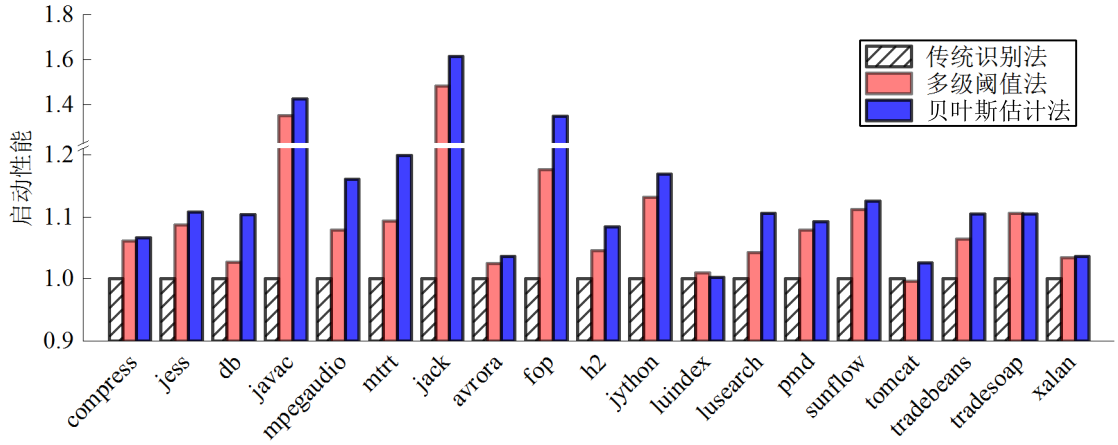


(a) 启动性能　　(b) 峰值性能

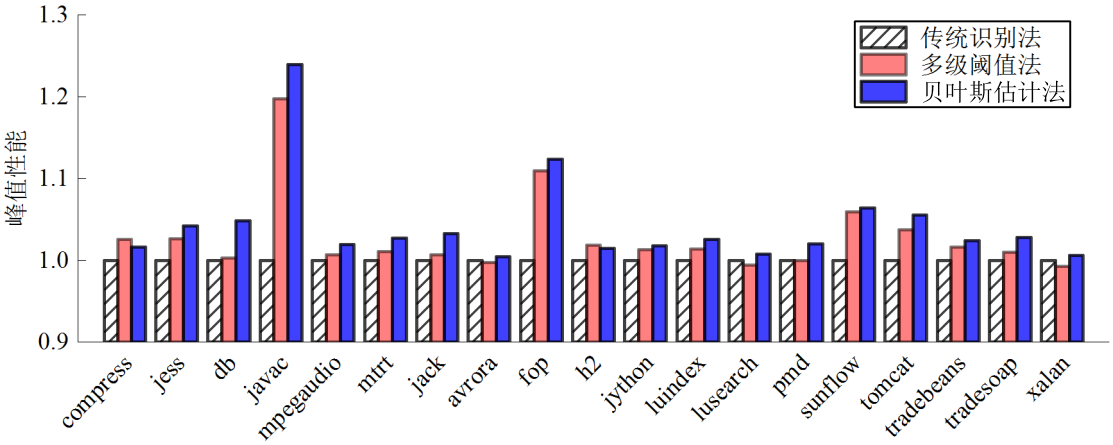
图 4.3: 虚拟机总体性能

4.5.3　实验分析

为了便于后续的实验分析，本文借鉴文献[107]中对热点方法定义的思想，规定热点方法为执行时间占程序运行时间的比例不低于0.1%的Java方法。本文使用Xprof程序分析器对所有测试项目进行剖面分析，以获得每个测试项目中的热点方法。为了进一步揭示贝叶斯估计法对虚拟机性能提升的原因，本文从热点方法的识别延迟、热点方法的编译延迟、热点识别率以及热点识别开销等方面进行了分析。



(a) 启动性能



(b) 峰值性能

图 4.4: 虚拟机在具体测试项目上的性能

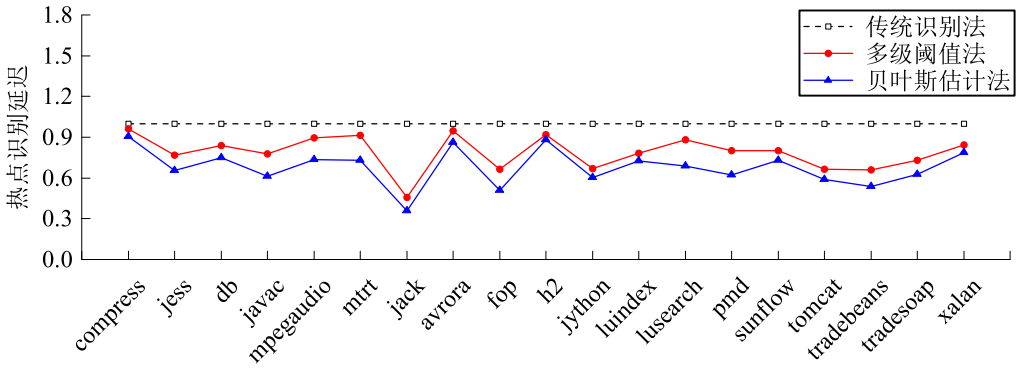


图 4.5: 热点方法的识别延迟

4.5.3.1　热点方法的识别延迟

图4.5展示了在三种热点识别算法下，各个测试项目的热点方法的识别延迟。热点方法识别延迟的定义为被识别的热点方法从程序启动到被识别的等待时间的加权平均值。其中权重系数等于各热点方法运行时间占程序运行时间的比例。从图4.5可以看出，对于所有测试项目，贝叶斯估计法均获得了最低的热点识别延迟。由于传统识别法和多级阈值法仅利用了程序过去的行为，故不能对热点方法进行提前识别。贝叶斯估计法不仅利用了程序过去的行为，还利用了未来短时间内可以预期的观测数据，因此能够对热点方法进行前瞻性识别，从而有效降低热点识别延迟。热点识别延迟越低，热点方法就可以更早地被编译，从而减小解释执行的比例，虚拟机的启动性能随之提升。因此，贝叶斯估计法可以获得最高的启动性能。

4.5.3.2　热点方法的编译延迟

图4.6展示了在三种热点识别算法下，各个测试项目的热点方法的编译延迟。热点方法的编译延迟是指所有被编译的热点方法从程序启动到被编译的等待时间的加权平均值。其中权重系数为各热点方法运行时间占程序运行时间的比例。从图4.6可以看出，对于所有测试项目，基于贝叶斯估计的热点识别法均获得了最低的热点方法编译延迟。热点方法的编译延迟越小，虚拟机便可以越早地执行编译生成的本地代码，从而大幅提升程序的性能。故编译延迟的降低是程序性能提升的直接原因。对比图4.4(a)可知，对于编译延迟下降幅度特别大的项目，如*jack*、*fop*、*javac*和*tradebeans*等，启动性能的提升幅度也较大。因此，贝叶斯估计法对程序性能的优化效果最为明显。

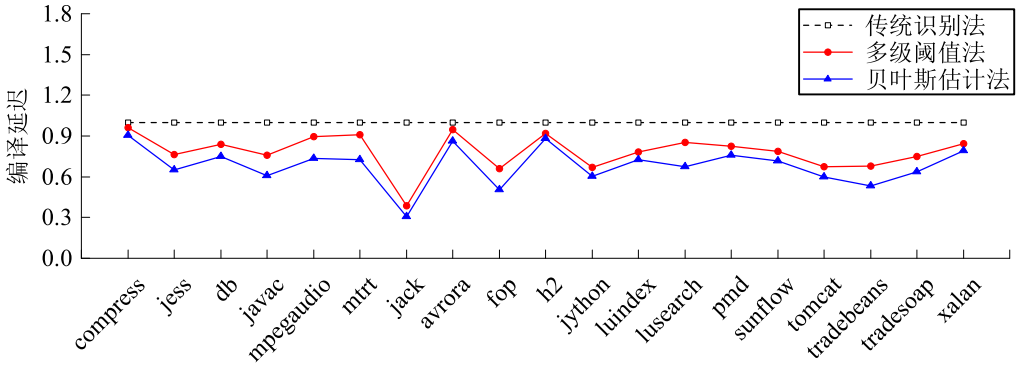


图 4.6: 热点方法的编译延迟

4.5.3.3　热点方法的识别率

图4.7展示了在三种热点识别算法下，测试项目中被识别出的热点方法占热点方法总数的比例。从图4.7可以看出，对于所有的测试项目，贝叶斯估计法获得了最高的热点识别率。热点方法识别率的提升，是虚拟机性能，尤其是峰值性能，提升的关键。

相对于传统的固定阈值技术，本文的热点识别算法采用了智能阈值策略，具有更高的灵活性、针对性以及对程序行为的自适应性，故能提高热点方法的识别率。热点方法的识别率越高，意味着虚拟机执行翻译后本地方法的比例也越大，故性能提升也越多。对比图4.4(b)可知，对于*javac*、*fop*、*sunflow*、*tradebeans*和*tradesoap*等热点识别率提升幅度较大的项目，相应的峰值性能的提升也较明显。因此，贝叶斯估计法可以获得最高的峰值性能。

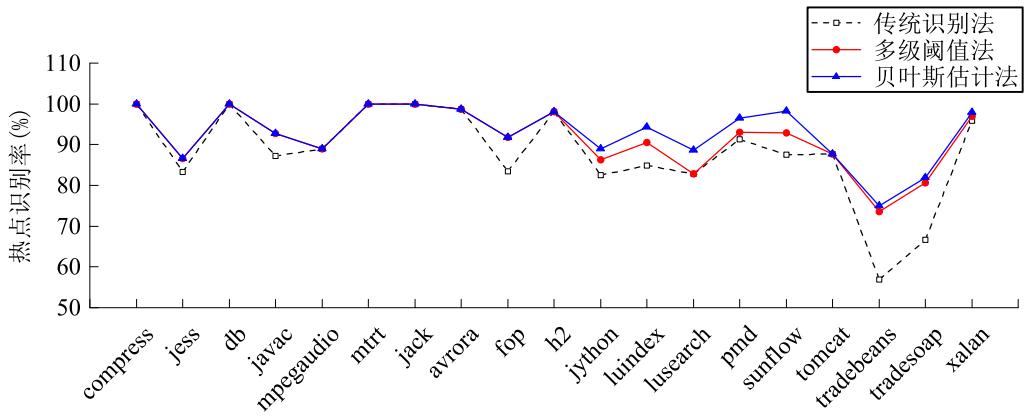


图 4.7: 热点方法识别率

4.5.3.4 热点识别的开销

为了进一步分析热点识别算法的运行时开销，本文对所有测试项目在三种不同算法下热点识别的时间进行了统计，如图4.8所示。图中的数据以传统识别法为基准进行了归一化处理。从图中可以看出，对于绝大多数的测试项目，贝叶斯估计法的开销明显小于其余两种热点识别算法。解释器在解释执行期间，当检测到某个方法满足热点识别条件时，便会向虚拟机发出编译请求。编译请求发起时，解释器首先需要暂停解释执行，随后调用虚拟机的运行时服务例程请求即时编译器进行编译。虚拟机接收到编译请求后，对于新的请求，则将所对应的方法加入编译队列中等待编译；而对于曾经接收并处理过的请求则予以忽略。编译请求处理完毕后解释器再恢复解释执行。上述编译请求发起、接收和处理的过程代价较高，是构成热点识别开销最主要的部分。图4.9中统计了在程序运行期间，采用不同的热点识别算法，虚拟机接收到的编译请求的数量。图中的数据以传统识别法为基准进行了归一化处理。从图4.8和图4.9中可以发现，热点识别开销曲线和编译请求数目曲线的走势高度一致，表明热点识别的开销和编译请求的数目具有很强的正相关性。本文的算法采用了周期性的热点识别触发机制，大幅减少了解释器向虚拟机发出编译请求的次数，尤其是重复的编译请求次数，故能充分降低热点识别的运行时开销。

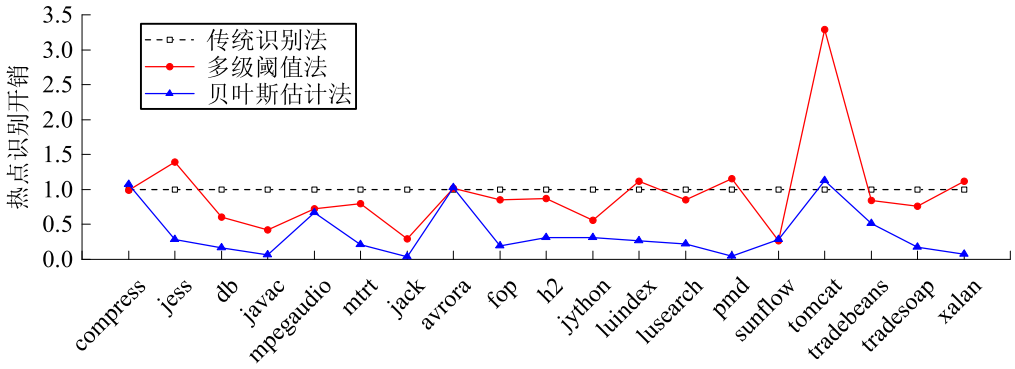


图 4.8: 热点识别开销

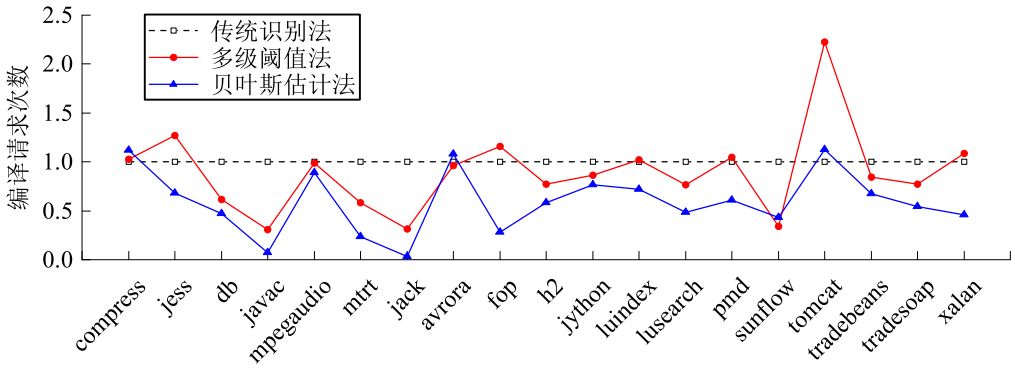


图 4.9: 编译请求次数

4.6　本章小结

热点识别是影响虚拟机性能的关键因素。本章首先设计了基于智能阈值的热点识别模型。该模型根据方法的静态特征和系统当前的运行状态实时计算智能阈值的值，可以提高热点识别的灵活性以及对程序行为的自适应能力。随后，以该模型为基础，本章进一步提出了一种基于贝叶斯估计的程序热点识别算法。该算法通过贝叶斯估计能够更为合理高效地预测方法在程序中的执行次数，实现了对程序热点的前瞻性识别。本章提出的热点识别算法可以显著降低热点识别的延迟，从而大幅降低热点方法的编译延迟，最终有效提升系统的性能。实验结果表明，本章的算法使得系统的启动性能平均提升14.6%，最高提升幅度高达61.3%；峰值性能平均提升4.2%，最高提升幅度高达24.0%。本章的算法通用性强，实现代价低，很容易推广应用于其它虚拟机系统，具有重要的实际意义。

第五章　编译队列监视下的高性能动态编译调度算法

5.1　引言

动态编译是决定虚拟机性能最关键的因素，一直是虚拟机研究的重点[25][26][61]。动态编译系统的性能优化是一个多维度的问题。在设计和优化动态编译系统时，“编译对象”和“编译时机”是首先需要解决的两个关键问题[13]。“编译对象”问题是关于如何确定程序中应当被编译的单元(方法、函数或者执行踪迹等)的问题，即上一章所研究的程序热点识别问题。而“编译时机”问题又被称为“动态编译调度”问题[69]，主要是关于如何确定程序中已经识别出的程序热点的编译顺序问题。在过去相当长的一段时期内，人们对动态编译优化的研究主要集中在解决“编译对象”问题上。例如，著名的选择性编译理论[12]便是一种十分有效的解决“编译对象”问题的方案。但是，由于对动态编译行为缺乏较为全面、系统的认识和了解，“编译时机”问题长期被人们所忽略，导致该问题至今都没有一个令人满意的解决方案，严重制约虚拟机性能的发展。近年来的研究发现[13][67]，不合理的编译顺序会导致热点方法在编译队列中等待编译的排队延迟较大，是造成虚拟机性能较低的重要原因。目前，越来越多的人开始认识到优化动态编译的顺序对于提升虚拟机性能的重要性。故动态编译调度迅速成为虚拟机优化研究的焦点[13][67][68]。

本章研究了动态编译调度的优化技术，对“编译时机”问题进行了较为系统地探索。首先，本章对动态编译系统进行了较为全面系统地分析，揭示了动态编译行为的特征，从而确定了编译调度的优化目标，即加快响应速度、降低编译队列的平均排队延迟(即热点方法在编译队列中的平均等待时间)以及减少运行时开销。根据上述优化目标，本章随后提出了编译队列监视下的高性能编译调度算法，包括收益代价分析的调度模型和自适应的编译队列监视机制。收益代价分析的调度模型既能提升系统对外界的响应速度，又能降低编译队列的平均排队延迟。调度期间的编译队列监视可以自适应地延迟编译队列中非活跃方法的编译，并且选择性地将它们从编译队列中删除，从而有效降低系统的运行时开销，并提高动态编译的效率。

本章其余部分组织如下：5.2节对本章研究内容的背景和相关工作进行了介绍；5.3节揭示了动态编译行为的特征，并在此基础上确定了编译调度的优化目标；5.4节提出了编译队列监视下的高性能动态编译调度算法；5.5节对本章算法进行了实验，从编译队列的平均排队延迟、编译队列监视机制的有效性和编译调度开销等方面进行了分析，展示了本章算法对提升虚拟机性能的有效性；最后对本章进行了小结。

5.2 相关工作

20世纪90年代，为了提升虚拟机的性能，Hölzle等人[26]在Self-93虚拟机中设计并实现了著名的分层编译系统。该系统包含一个快速的、非优化的JIT编译器和一个慢速的、优化的JIT编译器。非优化的编译器编译速度快，但生成的机器代码质量不高；优化的编译器能生成高质量的机器代码，但是编译速度较慢。程序中所有的方法首次运行时，均由非优化的编译器迅速进行编译，而优化的编译器仅编译程序中频繁执行的方法。虽然分层编译系统能有效提升虚拟机性能，但该方法要求系统中包含多个JIT编译器，并且还需要设计一系列复杂的控制和状态转换算法，设计难度很大，实现代价很高[108]。

文献[66][30]提出了一种相对简单的虚拟机优化技术。其主要思想是提升编译线程的优先级，使得编译线程优先被操作系统调度。文献[30]还规定了在多线程环境下编译线程对CPU占用率的下限，优先满足编译线程对系统资源的需求，故能加快编译的速度。然而该方法总是将系统资源优先分配给编译线程，在编译负荷较大(如系统启动或程序行为突变)时，应用程序本身的执行将受到抑制。此外，由于该方法还要求操作系统内核提供特定的线程调度支持，缺乏通用性，因而其适用范围受到限制。

近年来，有人提出通过动态编译调度来优化虚拟机性能的思路[13][67][68]。Kulkarni等人的研究表明，编译队列的排队延迟较大是导致虚拟机性能低下的重要原因[13][67]，并指出动态编译调度是解决上述问题的有效途径。此后，对动态编译调度的探索和研究逐渐增多。不久，Stadler等人提出了基于方法运行速度的动态编译调度算法[68]。该算法实时计算程序中方法的运行速度，优先选择运行速度较大的方法进行编译。Stadler等人的方法虽然有助于提高虚拟机对外界的响应速度，却忽略了对编译队列平均排队延迟的优化，未能充分发挥动态编译调度优化的潜能。

2014年，Ding等人首次从理论上证明了动态编译调度是一个NP-完全的问题[69]。该项研究成果表明，最优编译顺序的在线求解是不可能的。因此，实际系统中的编译调度通常借助一系列启发式规则获取一个近似解。为了进一步估计动态编译调度可能的性能优化空间，他们设计了一个名为*Initialization, Appending* and *Replacement*(IAR)的启发式调度算法。该算法要求事先收集程序中大量的运行时信息，并利用这些信息对相同程序在相同输入条件下的后续运行实施编译调度。IAR动态编译调度算法的实验结果表明，编译调度可以显著提升系统的性能。由于IAR算法要求在程序正式运行前收集大量的运行时信息，故该算法不能用于在线使用。同时，相应的调度策略也仅适用于特定的应用程序，并且对应用程序的输入十分敏感，因而不具备通用性。因此，该算法的设计主要是为了证明编译调度对于提升系统性能的有效性，对于实际的应用系统并不适用。

自虚拟机诞生以来，人们已经提出了许多优化方案。但以“编译时机”问题为代表的大量问题依然有待解决。其中，一个十分重要的原因是人们对动态编译行为的特点

缺乏全面、系统的认识和了解，从而导致人们对现有动态编译系统中存在的缺陷认识不够。针对上述研究工作的不足，本章对动态编译行为的特点进行了较为全面、系统地分析，并在此基础上提出了编译队列监视下的高性能动态编译调度算法。

5.3　动态编译的特征与优化目标

本节以HotSpot虚拟机为例，揭示了动态编译的关键特征，并在此基础上提出了动态编译调度的优化目标。

5.3.1　动态编译的特征

本节利用DaCapo测试集[95]对动态编译的特征进行了深入分析，以更好地优化动态编译系统。现将动态编译的关键特征总结如下。

特征1：热点方法识别具有爆发性特征。图5.1展示了每10ms新识别的热点方法的数量。从图中可以看出，热点方法识别数量的分布具有明显的爆发性特征。程序行为的易变性是导致热点方法识别具有爆发性的主要原因。当程序的行为发生变化时，许多不同的方法几乎同时会被调用，导致这些方法的调用次数几乎同时超过热点识别的阈值。于是在某一瞬间，虚拟机可能同时识别出多个热点方法。所有被识别的热点方法需要尽快被编译，以迅速适应不断变化的外部环境。但是，受到底层硬件能力的限制，虚拟机能够立即编译的方法数量十分有限。因此，系统对外界的响应能力很容易受到爆发性热点识别的影响。

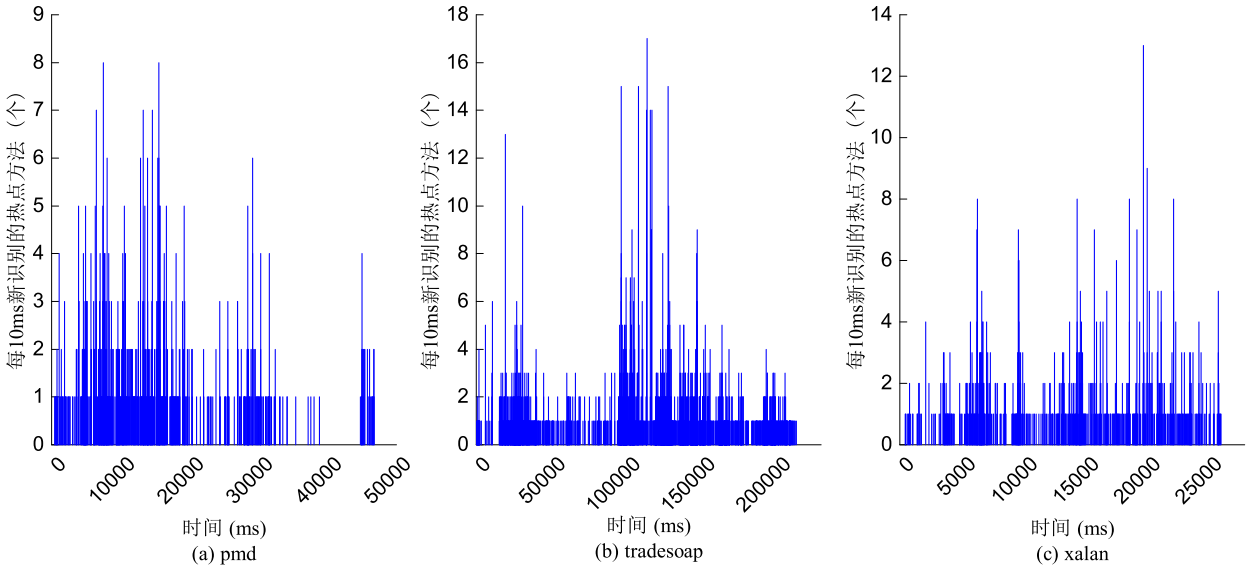


图 5.1: 每10ms内新识别的热点方法数目

特征2：热点方法在编译队列中的排队延迟可能相当大。图5.2展示了热点方法在编译队列中的排队延迟随时间的变化。从图中可以看出，对于*pmd*、*tradesoap*和*xalan*等测试项目，热点方法在编译队列的排队延迟占执行总时间的比例可高达20%以上。

图5.3展示了编译队列中热点方法的数量随时间的变化。虽然热点方法在编译队列中的排队延迟和编译队列长度之间具体的数量关系尚需进一步探索，但是，一个普遍的现象是排队延迟会随着编译队列长度的增加而变得更大。由于新识别的热点方法被添加到编译队列的尾部，当且仅当编译队列中前面所有的方法编译完成后，新识别的热点方法才能开始编译。因此，热点方法在编译队列中的排队延迟会随着编译队列长度的增加而增大。研究表明，热点方法在编译队列中的排队延迟过大是导致系统性能较低的重要原因。

特征3：编译队列中的部分方法在编译后可能变为僵尸方法。表5.1展示了僵尸方法占所有被编译的方法的比例。对于程序中的某个方法，该方法被称为“僵尸方法”，当且仅当 1）该方法已经被编译；且 2）该方法被编译后执行的次数为0。由于热点识别缺乏足够的全局信息，僵尸方法的出现难以避免。表5.1中的数据表明，僵尸方法所占的比例平均可达22.26%，最高为40.29%(*pmd*)。由于动态编译需要耗费计算和访存等大量的系统资源，并且对僵尸方法的编译不能获得任何收益，故对僵尸方法的编译会在某种程度上损害系统的性能。此外，对僵尸方法的编译还会进一步延迟虚拟机对程序中热点方法的编译。

表 5.1: 僵尸方法的比例

测试项目	僵尸方法的数目(个)	被编译方法的数目(个)	比例(%)
avrora	92	379	24.27
fop	123	506	24.31
h2	73	863	8.46
jython	404	1480	27.30
luindex	47	241	19.50
lusearch	125	350	35.71
pmd	359	891	40.29
sunflow	91	262	34.73
tomcat	201	766	26.24
tradebeans	96	699	13.73
tradesoap	97	703	13.80
xalan	119	530	22.45

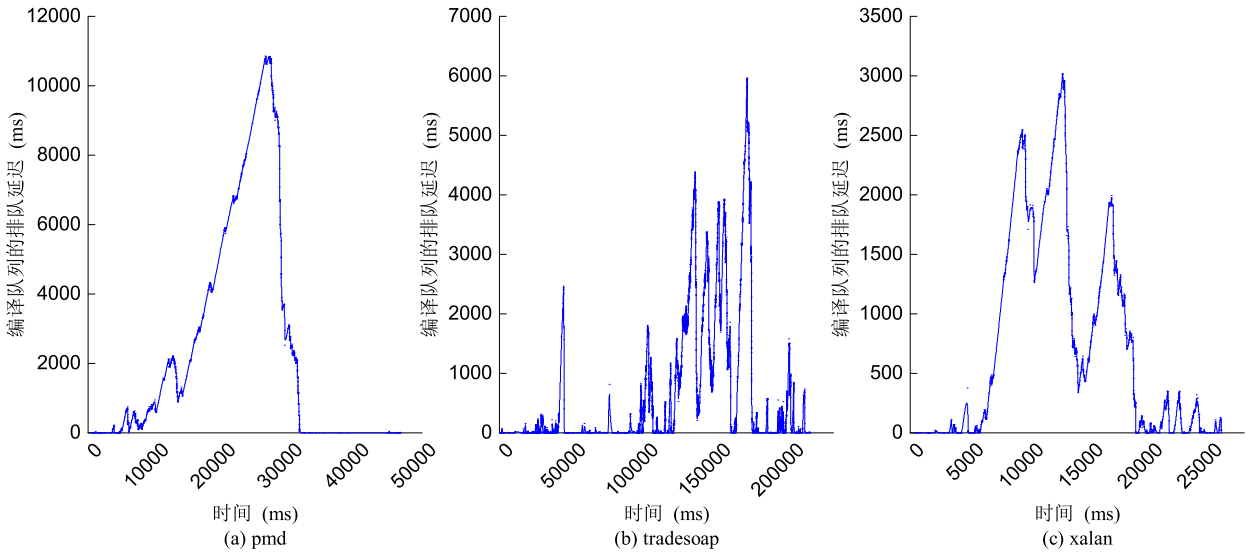


图 5.2: 热点方法在编译队列中的排队延迟随时间的变化

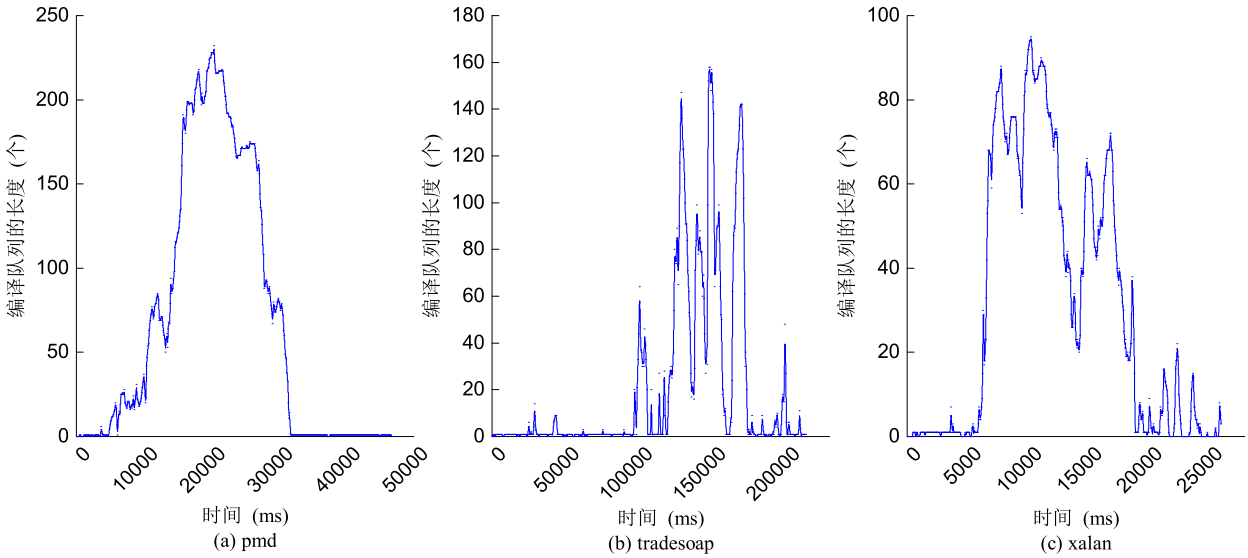


图 5.3: 编译队列长度随时间的变化

5.3.2　动态编译的优化目标

5.3.1节中揭示的动态编译的特征对高性能虚拟机系统的设计提出了巨大的挑战。为了有效应对这些挑战，本节针对上述特征提出了虚拟机设计和优化应当重点考虑的三大目标。

1. 优先编译对性能影响较大的方法，以应对热点识别的爆发性。通过这种方式，即使在硬件能力十分有限的条件下，系统对外部变化的响应时间也能被最大限度地

缩短，从而提升系统对外界的响应能力。

2. 减少热点方法在编译队列中的排队延迟，以尽可能快地编译更多的方法。通过这种方式，程序可以尽早执行编译后生成的、效率更高的本地方法，从而提升系统的性能。

3. 减少动态编译的运行时开销，尤其是僵尸方法的比例。通过这种方式，虚拟机可以减少对收益较低的方法的编译，从而减少系统的运行时开销，并提高动态编译效率。

上述优化目标均可通过动态编译调度来实现。目标1可以通过在调度时对重要的方法赋予更高的调度优先级来实现。目标2可以通过在调度时降低热点方法在编译队列中的平均排队延迟来实现。目标3可以通过在调度时建立一个在线的运行时反馈机制来实现，该机制可以侦测到非活跃的方法，并且有选择性地将它们从编译队列中删除。因此，设计一个融合了上述三个目标的动态编译调度算法是提升系统性能的关键。

5.4 编译队列监视下的高性能动态编译调度算法

为了实现5.3.2节的优化目标，本节首先提出了一种收益代价分析的调度模型，以提升系统的响应能力，并且降低热点方法在编译队列中的平均排队延迟。随后，又进一步设计了一种编译队列监视机制，以降低动态编译的运行时开销。

5.4.1 收益代价分析的调度模型

设在t时刻，编译队列中共有n个等待编译的方法，分别记为$m_1, m_2, ..., m_n$。对任意的一个方法$m_k(k \in [1, n])$，设其对程序性能影响的重要程度为$h_t(k)$。方法的重要性越高，则编译该方法所带来的性能提升也越大。因此，$h_t(k)$可以视为在t时刻编译方法m_k的收益。假定在t时刻选择方法m_k进行编译，编译队列中其余$n-1$个方法的编译均会被延迟，延迟的时间记为$W_k(W_k \geq 0)$。显然，上述编译延迟将对系统的性能产生负面影响。对方法$m_i(i \in [1, n], i \neq k)$而言，其重要程度$h_t(i)$越大，则编译延迟所造成的损失也越大。记函数$l_t(k)$表示$t$时刻选择编译方法$m_k$产生的代价，则$l_t(k)$可表示为：

$$l_t(k) = W_k \cdot \left[\sum_{i=1}^{k-1} h_t(i) + \sum_{i=k+1}^{n} h_t(i) \right] \tag{5.1}$$

在多线程并发编译的条件下，对于延迟时间W_k的确定非常困难。记方法m_k所需的编译时间为C_k。显然，W_k的上限为C_k。为了降低模型的复杂度，本文使用C_k来估计W_k的值。因此，$l_t(k)$可以由5.2式来估计。此时$l_t(k)$表示t时刻选择编译方法m_k可能造成的性能损失的最大值。

$$l_t(k) = C_k \cdot \left[\sum_{i=1}^{k-1} h_t(i) + \sum_{i=k+1}^{n} h_t(i) \right] \tag{5.2}$$

于是，在选择某个方法进行编译时，需要对其所带来的收益和所产生的代价进行权衡。因此，调度时方法m_k的优先级函数被定义为：

$$y(k) = \alpha \cdot h_t(k) - (1-\alpha) \cdot l_t(k) \tag{5.3}$$

其中，α为可调参数，且$\alpha \in [0,1]$。则t时刻应选择编译的最优方法m_K的判别准则为：

$$K = \underset{k}{\operatorname{argmax}} \{y(k)\} \tag{5.4}$$

而

$$\begin{aligned} y(k) &= \alpha \cdot h_t(k) - (1-\alpha) \cdot l_t(k) \\ &= \alpha \cdot h_t(k) - (1-\alpha) \cdot C_k \cdot \left[\sum_{i=1}^{k-1} h_t(i) + \sum_{i=k+1}^{n} h_t(i) \right] \\ &= \alpha \cdot h_t(k) - (1-\alpha) \cdot C_k \cdot \left[\sum_{i=1}^{n} h_t(i) - h_t(k) \right] \end{aligned} \tag{5.5}$$

令

$$H(t) = \sum_{i=1}^{n} h_t(i) \tag{5.6}$$

则$H(t)$在时刻t为一常量，且

$$y(k) = \alpha \cdot h_t(k) - (1-\alpha) \cdot C_k \cdot [H(t) - h_t(k)] \tag{5.7}$$

将$y(k)$对$h_t(k)$求偏导，得

$$\frac{\partial y(k)}{\partial h_t(k)} = \alpha + (1-\alpha) \cdot C_k > 0 \tag{5.8}$$

将$y(k)$对C_k求偏导，得

$$\begin{aligned} \frac{\partial y(k)}{\partial C_k} &= -(1-\alpha) \cdot [H(t) - h_t(k)] \\ &= -(1-\alpha) \cdot \left[\sum_{i=1}^{k-1} h_t(i) + \sum_{i=k+1}^{n} h_t(i) \right] \leq 0 \end{aligned} \tag{5.9}$$

当且仅当$\alpha = 1$时，不等式5.9中的等号成立。

5.8式和5.9式表明$y(k)$随$h_t(k)$单调递增，且随C_k单调递减。因此，在本文的编译调度模型中，重要程度较高、所需编译时间较少的方法将被优先编译。由于该模型在确定编译的先后顺序时，综合考虑了编译的收益和可能导致的性能损失，故该模型被称为收益代价分析的调度模型。

5.4.2 调度模型的实现

在t时刻，方法m_k对性能影响的重要程度$h_t(k)$与程序当前及未来一段时间的行为密切相关。至今对$h_t(k)$的估计都是一个开放性问题。一般地，在$[t,\ t+\Delta t]$的时间段内(其中$\Delta t > 0$)，方法m_k执行次数越多，其对性能的影响也越大。此外，在执行次数相同的前提下，方法单次解释执行的开销越大，其对性能的影响也越大。令$s(m_k)$表示方法m_k在$[t,\ t+\Delta t]$时间段内执行的次数，$\gamma(m_k)$表示方法m_k单次解释执行的开销，则$h_t(k)$可表示为

$$h_t(k) = s(m_k) \cdot \gamma(m_k) \tag{5.10}$$

$s(m_k)$的值可以通过方法在$[t-\Delta t,\ t]$时间段内的执行次数(即方法m_k的运行速度[68]，记为$speed(m_k)$)进行估计，故$h_t(k)$可进一步表示为

$$h_t(k) = speed(m_k) \cdot \gamma(m_k) \tag{5.11}$$

$speed(m_k)$的定义为

$$speed(m_k) = \frac{\Delta execute(m_k)}{\Delta t} \tag{5.12}$$

其中，Δt为t时刻前的一个较短的时间段，且$\Delta execute(m_k)$为

$$\Delta execute(m_k) = \Delta ic(m_k) + \frac{1}{8}\Delta lc(m_k) \tag{5.13}$$

其中，$\Delta ic(m_k)$表示方法m_k在Δt的时间段内被调用的次数，$\Delta lc(m_k)$表示方法m_k在Δt的时间段中内部循环迭代的次数。

而$\gamma(m_k)$的值可通过对方法m_k进行静态分析得到，其值可表示为

$$\gamma(m_k) = \sum_{i \in B(m_k)} w(i) \tag{5.14}$$

其中$B(m_k)$表示方法m_k所包含的字节码的集合，$w(i)$表示字节码i的权重。每个字节码的权重可以通过对该字节码在特定平台上解释执行的时间进行度量或估算得出。

文献[96]的研究表明方法的编译时间与方法的大小存在线性关系。一般地，方法越大，所需的编译时间也越长。同时，由于现代编译器对方法中循环结构的深度优化也会占用大量系统资源，其内部循环越多，编译耗时也越大。因此，本节使用方法的大小和所包含的循环数目来估计所需的编译时间，即令

$$C_k = [loop_num(m_k) + 1] \cdot size(m_k) \tag{5.15}$$

其中$loop_num(m_k)$表示方法m_k中循环结构的数目。

至此，将5.11和5.15 式代入5.7式中，编译调度的优先级函数$y(k)$可以表示为

$$\begin{aligned} y(k) = {} & \alpha \cdot speed(m_k) \cdot \gamma(m_k) \\ & - (1-\alpha) \cdot [loop_num(m_k) + 1] \cdot size(m_k) \cdot H(t) \\ & + (1-\alpha) \cdot [loop_num(m_k) + 1] \cdot size(m_k) \cdot speed(m_k) \cdot \gamma(m_k) \end{aligned} \tag{5.16}$$

其中α为可调参数，满足$\alpha \in [0,1]$，且

$$H(t) = \sum_{i=1}^{n} speed(m_i) \cdot \gamma(m_i) \tag{5.17}$$

于是，t时刻动态编译调度问题可转换为如下的一个优化问题：从编译队列中寻找一个方法，使得5.16式中优先级函数的值最大。求解该优化问题的具体步骤如下：

- **步骤1**：对编译队列中的每个方法按5.12式计算其运行速度；
- **步骤2**：遍历整个编译队列，按5.17式计算$H(t)$的值；
- **步骤3**：对编译队列中的每个方法按5.16式计算调度的优先级；
- **步骤4**：遍历整个编译队列，选择优先级最大的方法进行编译。

从上述求解步骤不难看出，对于长度为n的编译队列，该算法的时间复杂度为$O(n)$。

5.4.3　自适应的编译队列监视机制

为了减少动态编译的运行时开销，尤其是降低僵尸方法的比例，本节提出了一种自适应的编译队列监视机制。该机制能够自适应地侦测编译队列中非活跃的方法，并且有选择性地将它们从编译队列中删除。本节设计的编译队列监视机制主要包括三个步骤：1）计算编译队列中方法的活跃性；2）标记编译队列中非活跃的方法；3）删除编译队列中的疑似僵尸方法。

1. **计算编译队列中方法的活跃性。**

   首先，对编译队列中所有的方法，计算其活跃性。令$activeness(m)$表示方法m的活跃性。方法的活跃性和程序未来的行为密切相关，很难实时地、精确地计算。因此，实际系统通常借助程序过去的运行时信息来估计方法的活跃性。本章联合使用方法的局部信息和全局信息来对方法的活跃性进行估计。对于方法m，其活跃性的计算方法如下

   $$activeness(m) = [speed(m) + 1] \cdot [\frac{execute(m)}{HotThreshold} + 1] \tag{5.18}$$

   其中$speed(m)$表示方法m的运行速度，用于表征方法m的局部活跃性。$execute(m)$用于表征方法m的全局活跃性，其定义为

   $$execute(m) = ic(m) + \frac{1}{8} lc(m) \tag{5.19}$$

   其中$ic(m)$和$lc(m)$分别表示方法m的调用次数和内部循环迭代次数。$HotThreshold$为热点识别的阈值。

2. **标记编译队列中非活跃的方法。**

   由于对编译队列中非活跃方法的编译所获得的性能提升效果十分微弱，故它们的编译可以适当延迟，以提升虚拟机动态编译的收益率。因此，在编译调度期间，应当对编译队列中非活跃的方法进行过滤。为此，本节引入阈值$ActiveThreshold$，用于判别编译队列中非活跃的方法。在时刻t，对于编译队列中的方法m，若满足$activeness(m) < ActiveThreshold$，则称方法$m$是非活跃的，并标记方法$m$为非活跃的方法。编译队列中被标记为非活跃的方法不会参与编译调度，除非编译队列中所有的方法均被标记为非活跃的。

3. **删除编译队列中的疑似僵尸方法。**

   虽然编译队列中非活跃的方法有很高的风险变成僵尸方法，但由于程序行为的易变性，立即从编译队列中删除侦测到的非活跃方法是不明智的。因此，只有当编译队列中的某个方法长时间处于非活跃状态时，才将该方法从编译队列中删除。若编译队列中某个方法长期处于非活跃状态，则称其为“疑似僵尸方法”。为了识别疑似僵尸方法，本节为编译队列中的任意方法m设计了一个名为$victim(m)$的计数器。当计数器$victim(m)$的值超过阈值$VictimThreshold$时，方法m则被判定为疑似僵尸方法，并且立即将它从编译队列中删除。在编译调度期间，计数器$victim(m)$的更新规则如下：

   - $victim(m) = victim(m) + 1$

     如果方法m被标记为非活跃方法并且$speed(m) = 0$。

   - $victim(m) = 0$

     如果方法m未被标记为非活跃方法。

5.4.4 算法的具体实现

编译队列监视下的高性能动态编译调度的主要步骤为：

- **步骤1**：遍历编译队列Q，对其中的非活跃方法进行标记，同时删除Q中所有疑似僵尸方法；

- **步骤3**：对Q中所有未被标记的活跃方法，按本章的调度模型计算其调度优先级；

- **步骤4**：对Q中所有未被标记的活跃方法，选择其中优先级最大的方法进行编译，并将该方法从Q中移除。

算法的具体实现如算法4所示。其中，Q'表示一个临时队列，用于缓存编译队列监视期间Q中未被标记的活跃方法。第1 $\sim$ 16行将编译队列Q中的活跃方法缓存到Q'中，

Algorithm 4 编译队列监视下的高性能动态编译调度算法

Input: Q, 编译队列

Output: $Target$, 最终被调度的方法

```
1:  Q' ← ∅;
2:  for 每个方法m ∈ Q do
3:      根据5.12式计算方法m的运行速度speed(m);
4:      根据5.18式计算方法m的活跃性activeness(m);
5:      if activeness(m) ≥ ActiveThreshold then
6:          victim(m) ← 0;
7:          Q' ← Q' ∪ {m};
8:      else
9:          if speed(m) = 0 then
10:             victim(m) ← victim(m) + 1;
11:         end if
12:         if victim(m) > VictimThreshold then
13:             Q ← Q − {m};
14:         end if
15:     end if
16: end for
17:
18: if Q' = ∅ then
19:     Target ←(Q中运行速度最高的方法) 或者 (NULL 如果Q = ∅);
20:     return Target;
21: end if
22:
23: H(t) ← 0;
24: for 每个方法m ∈ Q' do
25:     H(t) ← H(t) + speed(m) · γ(m);
26: end for
27: for 每个方法m ∈ Q' do
28:     根据5.16式计算方法m的调度优先级;
29: end for
30: Target ← Q'中优先级最高的方法;
31: return Target;
```

并删除Q中所有疑似僵尸方法。第18 ~ 21行处理Q中所有方法均为非活跃方法的特殊情形。此时，若Q非空，则返回Q中运行速度最大的方法，否则返回空。第23 ~ 30行按本章提出的调度模型实施调度。对于包含n个方法的编译队列，由于编译队列监视和调度模型中优先级计算的时间复杂度均为$O(n)$，故编译队列监视下的高性能动态编译调度算法总的时间复杂度也为$O(n)$。

5.5 实验结果与分析

5.5.1 实验设计

5.5.1.1 实验设置

本文使用业界广泛使用的SPECjvm98[94]和DaCapo-9.12-bach[95]来度量本章算法的效果。虚拟机使用默认的两个并行编译线程的配置。同时，为了降低垃圾收集等对性能测试的影响，实验通过设置虚拟机参数“-Xms1G -Xmx1G”，统一使用1G大小的堆空间。实验中使用了MIPS兼容的龙芯3B1500处理器，主频900MHz，内存8G，操作系统为64位的CentOS 6.4。

本文进行了以下3组实验：

1. 传统调度，即HotSpot虚拟机中默认使用的“先来先服务”的调度算法；
2. Speed调度，即文献[68]提出的调度算法，优先选择运行速度高的方法进行编译；
3. 本文调度，即本章所提出的调度算法。

为了尽可能减小实验误差，在度量启动性能时，每组实验均以相同的参数单独运行10次，取其算术均值作为启动性能最终的实验结果。而在度量峰值性能时，每组实验均以相同的参数连续迭代运行20次，并取最后10次结果的算术均值作为峰值性能最终的实验结果。由于不同测试项的运行时间存在较大差异，故在对不同测试项的结果求均值运算时，采用了几何平均的计算方法。所有测试项目的结果均以传统调度的结果为基准进行了归一化处理。

5.5.1.2 参数确定

算法引入了三个待定的参数，分别为α、$ActiveThreshold$和$VictimThreshold$。由于上述几个参数的组合构成了一个庞大的搜索空间，通过人工试验很难找到一组较优的值。因此，本文使用了遗传算法[105][106]来搜索上述三个参数的最优值。遗传算法的具体设置与4.5.1.2节一致。遗传算法收敛时，三个待定参数的值分别为$\alpha = 0.48$，$ActiveThreshold = 6$和$VictimThreshold = 3$。

5.5.2　实验结果

图5.4展示了在不同的编译调度算法下，虚拟机总体的启动性能。由图5.4可知，本文调度算法的性能提升幅度高达8.9%，而Speed调度的性能提升效果仅为3.7%。

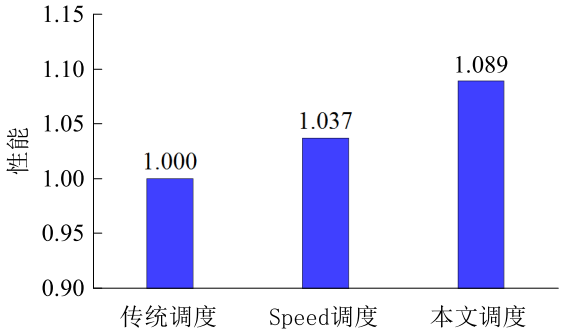


图 5.4: 虚拟机总体的启动性能

图5.5展示了在不同的调度算法下，虚拟机总体的峰值性能。从图中很容易看出，即使采用不同的调度算法，虚拟机总体的峰值性能几乎相同。故编译调度对虚拟机峰值性能的影响十分微弱。由于当峰值性能测量时，程序中所有重要的方法均已被编译。因此，编译的顺序几乎不会影响虚拟机的峰值性能。

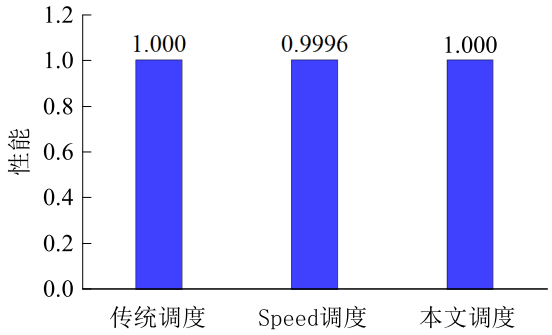


图 5.5: 虚拟机总体的峰值性能

图5.6展示了使用了不同的调度算法，虚拟机在SPECjvm98和DaCapo测试集中每个项目的性能。显然，对于绝大多数的测试项目，本文编译调度获得了最高的性能提升幅度。并且该算法对于测试项目*javac* (41.8%)，*pmd* (38.2%)，*xalan* (35.1%)，*tradesoap* (11.9%)和*lusearch* (10.0%)的性能提升效果尤其明显。由于同时考虑了对系统的响应速度和编译队列的平均排队延迟进行优化，故本文调度算法对程序性能的提升十分有效。对于*luindex*，*tomcat*和*tradebeans*等测试项目，由于解释执行占相当大的比例，故由编译调度带来的性能提升幅度相对较小。而对于*compress*和*db*等测试项目，编译调度对性能几乎没有影响。这是由于在编译调度期间的任意时刻，编译队列

中等待编译的热点方法数量均未超过一个。因此，虽然采用了不同的编译调度算法，其编译顺序始终相同，故性能也几乎一样。

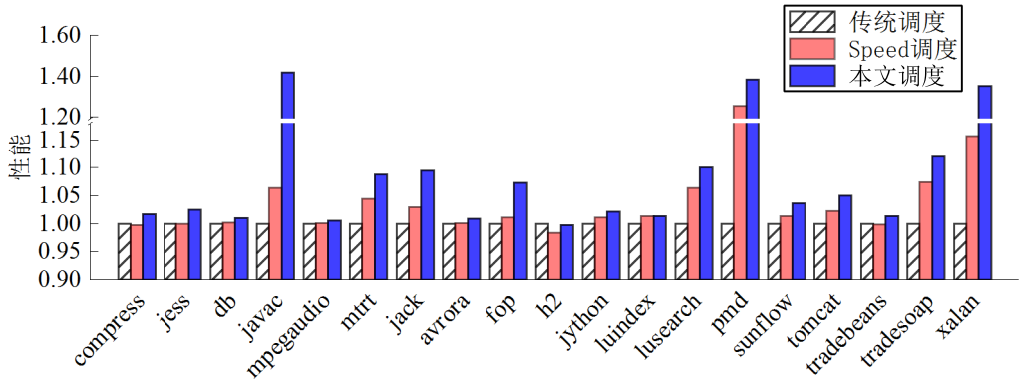


图 5.6: 虚拟机在具体测试项目上的启动性能

由于传统调度并未考虑任何优化，故其获得的性能最低。Speed调度仅考虑了方法的重要性对系统响应速度的影响，却忽略了降低热点方法在编译队列中的平均排队延迟。因此，Speed调度在计算调度的优先级时，仅考虑了动态编译可能带来的收益，却没有考虑由动态编译所产生的代价。而本文调度同时考虑了动态编译的收益和可能导致的损失，故能获得较高的性能提升幅度。

5.5.3 实验分析

5.5.3.1 编译队列平均排队延迟的变化

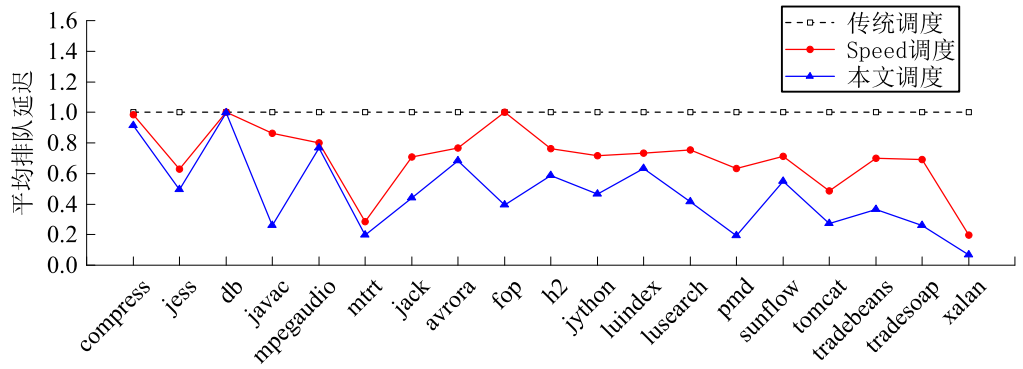
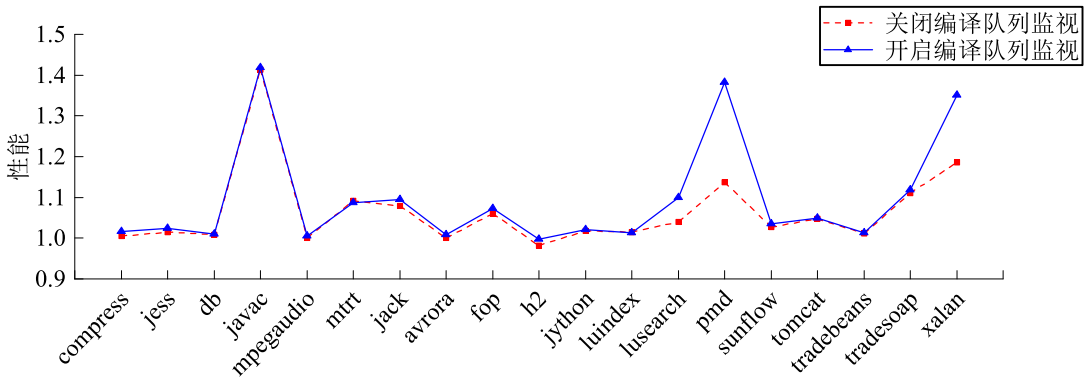


图 5.7: 热点方法在编译队列中的平均排队延迟

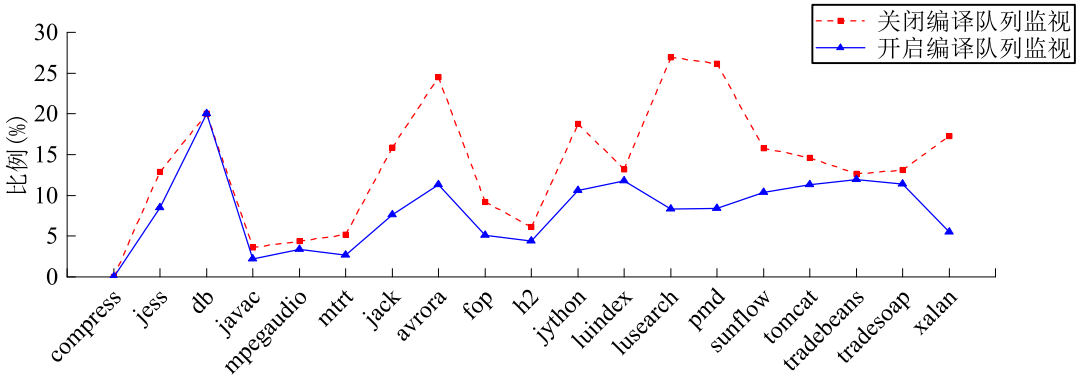
图5.7展示了在不同的编译调度算法下，热点方法在编译队列中的平均排队延迟。对于每个测试项目中的任意方法，当且仅当该方法的执行时间占总的执行时间的比例不低于0.1%时，该方法被称为热点方法。为了识别上述热点方法，本文使用Xprof程序

分析器对所有测试项目进行了剖面分析。图5.7中热点方法在编译队列中的平均排队延迟是指所有被调度的热点方法在编译队列中排队时间的加权均值，其权重系数为各个热点方法执行时间占总执行时间的比例。显然，和其它调度算法相比，本文调度显著降低了编译队列的平均排队延迟。

由5.4.1节中的调度模型知(详见5.16式)，在其它条件相同的前提下，较小的热点方法将有更高的概率被调度。对于给定数量的字节码，在大方法中的编译通常比在小方法中编译花费更多的时间。这是由于相对于小方法，较大的方法中可能包含更多复杂的代码结构(如循环等)，编译和优化所需的时间也越大。故同其它调度算法相比，在相同的时间内，本章提出的算法将编译完成更多的热点方法。因此，采用本文的调度算法可以充分降低编译队列的平均延迟。



(a) 性能



(b) 僵尸方法的比例

图 5.8: 编译队列监视机制的效果

5.5.3.2　编译队列监视机制有效性分析

图5.8(a)展示了在打开和关闭编译队列监视机制的两种情况下，本章所提出的算法的性能。从图中可以看出，相对于关闭编译队列监视的情形，编译队列监视开启后，*pmd*、*xalan*和*lusearch*等测试项目的性能明显提升。同时，对于所有的测试项目，编

译队列监视机制并不会导致性能的降低。因此，编译队列监视机制对于系统性能的提升是十分有益的。进一步分析知，编译队列监视对于系统性能的优化主要归功于有效减少了运行时的编译开销，尤其是僵尸方法的比例。图5.8(b)展示了在打开和关闭编译队列监视机制的两种情况下，僵尸方法的比例。显然，编译队列监视机制大幅减少了僵尸方法的比例，并且对于*pmd*、*lusearch*、*avrora*和*jack*等项目效果尤其明显。在图5.9中，本文进一步比较了在不同的调度算法下，僵尸方法的比例。对于所有测试项目，本文调度使得僵尸方法的比例下降到了最低。僵尸方法比例的降低，不仅可以有效降低运行时的编译开销，同时还能减少热点方法在编译队列中的排队延迟。因此，编译队列监视机制在本章的算法中扮演着不可或缺的重要角色。

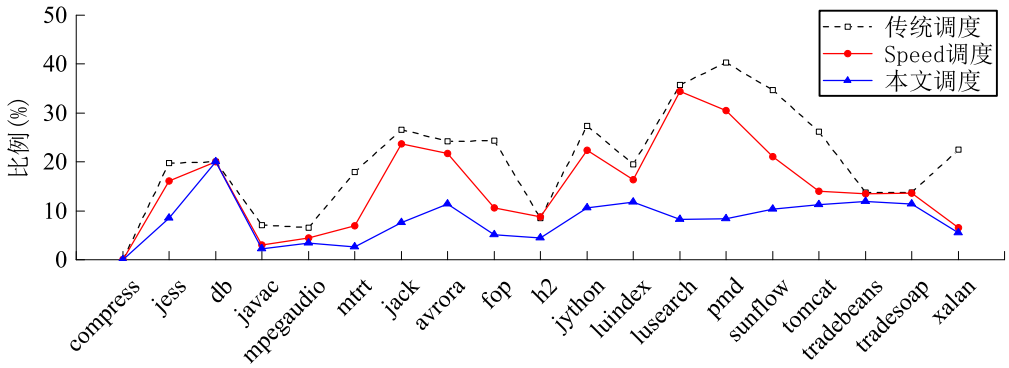


图 5.9: 僵尸方法的比例

5.5.3.3 编译调度的开销

为了进一步分析编译队列监视下的高性能调度算法的运行时开销，本文对所有测试项目分别统计了其编译调度的时间和动态编译的时间，如表5.2所示。编译调度的时间占动态编译总时间的比例从0.128‰ (*compress*)到1.608‰ (*tradesoap*)不等。并且，对于绝大多数的测试项目，该比例均低于0.500‰。因此，编译队列监视下的高性能调度算法引入的运行时开销极小，在实际系统中可以完全忽略。

5.6 本章小结

动态编译调度对虚拟机的性能有着十分重要的影响。本章首先研究了动态编译行为的主要特征，并指明了动态编译调度的优化目标。在此基础上，本章提出了一种编译队列监视下的高性能动态编译调度算法。该算法主要由收益代价分析的调度模型和自适应的编译队列监视机制组成。收益代价分析的调度模型综合考虑了方法的重要性和编译代价，可以加快系统的响应速度并降低编译队列的平均排队延迟。编译队列监视机制自适应地延迟编译队列中不活跃方法的编译，并且有选择性地将它们从编译队

列中删除，从而有效降低系统的运行时开销。编译队列监视下的高性能动态编译调度可以显著提升系统的启动性能。实验结果表明，本章的算法使得系统的启动性能平均提升8.9%，最高提升幅度高达41.8%。此外，该算法还大幅降低了僵尸方法的比例。本章的算法通用性强，实现代价低，很容易推广应用于其它虚拟机系统，对优化虚拟机的性能具有很好的参考意义。

表 5.2: 本文调度的运行时开销

测试项目	调度时间 (μs)	编译时间 (μs)	比例 (‰)
compress	93	725522	0.128
jess	639	3435631	0.186
db	208	1099601	0.189
javac	11880	39645414	0.300
mpegaudio	872	3650481	0.239
mtrt	1685	7725215	0.218
jack	3254	21101050	0.154
avrora	3310	6482278	0.511
fop	10059	28569785	0.352
h2	9372	43185015	0.217
jython	17888	78902132	0.227
luindex	2140	8388727	0.255
lusearch	21700	28503948	0.761
pmd	32812	34797379	0.943
sunflow	3439	9872952	0.348
tomcat	42785	66499669	0.643
tradebeans	21977	77112513	0.285
tradesoap	349818	217490428	1.608
xalan	45556	33929666	1.343

第六章　软硬件协同设计的解释器指令分派优化

6.1　引言

解释器[9][10][32]是最常见的处理计算机高级语言的系统软件，是编程语言实现的首选工具。同JIT编译器相比，虽然解释器的执行速度较慢，但由于具有结构简单、易于实现和维护以及运行时消耗资源少等优势[109]，解释器依然大规模应用于各种软件系统中。解释器和JIT编译器通常以互补的方式共同组成Java等高级语言虚拟机的执行引擎[12][13][15]。在虚拟机中，应用程序由解释器开始执行，JIT编译器仅编译程序中的热点方法。故解释执行的速度直接影响整个系统的启动性能。而对于客户端浏览器中的JavaScript引擎，解释器的性能在很大程度上决定了系统的响应速度，直接影响用户的使用体验[110]。因此，解释器的运行速度较慢成为制约系统可用性和易用性的重要因素。此外，对于高档数控机床和工业机器人等领域，加工程序和控制指令的解析普遍采用解释器来实现，解释器的性能决定了自动化生产的效率和智能化控制的实时性。随着人类社会信息化的不断发展，由于解释器性能不足而导致的系统性能和效率等问题变得越来越普遍和突出。因此，解释器的分析和优化对于改善应用系统的性能，提高社会生产效率具有十分重要的意义。

字节码是解释执行的基本单位。一般地，每条字节码对应一段硬件能够直接执行的本地指令序列。字节码的解释执行，最终由所对应的本地指令序列在硬件上的执行来实现。解释执行的核心循环包括取指令、指令分派、取操作数和执行等四个关键步骤。其中，指令分派是指当前字节码执行完毕后，查找并跳转到下一个字节码的本地指令序列入口的过程。解释执行期间，指令分派频繁发生，会引入大量额外的运行时开销，严重影响解释器的性能[52][53]。本章首先对解释器指令分派的过程进行了深入分析，指出指令分派时获取本地指令序列入口地址的代价很高，构成解释器指令分派的瓶颈。在此基础上，本章提出了一种软硬件协同设计的解释器指令分派优化方案，可以大幅降低指令分派过程中获取本地指令序列入口地址的开销。

本章其余部分组织如下：6.2节对本章研究内容的背景和相关工作进行了介绍；6.3节展示了现代高性能解释器设计的关键技术；6.4节揭示了解释器指令分派的性能瓶颈；6.5节提出了软硬件协同设计的解释器指令分派优化方案；6.6节对本章算法进行了实验和分析，展示了本章算法对提升解释器性能的有效性；最后对本章进行了小结。

6.2　相关工作

早期的解释器大都采用Switch结构来实现[111]。Switch结构是最简单的解释器构造模式，采用了集中式的指令分派方法，即所有指令分派均由同一段机器指令序列实现。

Switch结构的解释器可以全部使用高级语言来实现，因而可移植性较高。20世纪70年代，Bell研究发现Switch结构的解释器在指令分派时存在二次跳转和边界检查等冗余操作，严重影响解释器的性能。为了加速指令分派的过程，Bell提出了著名的线索化代码(threaded code)结构的解释器[8]。线索化代码结构的解释器采用了分布式的指令分派方法。在每个字节码的本地指令序列的末尾，都有能够独立实现指令分派功能的机器指令序列。由于消除了Switch结构解释器中的冗余操作，线索化代码结构的解释器能够获得更高的性能。此后，线索化代码结构成为高性能解释器设计的主流。

根据应用程序的表示形式，线索化代码结构又可细分为符号线索化代码[112](token threaded code)和直接线索化代码[113](direct threaded code)两类。符号线索化代码结构的解释器直接使用字节码作为应用程序的中间表示，解释执行期间，需要查找指令分派表以获取下一条字节码本地指令序列的入口地址。直接线索化代码结构的解释器直接使用字节码本地指令序列的入口地址作为应用程序的中间表示，无需查表。但是，由于直接线索化代码结构的解释器使用了地址作为程序的中间表示，存储开销显著增加。此外，将字节码转换为相应的本地指令序列的入口地址也额外增加了程序运行时的开销。因此，在实际系统中，符号线索化代码结构的解释器更为常见。

然而，符号线索化代码结构的解释器进行指令分派时依然存在较大开销。为了提升指令分派的速度，文献[58][114]提出了超级指令技术(super instruction)。该方法将频繁执行的连续字节码序列等效替换为一条新定义的字节码(称为“超级指令”)，以消除被替换的字节码之间的指令分派。随后，Casey等人[9]进一步指出由于指令分派中用于实现跳转功能的间接转移指令存在多个跳转目标地址，导致处理器对间接转移猜测的正确率下降，从而降低解释器的性能。他们有针对性地提出了指令冗余技术(instruction replication)，通过增加若干语义功能相同的冗余的字节码，以减少存在多个跳转目标的间接转移指令数目，从而在一定程度上提升分支预测的准确性。但是，受字节码中操作码域编码长度的限制，实际能够提供的超级指令和冗余字节码的数量十分有限，因而其优化效果受到限制。

文献[55]针对解释器的特点设计了一种新的分支预测器。该方法利用字节码指针来区分不同的转移场景，并在解释器中插入引导指令来提升处理器对间接转移猜测的正确率。然而该方法要求对处理器内部的分支预测器进行定制，硬件实现开销较大。此外，最新的研究表明[115]，随着分支预测技术的不断发展，在当今主流的处理器上，间接转移猜测已不再是制约解释器性能最主要的因素，因而其优化空间非常有限。

综上所述，为了降低指令分派开销，研究人员从系统软件、工程实现和体系结构等方面提出了多种优化方法，包括线索化代码结构、超级指令技术、冗余指令技术和面向解释器的专用分支预测器等。但是，上述优化方法主要关注对指令分派中跳转操作的优化，并且其效果往往受到体系结构和编码长度等软硬件平台因素的制约，缺乏通用性。针对上述研究的不足，本文以降低获取本地指令序列入口地址的开销为突破

口，对解释器指令分派进行了优化。本文首先对解释器的性能瓶颈进行量化分析，并在此基础上，提出了一种软硬件协同设计的解释器指令分派方法。最后通过对比实验，分析和验证了本文方法的有效性。

6.3　解释器设计的关键技术

本节以当前应用最广泛的HotSpot Java虚拟机的解释器为例，展示了现代高性能解释器设计的关键技术。HotSpot的解释器基于符号线索化代码结构进行设计，模拟了一个基于栈式体系结构的抽象处理器。所有的操作数都存储在解释器的操作数栈中。解释器运行时，需要频繁地访问操作数栈，以获取运算所需的操作数。HotSpot的解释器采用了栈顶缓存(stack caching)的优化技术[57]，将操作数栈顶的一个元素缓存到一个通用寄存器中，从而减少取操作数时冗余的栈顶访存操作。例如，对于表示整数加法的字节码iadd，在栈顶缓存优化前，需要访问两次操作数栈，以获取加法运算所需的两个操作数。采用栈顶缓存优化后，其中一个操作数被缓存到一个通用寄存器中，因此对操作数栈的访存操作由原来的两次减少为一次。栈顶缓存技术可以大幅减少解释器运行时对内存的访问次数，故可以有效地提升解释器的性能。

表 6.1: 解释器栈顶缓存状态

缓存状态	缓存对象
btos	一个byte类型数据
ctos	一个char类型数据
stos	一个short类型数据
itos	一个int类型数据
ltos	一个long类型数据
ftos	一个float类型数据
dtos	一个double类型数据
atos	一个引用类型数据
vtos	无缓存

栈顶缓存技术使得字节码的本地指令序列存在多个不同的入口。在指令分派阶段，解释器根据下一个字节码和当前的栈顶缓存状态查询指令分派表，以获取本地指令序列的入口地址。表6.1展示了所有的栈顶缓存状态及其含义。从表6.1中可以看出，由于存在9种不同的栈顶缓存状态，故字节码本地指令序列的入口最多可以有9个。图6.1展示了指令分派表的基本结构，其中每列存储一个字节码的本地指令序列所有可能的入口地址。例如，图6.1中行列标号为(3, 96)和(8, 96)的两个表项，如图中阴影部分所示，

分别存储字节码iadd(其编码为96)对应于itos和vtos栈顶缓存状态下本地指令序列的入口地址。由于指令分派表在解释执行期间被频繁访问，因此合理地设计指令分派表的结构，以加快指令分派的查表操作，是提升解释器性能的关键。

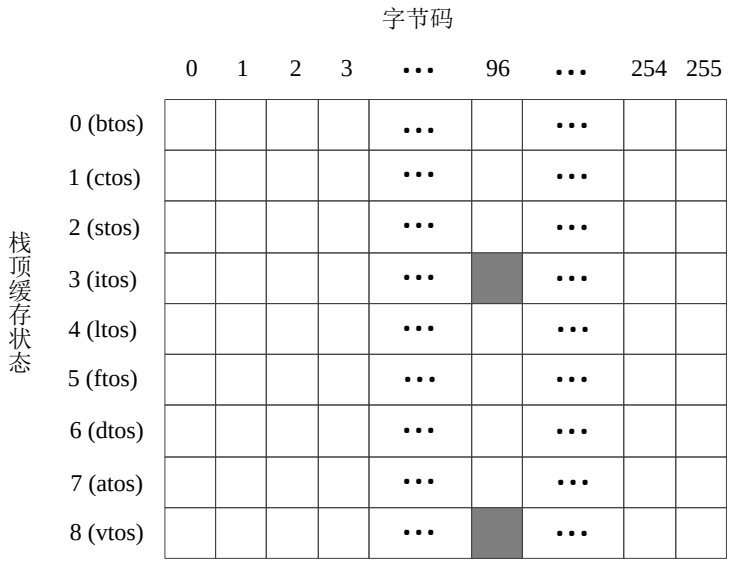


图 6.1: 解释器指令分派表

6.4 解释器指令分派的瓶颈分析

解释执行期间，指令分派的开销对解释器的性能影响很大[52–54]。由于直接统计指令分派的执行时间较为困难且存在较大误差，本文使用需要执行的机器指令的数量来间接反映指令分派的运行时开销。本节首先对字节码的本地指令序列进行了分析，从静态的角度说明了指令分派的开销。随后，又统计了解释器运行时指令分派部分实际执行的机器指令数目，从动态的角度进一步展示了指令分派的开销。最后，通过对指令分派具体步骤的分析，深刻揭示了指令分派的瓶颈。

首先，从静态角度看，在每个字节码的本地指令序列中，用于指令分派的指令占了相当大的比例。图6.2展示了在MIPS架构处理器的64位系统中，字节码iadd的本地指令序列。为了更清晰地展示本地指令序列的结构和语义，图中省略了对分支指令延迟槽的优化。由图6.2可知，iadd的本地指令序列共有18条机器指令，按功能可划分为取操作数、执行规定的语义动作、取下一个字节码和指令分派四部分。其中用于指令分派的机器指令多达11条，占本地指令序列的61.1%。图6.3则进一步对所有字节码，统计了指令分派部分的指令条数占本地指令序列指令数目的比例。图中的统计结果表明，在绝大多数字节码的本地指令序列中，用于指令分派的本地指令数目占本地指令总数

的25%以上，几何均值达到21.4%，最高可达68.8%。因此，可以预见，解释器运行时指令分派实际执行的指令平均占总的执行指令数目的20%左右，甚至达到更高的比例。

```
#   # 取操作数
1   lw      v0, 0x0(sp)    # vtos入口
2   daddiu sp, sp, 0x8
3   lw      v1, 0x0(sp)    # itos 入口
4   daddiu sp, sp, 0x8

#   # 执行规定的语义动作
5   addu    v0, v1, v0     # 相加求和

#   # 取下一个字节码
6   lbu     s1, 0x1(s0)    # 加载下一个字节码
7   daddiu s0, s0, 0x1     # 字节码指针s0前移

#   # 指令分派
8   lui     t3, 0x0                  ┐
9   ori     t3, t3, 0x55             │
10  dsll    t3, t3, 16               ├ 加载指令分派
11  ori     t3, t3, 0x56cf           │ 表的行首地址
12  dsll    t3, t3, 16               │ 到寄存器t3中
13  ori     t3, t3, 0xffffc070       ┘
14  dsll    t2, s1, 3          # 计算offset于t2
15  daddu   t3, t3, t2         # 计算Entry的地址
16  ld      t3, 0x0(t3)        # 加载Entry的值到t3
17  jr      t3                 # 跳转到Entry
18  nop                        # 延迟槽
```

图 6.2: 字节码iadd的本地指令序列

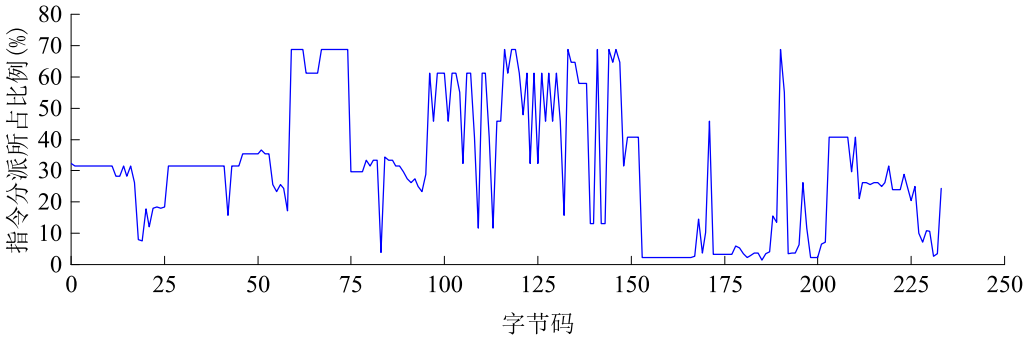


图 6.3: 指令分派在本地指令序列中所占比例

其次，从动态角度看，解释器运行时，指令分派执行的指令条数在总共执行的指令数目中占了相当大的比例。表6.2统计了在SPECjvm98和DaCapo测试集上，解释器的运行时数据。从表6.2中可以看出，解释执行期间，指令分派频繁发生，可高达百亿次的数量级。此外，表中的数据进一步表明，在绝大多数的测试项目中，解释器运行时指令分派执行的指令条数占总共执行的指令数目的25%以上，几何均值达到27.33%，最高可达50.15%。故动态度量的结果与静态分析的结果一致。

综合上述静态和动态两方面的分析可知，由于指令分派部分的指令在字节码的本地指令序列中占比很高，并且指令分派频繁发生，故指令分派在解释执行期间产生的额外开销很大，构成解释器性能的瓶颈。

指令分派时，解释器首先根据下一个字节码和当前的栈顶缓存状态查询指令分派表，以获取本地指令序列的入口地址(记为$Entry$)，随后跳转到$Entry$处继续执行。设下一个待执行的字节码编码为$x(x \in [0, 255])$，当前栈顶的缓存状态为$s(s \in [0, 8])$，则指令分派可进一步细分为以下几个步骤：

1. 加载待查询的指令分派表的行首地址到通用寄存器Reg中(即图6.2中第8至第13条指令)；

2. 计算存储$Entry$的表项在待查询的行内的偏移$offset$(即图6.2中第14条指令)；

3. 从指令分派表中地址为$Reg + offset$的位置加载$Entry$的值(即图6.2中第15和16条指令)；

4. 跳转到$Entry$继续执行(即图6.2中第17条指令)。

上述四个步骤中，前三步为获取$Entry$的操作，最后一步执行跳转动作。由于获取$Entry$需要9条机器指令(即图6.2 中第8至第16条)，占据了指令分派的大部分执行时间，故获取$Entry$的操作构成了指令分派的瓶颈。进一步分析知，第1步中加载待查询的指令分派表的行首地址耗费了6条机器指令(即图6.2中第8至第13条)，是导致获取$Entry$代价过高的主要原因。因此，指令分派时获取本地指令序列的入口地址代价很高，构成了解释器指令分派的瓶颈。

6.5 软硬件协同设计的解释器指令分派优化

本节针对指令分派时获取本地指令序列的入口地址代价过高的问题，设计并实现了一种软硬件协同的指令分派方法，主要包括软件优化的指令分派表访问技术和基于硬件支持的访存加速策略。

表 6.2: 解释执行的运行时数据

测试项目	指令分派次数(亿次)	指令分派执行的指令数目(亿条)	总共执行的指令数目(亿条)	指令分派执行指令的比例(%)
compress	113	1243	4107	30.27
jess	16	176	707	24.89
db	32	352	1459	24.13
javac	19	209	804	26.00
mpegaudio	108	1188	3580	33.18
mtrt	18	198	910	21.76
jack	9	99	534	18.54
avrora	74	814	3056	26.64
fop	8	88	321	27.41
h2	376	4136	8247	50.15
jython	131	1441	4268	33.76
luindex	25	275	852	32.28
lusearch	74	814	3158	25.78
pmd	19	209	946	22.09
sunflow	437	4807	17550	27.39
tomcat	36	396	1355	29.23
tradebeans	220	2420	9567	25.30
tradesoap	235	2585	9422	27.44
xalan	74	814	3295	24.70
几何均值	55	606	2216	27.33

6.5.1 指令分派表的优化设计

指令分派期间，为了查找本地指令序列的入口地址，需加载待查询的指令分派表的行首地址到寄存器中。由图6.2可知，完成该操作需要6条机器指令，实现代价较高。本文设计了一种便于高效访问的指令分派表，其结构如图6.4所示，完全消除了加载指令分派表行首地址的开销。本文从以下四个方面对指令分派表进行了优化设计。

- 对指令分派表的行列进行了转置，为后文基于硬件支持的指令分派表的访问加速奠定了基础。转置前的指令分派表每行有256个表项。当加载行内的某个表项时，行内偏移的取值范围较大。对于64位的系统，转置前行内偏移的范围

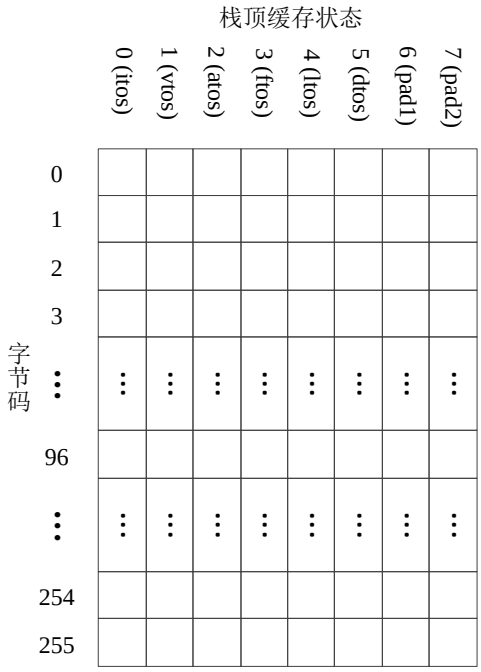


图 6.4: 优化后的指令分派表结构

是$[0,\ 2^{11}-8] \in [-2^{11},\ 2^{11}-1]$，故要求访存指令中偏移量编码域的长度不少于12位。但是由于精简指令集架构的处理器采用了固定长度的指令编码格式，访存指令中所能提供的编码偏移量的长度非常有限。本文对原有指令分派表进行了转置，可以减少机器指令中偏移量编码所需的长度，使得通过硬件支持的方式加速指令分派表访问成为可能。

- 剔除了三类无用表项，减少了解释执行时对系统资源的消耗。指令分派时，需根据当前栈顶的缓存状态确定下一个字节码的本地指令序列的入口地址。通过将缓存状态为btos、ctos和stos的入口地址映射到itos的入口地址，图6.4中的指令分派表消除了btos、ctos和stos相关的表项，进一步减少了解释执行对内存等系统资源的消耗。

- 添加了两列填充表项，以减少查表的计算开销。指令分派表在添加填充表项前，需执行一次乘法操作来计算待查表项的行偏移。由于乘法运算的代价较高，在设计指令分派表时，每行的末尾均添加了两个填充表项，如图6.4中阴影部分所示，从而可通过一次快速的左移位操作计算出行偏移。因此，填充表项的引入可将代价较高的乘法运算转换为高效的移位操作，查表时计算代价明显降低。

- 合理设计指令分派表每列的顺序，提高访存效率。图6.5展示了访问指令分派表时栈顶缓存状态的频率分布，从图中可知指令分派表的访问集中在对vtos、

itos、atos和ftos缓存状态的查询。因此在设计指令分派表时，将vtos、itos、atos和ftos这四个表项安排在相邻位置，以增强访存局部性，提高访存效率。

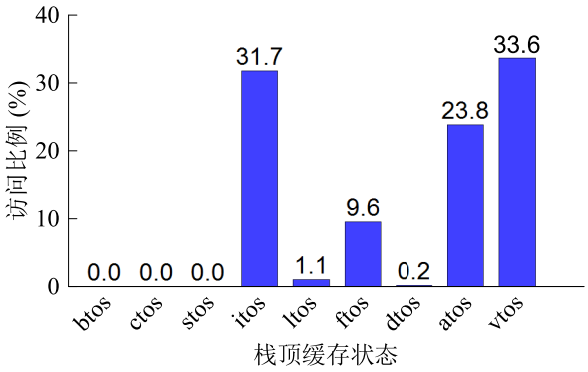


图 6.5: 指令分派表访问时栈顶缓存状态的频率分布

6.5.2　软件优化的指令分派表访问技术

根据6.5.1节中指令分派表结构的设计，指令分派表的访问技术也做了相应的软件优化。其核心思想是将指令分派表的起始地址缓存到一个可以高速访问的部件中，以消除代价较高的行首地址加载操作。

图6.6具体展示了基于软件优化的指令分派表查找本地指令序列入口地址的方法。本文选择了一个通用寄存器作为指令分派表起始地址的高速缓存部件，因此，该部件称为指令分派寄存器。解释器初始化时，将指令分派表的起始地址(即图6.6中标“0”的箭头所指向的位置)保存到指令分派寄存器gpr中。设下一个待执行的字节码编码为$x(x \in [0,\ 255])$，当前栈顶的缓存状态为$s(s \in [0,\ 5])$，待查找的本地指令序列的入口地址为$Entry$，则获取$Entry$的步骤如下：

- **步骤1**：根据字节码x的编码，计算待访问表项的行偏移row_offset(即图6.6中标“1”的箭头所指向的位置)：

  $$row_offset = 8 \times AddressLength \times x$$

  其中$AddressLength$表示系统中一个指针所占的字节数目，为一个固定的常量。row_offset保存在一个通用寄存器中。它的值将在解释执行的过程中实时计算。具体实现时，上述乘法操作可以转换为对x的快速左移位操作，需要左移的位数为$log_2(8 \times AddressLength)$。

- **步骤2**：根据当前的栈顶状态，计算待访问表项的列偏移col_offset(即图6.6中标“2”的箭头所指向的位置)：

  $$col_offset = AddressLength \times s$$

由于$AddressLength \leq 8$，故$col_offset \in [0,\ 40]$，为一个较小的立即数。其值在解释器初始化时确定。

- **步骤3：** 从指令分派表中地址为$gpr + row_offset + col_offset$的表项(即图6.6中标“3”的箭头所指向的表项)中加载$Entry$的值。

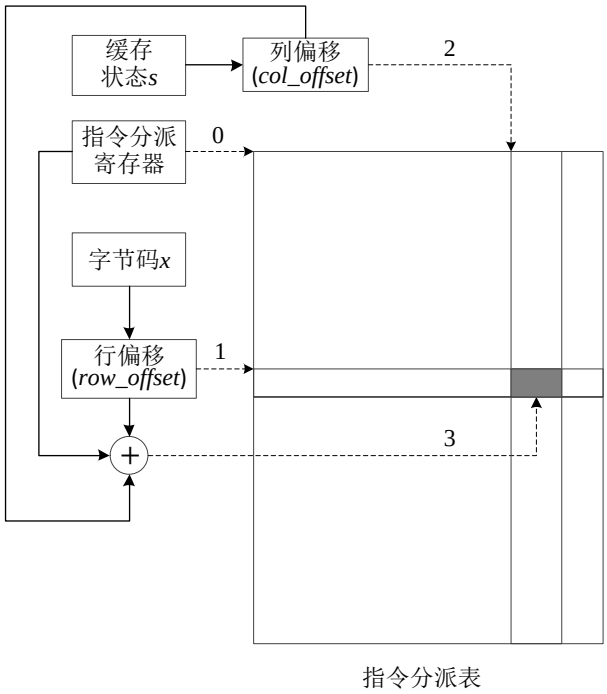


图 6.6: 软件优化的指令分派表访问方法

图6.7展示了采用图6.6的方法实现的指令分派方案。行偏移row_offset的值在第1条左移位指令dsll中完成计算，结果存于寄存器t2中。在第2条加法指令daddu中，指令分派寄存器gpr与t2相加得到待查询的指令分派表的行首地址，并将其保存到寄存器t3中。随后，第3条访存指令ld实现从地址为t3 + col_offset的指令分派表中加载$Entry$的值。最后，通过第4条间接转移指令jr跳转到入口为$Entry$的本地指令序列中继续执行，从而完成一次指令分派。

对比图6.2可以看出，优化前用于加载指令分派表待查询的行首地址的6条机器指令已经全部消除，指令分派部分的机器指令数目由原来的11条减少为5条。通过表6.2中对SPECjvm98和DaCapo等测试集的分析知，解释执行期间指令分派的次数可高达数百亿次。因此优化后，可以减少千亿条数量级机器指令的执行，从而大幅度提高指令分派的效率。

```
#   # 指令分派
1   dsll    t2, s1, 6              # 计算row_offset于t2
2   daddu   t3, gpr, t2            # 计算Entry所在行的首地址
3   ld      t3, col_offset(t3)     # 加载Entry的值到t3
4   jr      t3                     # 跳转到Entry
5   nop                            # 延迟槽
```

图 6.7: 纯软件优化的指令分派

6.5.3　基于硬件支持的访存加速策略

由6.5.2节中的分析可知，为了获取$Entry$的值，需要访问地址为$gpr+row_offset+col_offset$的内存单元。其中，$gpr$为基址寄存器，保存指令分派表的起始地址；$row_offset$为索引寄存器，保存待查询的行的索引；$col_offset$为一个较小的立即数，表示目标单元在行内的偏移。由于MIPS等精简指令集的处理器不支持“基址+索引+偏移”的寻址方式，故上述操作需要两条机器指令(图6.7中第2、3条)才能完成。

为了进一步加快指令分派的速度，本文对处理器的访存指令进行了扩展，新引入了一种支持“基址+索引+偏移”的寻址方式的访存指令，使得对$Entry$的加载操作仅由一条指令即可完成，从硬件层面直接加速指令分派时对内存地址$gpr + row_offset + col_offset$的访问速度。其硬件实现原理如图6.8所示。扩展指令的实现主要涉及对处理器流水线中指令译码器和访存部件的修改。由于扩展指令使用了新定义的指令操作码，指令译码器需增加对扩展指令的译码功能。扩展指令的长度不变，并且遵循原有指令集的编码格式，故原有的指令译码器基本兼容扩展指令的译码，其修改非常小。虽然扩展指令中新增了一个源寄存器(索引寄存器)，由于原有的处理器可以直接对两个源寄存器和一个目的寄存器进行重命名，故寄存器重命名无需任何修改，处理器中没有增加额外的数据通路。由于访存的有效地址计算需要对“基址+索引+偏移”进行求和运算，故访存部件中原有的双操作数地址运算单元需要进行简单的扩展，以支持3操作数的求和运算，其设计和实现的代价也非常小。访存处理的后端根据有效地址完成指定的访存操作，并将访存结果写回结果寄存器中，这部分功能部件无需任何修改。综上，由于扩展指令在实现时仅需简单修改译码器和访存地址运算单元，没有额外增加数据通路并且访存处理的后端可直接复用，故实现代价很低。

在MIPS架构中，指令编码长度固定为32位。原有的访存指令(如ld指令等)最大仅支持16位的偏移量。而扩展的访存指令中需要额外使用5位用于对新增的索引寄存器进行编码，故无论扩展的访存指令如何设计，其所支持的偏移量最大不能超过16 - 5 = 11位。由6.5.1节中的分析可知，优化前的指令分派表要求访存指令中的偏移量不少

于12位。因此，本文对原有指令分派表的行和列进行了转置。转置后，即使对于64位的系统，指令分派表行内偏移的取值范围是$[0, 40] \subseteq [-2^6,\ 2^6 - 1]$，故偏移量的编码只需要7位，满足了硬件扩展的约束。

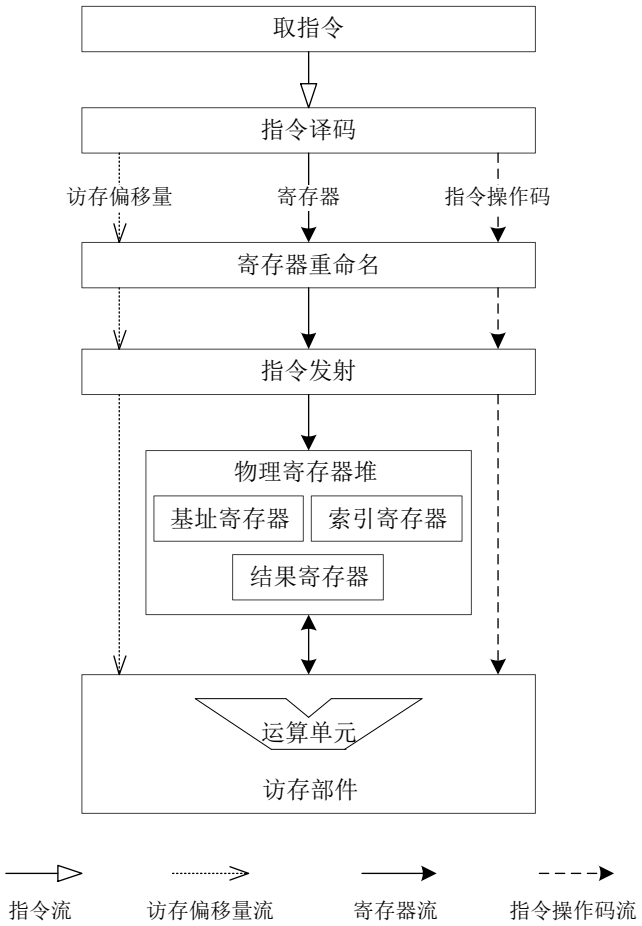


图 6.8: 扩展访存指令的硬件实现原理

```
#   # 指令分派
1   dsll    t2, s1, 6                # 计算row_offset
2   gsldx   t3, gpr, t2, col_offset  # 加载Entry的值
3   jr      t3                       # 跳转到Entry
4   nop                              # 延迟槽
```

图 6.9: 软硬件协同的指令分派

基于上述分析，本文总共实现了12条扩展的访存指令，包含8条定点指令和4条浮点指令，寻址方式均为“基址+索引+偏移”的访存模式。如表6.3所示，上述扩展指令分别为存取字节、存取半字、存取字、存取双字、存取单精度浮点数和存取双精度浮点数。表6.3中的指令均支持长度为8位的访存偏移量，完全满足优化后指令分派表访问时对7位偏移量的需求。

图6.9展示了添加硬件支持后，指令分派的实现方案。图中使用了一条名为gsldx的扩展访存指令。由表6.3知，该指令用于从内存中加载一个64位的双字。对比图6.7可以看出，加载$Entry$的操作由图6.7中的两条机器指令减少为图6.9中的一条gsldx指令即可完成。由于gsldx与原有64位访存加载指令ld所需的执行周期相同，故扩展指令的引入可以消除一条加法指令daddu，从而进一步提高指令分派的效率。

表 6.3: 扩展的访存指令

指令格式	功能描述
gsLBX	取字节
gsSBX	存字节
gsLHX	取半字
gsSHX	存半字
gsLWX	取字
gsSWX	存字
gsLDX	取双字
gsSDX	存双字
gsLWXC1	取单精度浮点数
gsSWXC1	存单精度浮点数
gsLDXC1	取双精度浮点数
gsSDXC1	存双精度浮点数

6.6　实验结果与分析

6.6.1　实验设计

本文使用业界广泛使用的SPECjvm98[94]和DaCapo-9.12-bach[95]评估本章算法的效果。为了降低垃圾收集等对性能测试的影响，实验通过设置虚拟机参数“-Xms1G

-Xmx1G”，统一使用1G大小的堆空间。实验中使用了MIPS兼容的龙芯3B1500处理器，主频900MHz，内存8G，操作系统为64位的CentOS 6.4。

本文进行了以下3组实验：

1. 传统的实现方案，即图6.2中的方法；
2. 纯软件的优化方案，即图6.7中的方法；
3. 软硬件协同的优化方案，即图6.9中的方法。

为了减小实验误差，每个测试程序均运行10次，取其均值作为最终的实验结果，并以传统方案的结果为基准对实验结果进行了归一化处理。

6.6.2 实验结果

图6.10展示了解释器在三种方案下的总体性能。从图中可以看出，相对于传统方案，采用纯软件的优化方案，解释器总体性能提升幅度已高达8.3%。而增加硬件支持后，解释器总体性能进一步提升约3.2%，最终达到11.5%的总体性能提升。

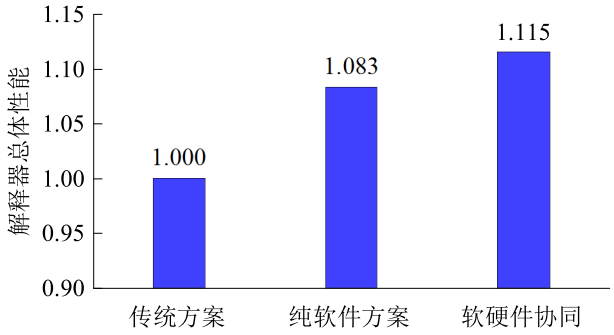


图 6.10: 解释器总体性能

图6.11展示了在三种方案下，解释器在SPECjvm98和DaCapo测试集上的性能。图中的数据表明，软硬件协同的优化使得绝大多数的测试项目获得10%以上的性能提升，最高提升幅度高达15.4%。此外，从图6.11中还可以看出，使用硬件支持，解释器的性能可在纯软件优化的基础上进一步提升约2 ∼ 5%，故硬件支持是纯软件优化的重要补充。因此，软硬件协同设计的指令分派对解释器性能的提升具有非常好的效果。

6.6.3 实验分析

为了进一步揭示解释器性能提升的原因，本文从静态和动态两个方面进行分析。从静态角度看，字节码本地指令序列中，指令分派部分机器指令数目的减少是解释器性能提升的根本原因。图6.12展示了软硬件协同优化前后，指令分派在字节码的本地指

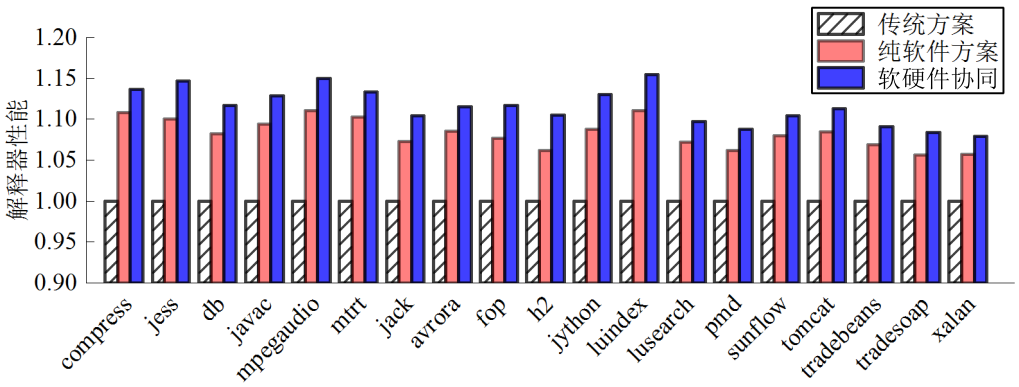


图 6.11: 解释器在SPECjvm98和DaCapo测试集上的性能

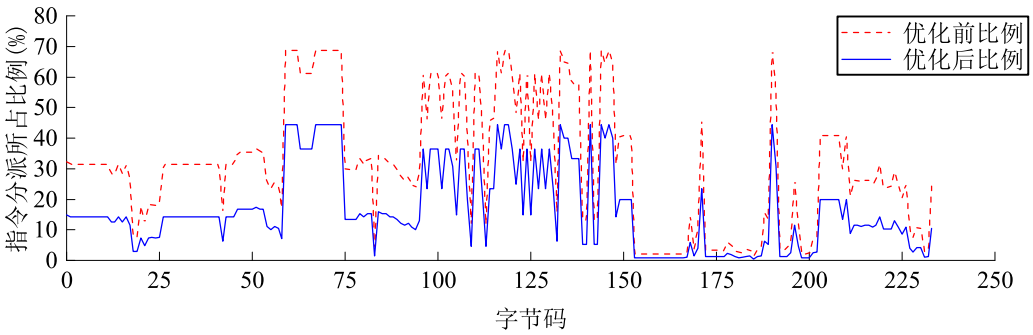


图 6.12: 软硬件协同优化前后，指令分派在字节码的本地指令序列中所占比例

令序列中所占的比例。图中的结果表明，优化后指令分派的比例显著下降。对于绝大多数字节码，该比例下降幅度超过了50%，所有字节码指令分派部分所占比例的几何均值由原来的21.4%下降为9.9%，下降幅度高达53.7%。此外，指令分派部分机器指令数目的减少，使得所有字节码的本地指令序列中机器指令的总数减少了6.7%，解释执行所需的存储空间进一步降低。指令分派中机器指令数目的大幅减少，从理论上揭示了软硬件协同设计对解释器指令分派优化的优良效果，并且预示着动态执行的机器指令数目的大幅减少。

从动态角度看，运行时实际执行的机器指令数目的减少是解释器性能提升的直接原因。表6.4展示了软硬件协同优化后，在SPECjvm98和DaCapo测试集上，解释器运行时减少执行的机器指令数目。由表6.4的数据知，软硬件协同优化后，解释器减少执行的机器指令数目高达千亿条的数量级，平均下降幅度为优化前的16.67%，最高优化幅度高达22.87%。实验结果与理论预期相符。

图6.13展示了软硬件协同优化后，在SPECjvm98和DaCapo测试集上，解释器减少执行的机器指令比例和性能提升的幅度。从图6.13中不难看出，减少的机器指令比例曲线和性能提升幅度曲线的走势高度一致，表明解释器性能的提升和减少执行的机器指令数目具有很强的正相关性。故图6.11中解释器最终的性能提升幅度，主要是由

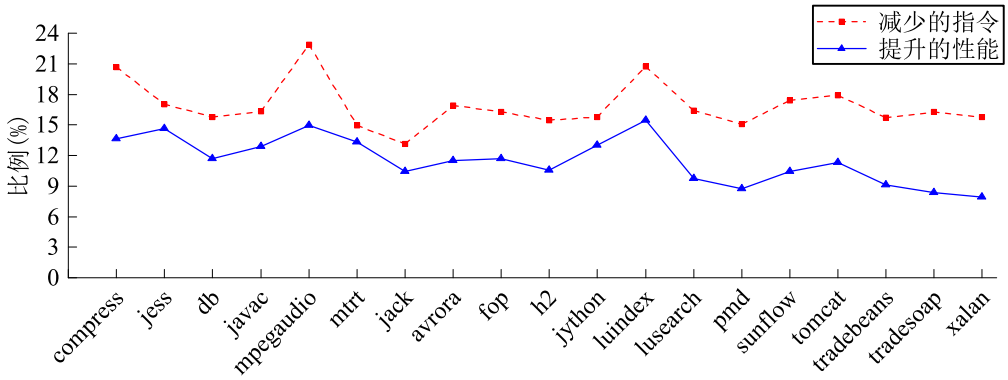


图 6.13: 软硬件协同优化后，解释器减少执行的机器指令比例与性能提升幅度

表6.4中减少执行的机器指令的比例决定。由于软硬件协同的优化可以大幅度减少指令分派所需的机器指令数目，故能充分地优化解释器的性能。

6.7 本章小结

解释器指令分派开销过大一直是制约解释器性能最关键的因素。本章首先对解释器指令分派的开销进行了量化分析，指出对字节码本地指令序列入口地址的获取构成了指令分派的瓶颈。在此基础上，提出了一种软硬件协同设计的解释器指令分派优化方案。该方案具体包括优化的指令分派表设计、软件优化的指令分派表访问技术和基于硬件支持的访存加速策略。软硬件协同的优化技术可以大幅降低指令分派过程中获取本地指令序列入口地址的开销。在软件层面，通过将优化后指令分派表的起始地址缓存到一个可以高速访问的部件中，消除了优化前代价较高的入口地址加载操作。在硬件层面，通过对原有访存指令进行扩展，进一步提高了访存效率。本章提出的优化方案可以大幅度减少指令分派所需执行的机器指令数目，从而显著提升解释器的性能。实验结果表明，本章的优化方案使得解释器的性能平均提升幅度为11.5%，最高提升幅度高达15.4%。本文的方法通用性强，实现代价低，对于现代高性能解释器的设计和优化具有很好的借鉴意义。

表 6.4: 软硬件协同优化后减少执行的机器指令

测试项目	优化前执行的指令数目(千亿条)	优化后减少的指令数目(千亿条)	减少的比例(%)
compress	4.11	0.85	20.68
jess	0.71	0.12	17.03
db	1.46	0.23	15.79
javac	0.80	0.13	16.31
mpegaudio	3.58	0.82	22.87
mtrt	0.91	0.14	14.95
jack	0.53	0.07	13.11
avrora	3.06	0.52	16.91
fop	0.32	0.05	16.30
h2	8.25	1.28	15.48
jython	4.27	0.67	15.77
luindex	0.85	0.17	20.72
lusearch	3.16	0.52	16.42
pmd	0.95	0.15	15.06
sunflow	17.55	3.06	17.42
tomcat	1.35	0.24	17.90
tradebeans	9.57	1.51	15.73
tradesoap	9.42	1.53	16.25
xalan	3.29	0.52	15.77
几何均值	2.22	0.37	16.67

第七章　总结与展望

7.1　本文工作总结

凭借其平台无关性、安全性和网络移动性等优势，Java广泛应用于实际系统中。近年来，云计算、大数据、移动互联网和物联网等新兴应用的迅猛发展对Java的性能提出了更高的要求，使得Java虚拟机的设计面临更为严峻的挑战。如何提高Java虚拟机的性能已成为现代信息产业亟需解决的关键问题。因此，对Java虚拟机性能优化技术的研究具有重要的现实意义，已成为现代计算机系统研究的热点。

本文从动态编译和解释执行两个方面研究了Java虚拟机的优化设计，并提出了一系列性能优化技术，主要研究成果包括：

1. 在动态编译的热点识别阶段，提出了一种基于贝叶斯估计的程序热点识别算法。热点识别是影响虚拟机性能的关键因素。本文首先设计了以智能阈值为核心的热点识别模型。其中，智能阈值由方法的静态特征和系统的运行状态共同确定。和传统的固定阈值相比，采用智能阈值的热点识别模型具有更高的灵活性和更强的自适应能力。在此基础上，提出了一种基于贝叶斯估计的程序热点识别算法。该算法采用幂率分布模型构建方法执行次数的先验分布。在进行贝叶斯估计时，该算法除了利用方法当前的执行次数外，还利用了方法在未来短时间内可以预期的前瞻性执行次数，实现了对程序热点的前瞻性识别。实验结果表明，该算法可以显著降低热点识别的延迟，从而降低热点方法的编译延迟，有效提升虚拟机的性能。其中系统的启动性能平均提升14.6%，最高提升幅度高达61.3%；峰值性能平均提升4.2%，最高提升幅度高达24.0%。

2. 在动态编译的调度阶段，提出了一种编译队列监视下的高性能动态编译调度算法。动态编译调度对虚拟机的性能有着十分重要的影响。本文首先深入分析了动态编译行为的特征，提出了动态编译调度的三个优化目标，即优先编译对性能影响较大的方法、减少热点方法在编译队列中的排队延迟和减少动态编译的运行时开销。基于上述优化目标，本文提出了一种编译队列监视下的高性能动态编译调度算法。该算法包括基于收益代价分析的调度模型和自适应的编译队列监视机制。基于收益代价分析的调度模型综合考虑了编译的收益和可能导致的性能损失，既可加快系统的响应速度，又能降低编译队列的平均排队延迟。同时，该算法在调度期间引入了编译队列监视机制，以实时侦测编译队列中非活跃的方法，并且选择性地将所侦测到的非活跃方法从编译队列中删除，从而有效降低系统的运行时开销。实验结果表明，该算法使得系统的启动性能平均提升8.9%，最高提升幅度高达41.8%。

3. 在解释执行优化方面，提出了一种软硬件协同设计的解释器指令分派优化技术。解释器指令分派一直是制约解释器性能最关键的因素。本文首先对解释器指令分派的开销进行了量化分析，指出对字节码本地指令序列入口地址的获取构成了指令分派的瓶颈。在此基础上，提出了一种软硬件协同设计的指令分派优化技术，包括优化的指令分派表设计、软件优化的指令分派表访问技术和基于硬件支持的访存加速策略。指令分派表的优化设计是软硬件协同优化的基础，既提高了软件设计的效率，又为访存指令的扩展创造了条件。在软件层面，通过将优化后指令分派表的起始地址缓存到一个可以高速访问的部件中，消除了优化前代价较高的入口地址加载操作。在硬件层面，通过对原有访存指令进行扩展，进一步提高了访存效率。实验结果表明，软硬件协同的优化方案大幅降低了指令分派的开销，有效提升了解释器的性能。优化后解释器执行的机器指令数目平均下降了16.7%，解释器的平均性能提升11.5%，最高提升幅度为15.4%。

7.2 进一步研究工作

本文对Java虚拟机性能的优化技术进行了研究，重点关注了动态编译和解释执行这个两方面，提出了相应的优化算法，并取得了良好的效果。在未来的工作中，Java虚拟机以下几个方面的问题仍值得进一步研究：

1. 异构计算方面：传统Java虚拟机的计算以CPU为核心，Java虚拟机缺乏对GPU等异构计算模型的有力支持。随着海量数据处理和多媒体技术的不断发展，实际应用在计算规模、运算速度和并发度等方面的要求不断攀升，以CPU为中心的Java虚拟机的运算能力已无法满足上述需求。由于其高效的运算速度和强大的并行处理能力，以GPU为代表的异构计算是解决上述问题的有效方法。因此，在Java虚拟机中实现对异构计算模型的支持是未来重要的研究方向。

2. 垃圾收集方面：传统的垃圾收集器仅针对中小规模的数据进行设计，当前Java虚拟机中亟需实现适用于大数据应用的垃圾收集器。随着应用程序规模的不断增大，系统对内存的需求也持续增长，导致Java虚拟机中垃圾收集面临前所未有的压力。云计算和大数据时代，高效的垃圾收集器对于减少系统的停顿时间、提高系统的并发度和吞吐量非常重要。因此，面向大规模数据的垃圾收集算法是未来Java虚拟机研发的重点。

3. 安全性方面：现有Java虚拟机系统面对专门的恶意攻击缺乏足够的防护机制[116]。随着Java在通信、金融和电子商务等领域的应用不断扩大，面对日益复杂的外部环境，其安全性问题变得越来越突出。目前，Java虚拟机在一定程度上能够抵御一些常见的编程失误，但对于恶意的攻击却缺乏足够的抵抗能力。

如何提高系统的安全性已成为广大用户重点关注的问题。因此，关于Java虚拟机安全性方面的研究在未来也是必不可少的。

参 考 文 献

[1] John Whaley. Partial method compilation using dynamic profile information. In *Proceedings of the 16th ACM SIGPLAN Conference on Object-oriented Programming, Systems, Languages, and Applications*, OOPSLA '01, pages 166–179, New York, NY, USA, 2001. ACM.

[2] Weiwu Hu, Yanping Gao, Tianshi Chen, and Junhua Xiao. The godson processors: Its research, development, and contributions. *Journal of Computer Science and Technology*, 26(3):363–372, 2011.

[3] 龙芯中科技术有限公司. 龙芯3b1500处理器用户手册. `http://www.loongson.cn/product/cpu/3/Loongson3B1500.html`.

[4] James R. Larus. Look up!: Your future is in the cloud. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '13, pages 1–2, New York, NY, USA, 2013. ACM.

[5] H. V. Jagadish, Johannes Gehrke, Alexandros Labrinidis, Yannis Papakonstantinou, Jignesh M. Patel, Raghu Ramakrishnan, and Cyrus Shahabi. Big data and its technical challenges. *Commun. ACM*, 57(7):86–94, July 2014.

[6] Tim Lindholm, Frank Yellin, Gilad Bracha, and Alex Buckley. *The Java Virtual Machine Specification, Java SE 7 Edition*. Addison-Wesley Professional, 1st edition, 2013.

[7] TIOBE. TIOBE编程语言排行榜. `http://www.tiobe.com/tiobe_index`.

[8] James R. Bell. Threaded code. *Commun. ACM*, 16(6):370–372, June 1973.

[9] Kevin Casey, M. Anton Ertl, and David Gregg. Optimizing indirect branch prediction accuracy in virtual machine interpreters. *ACM Trans. Program. Lang. Syst.*, 29(6), October 2007.

[10] Yunhe Shi, Kevin Casey, M. Anton Ertl, and David Gregg. Virtual machine showdown: Stack versus registers. *ACM Trans. Archit. Code Optim.*, 4(4):2:1–2:36, January 2008.

[11] Timothy Cramer, Richard Friedman, Terrence Miller, David Seberger, Robert Wilson, and Mario Wolczko. Compiling java just in time. *IEEE Micro*, 17(3):36–43, May 1997.

[12] M. Arnold, S.J. Fink, D. Grove, M. Hind, and P.F. Sweeney. A survey of adaptive optimization in virtual machines. *Proceedings of the IEEE*, 93(2):449–466, Feb 2005.

[13] Michael R. Jantz and Prasad A. Kulkarni. Exploring single and multilevel jit compilation policy for modern machines. *ACM Trans. Archit. Code Optim.*, 10(4):22:1–22:29, December 2013.

[14] Michael Paleczny, Christopher Vick, and Cliff Click. The java hotspot^{TM} server compiler. In *Proceedings of the 2001 Symposium on Java^{TM} Virtual Machine Research and Technology Symposium - Volume 1*, JVM'01, pages 1–1, Berkeley, CA, USA, 2001. USENIX Association.

[15] Thomas Kotzmann, Christian Wimmer, Hanspeter Mössenböck, Thomas Rodriguez, Kenneth Russell, and David Cox. Design of the java hotspot^{TM} client compiler for java 6. *ACM Trans. Archit. Code Optim.*, 5(1):7:1–7:32, May 2008.

[16] Chandra Krintz, David Grove, Derek Lieber, Vivek Sarkar, and Brad Calder. Reducing the overhead of compilation delay. Technical report, La Jolla, CA, USA, 2000.

[17] Oscar Callaú, Romain Robbes, Éric Tanter, and David Röthlisberger. How developers use the dynamic features of programming languages: The case of smalltalk. In *Proceedings of the 8th Working Conference on Mining Software Repositories*, MSR '11, pages 23–32, New York, NY, USA, 2011. ACM.

[18] David Maxwell and Leif Azzopardi. Stuck in traffic: How temporal delays affect search behaviour. In *Proceedings of the 5th Information Interaction in Context Symposium*, IIiX '14, pages 155–164, New York, NY, USA, 2014. ACM.

[19] Avi Rushinek and Sara F. Rushinek. What makes users happy? *Commun. ACM*, 29(7):594–598, July 1986.

[20] Weiwu Hu, Yifu Zhang, and Jie Fu. An introduction to cpu and dsp design in china. *Science China Information Sciences*, 59(1):1–8, 2016.

[21] 周志明. *深入理解Java虚拟机：JVM高级特性与最佳实践*. 北京：机械工业出版社, 2014.

[22] Wing Hang Li, David R. White, and Jeremy Singer. Jvm-hosted languages: They talk the talk, but do they walk the walk? In *Proceedings of the 2013 International Conference on Principles and Practices of Programming on the Java Platform: Virtual Machines, Languages, and Tools*, PPPJ '13, pages 101–112, New York, NY, USA, 2013. ACM.

[23] John McCarthy. History of programming languages i. chapter History of LISP, pages 173–185. ACM, New York, NY, USA, 1981.

[24] Gilbert Joseph Hansen. *Adaptive Systems for the Dynamic Run-time Optimization of Programs.* PhD thesis, Pittsburgh, PA, USA, 1974. AAI7420364.

[25] L. Peter Deutsch and Allan M. Schiffman. Efficient implementation of the smalltalk-80 system. In *Proceedings of the 11th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL '84, pages 297–302, New York, NY, USA, 1984. ACM.

[26] Urs Hölzle and David Ungar. Reconciling responsiveness with performance in pure object-oriented languages. *ACM Trans. Program. Lang. Syst.*, 18(4):355–400, July 1996.

[27] Urs Hölzle, Craig Chambers, and David Ungar. Optimizing dynamically-typed object-oriented languages with polymorphic inline caches. In *Proceedings of the European Conference on Object-Oriented Programming*, ECOOP '91, pages 21–38, London, UK, UK, 1991. Springer-Verlag.

[28] Stephen J. Fink and Feng Qian. Design, implementation and evaluation of adaptive recompilation with on-stack replacement. In *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-directed and Runtime Optimization*, CGO '03, pages 241–252, Washington, DC, USA, 2003. IEEE Computer Society.

[29] Urs Hölzle, Craig Chambers, and David Ungar. Debugging optimized code with dynamic deoptimization. In *Proceedings of the ACM SIGPLAN 1992 Conference on Programming Language Design and Implementation*, PLDI '92, pages 32–43, New York, NY, USA, 1992. ACM.

[30] Prasad Kulkarni, Matthew Arnold, and Michael Hind. Dynamic compilation: The benefits of early investing. In *Proceedings of the 3rd International Conference on*

Virtual Execution Environments, VEE '07, pages 94–104, New York, NY, USA, 2007. ACM.

[31] J. W. Davidson and J. V. Gresh. Cint: A risc interpreter for the c programming language. *SIGPLAN Not.*, 22(7):189–198, July 1987.

[32] Benjamin Vitale and Tarek S. Abdelrahman. Catenation and specialization for tcl virtual machine performance. In *Proceedings of the 2004 Workshop on Interpreters, Virtual Machines and Emulators*, IVME '04, pages 42–50, New York, NY, USA, 2004. ACM.

[33] Yudi Zheng, Lubomír Bulej, and Walter Binder. Accurate profiling in the presence of dynamic compilation. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA 2015, pages 433–450, New York, NY, USA, 2015. ACM.

[34] Matthew Arnold, Stephen Fink, Vivek Sarkar, and Peter F. Sweeney. A comparative study of static and profile-based heuristics for inlining. In *Proceedings of the ACM SIGPLAN Workshop on Dynamic and Adaptive Compilation and Optimization*, DYNAMO '00, pages 52–64, New York, NY, USA, 2000. ACM.

[35] Matthew Arnold, Michael Hind, and Barbara G. Ryder. Online feedback-directed optimization of java. In *Proceedings of the 17th ACM SIGPLAN Conference on Object-oriented Programming, Systems, Languages, and Applications*, OOPSLA '02, pages 111–129, New York, NY, USA, 2002. ACM.

[36] Richard Jones, Antony Hosking, and Eliot Moss. *The Garbage Collection Handbook: The Art of Automatic Memory Management*. Chapman & Hall/CRC, 1st edition, 2011.

[37] Lokesh Gidra, Gaël Thomas, Julien Sopena, and Marc Shapiro. A study of the scalability of stop-the-world garbage collectors on multicores. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '13, pages 229–240, New York, NY, USA, 2013. ACM.

[38] Katherine Barabash, Yoav Ossia, and Erez Petrank. Mostly concurrent garbage collection revisited. In *Proceedings of the 18th Annual ACM SIGPLAN Conference on Object-oriented Programing, Systems, Languages, and Applications*, OOPSLA '03, pages 255–268, New York, NY, USA, 2003. ACM.

[39] Sven Auhagen, Lars Bergstrom, Matthew Fluet, and John Reppy. Garbage collection for multicore numa machines. In *Proceedings of the 2011 ACM SIGPLAN Workshop on Memory Systems Performance and Correctness*, MSPC '11, pages 51–57, New York, NY, USA, 2011. ACM.

[40] Christine H. Flood, David Detlefs, Nir Shavit, and Xiaolan Zhang. Parallel garbage collection for shared memory multiprocessors. In *Proceedings of the 2001 Symposium on Java^{TM} Virtual Machine Research and Technology Symposium - Volume 1*, JVM'01, pages 21–21, Berkeley, CA, USA, 2001. USENIX Association.

[41] Yossi Levanoni and Erez Petrank. An on-the-fly reference-counting garbage collector for java. *ACM Trans. Program. Lang. Syst.*, 28(1):1–69, January 2006.

[42] Pramod G. Joisha. A principled approach to nondeferred reference-counting garbage collection. In *Proceedings of the Fourth ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, VEE '08, pages 131–140, New York, NY, USA, 2008. ACM.

[43] Yoav Ossia, Ori Ben-Yitzhak, and Marc Segal. Mostly concurrent compaction for mark-sweep gc. In *Proceedings of the 4th International Symposium on Memory Management*, ISMM '04, pages 25–36, New York, NY, USA, 2004. ACM.

[44] Robin Garner, Stephen M. Blackburn, and Daniel Frampton. Effective prefetch for mark-sweep garbage collection. In *Proceedings of the 6th International Symposium on Memory Management*, ISMM '07, pages 43–54, New York, NY, USA, 2007. ACM.

[45] Narendran Sachindran and J. Eliot B. Moss. Mark-copy: Fast copying gc with less space overhead. In *Proceedings of the 18th Annual ACM SIGPLAN Conference on Object-oriented Programing, Systems, Languages, and Applications*, OOPSLA '03, pages 326–343, New York, NY, USA, 2003. ACM.

[46] Martin Kero, Johan Nordlander, and Per Lindgren. A correct and useful incremental copying garbage collector. In *Proceedings of the 6th International Symposium on Memory Management*, ISMM '07, pages 129–140, New York, NY, USA, 2007. ACM.

[47] Liangliang Tong and Francis C. M. Lau. Exploiting memory usage patterns to improve garbage collections in java. In *Proceedings of the 8th International Conference on the Principles and Practice of Programming in Java*, PPPJ '10, pages 39–48, New York, NY, USA, 2010. ACM.

[48] Kazuya Morikawa, Tomoharu Ugawa, and Hideya Iwasaki. Adaptive scanning reduces sweep time for the lisp2 mark-compact garbage collector. In *Proceedings of the 2013 International Symposium on Memory Management*, ISMM '13, pages 15–26, New York, NY, USA, 2013. ACM.

[49] Alan Demers, Mark Weiser, Barry Hayes, Hans Boehm, Daniel Bobrow, and Scott Shenker. Combining generational and conservative garbage collection: Framework and implementations. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '90, pages 261–269, New York, NY, USA, 1990. ACM.

[50] Tamar Domani, Elliot K. Kolodner, and Erez Petrank. A generational on-the-fly garbage collector for java. In *Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation*, PLDI '00, pages 274–284, New York, NY, USA, 2000. ACM.

[51] Matthias Grimmer, Manuel Rigger, Lukas Stadler, Roland Schatz, and Hanspeter Mössenböck. An efficient native function interface for java. In *Proceedings of the 2013 International Conference on Principles and Practices of Programming on the Java Platform: Virtual Machines, Languages, and Tools*, PPPJ '13, pages 35–44, New York, NY, USA, 2013. ACM.

[52] Marc Berndl, Benjamin Vitale, Mathew Zaleski, and Angela Demke Brown. Context threading: A flexible and efficient dispatch technique for virtual machine interpreters. In *Proceedings of the International Symposium on Code Generation and Optimization*, CGO '05, pages 15–26, Washington, DC, USA, 2005. IEEE Computer Society.

[53] Gülfem Savrun-Yeniçeri, Wei Zhang, Huahan Zhang, Chen Li, Stefan Brunthaler, Per Larsen, and Michael Franz. Efficient interpreter optimizations for the jvm. In *Proceedings of the 2013 International Conference on Principles and Practices of Programming on the Java Platform: Virtual Machines, Languages, and Tools*, PPPJ '13, pages 113–123, New York, NY, USA, 2013. ACM.

[54] M. Anton Ertl and David Gregg. The behavior of efficient virtual machine interpreters on modern architectures. In *Proceedings of the 7th International Euro-Par Conference Manchester on Parallel Processing*, Euro-Par '01, pages 403–412, London, UK, UK, 2001. Springer-Verlag.

[55] 黄明凯, 刘先华, 谭明星, 谢子超, 程旭. 一种面向解释器的间接转移预测技术. *计算机研究与发展*, 52(1):66–82, 2015.

[56] M. Anton Ertl and David Gregg. The structure and performance of efficient interpreters. *Journal of Instruction-Level Parallelism*, 5:2003, 2003.

[57] M. Anton Ertl. Stack caching for interpreters. In *Proceedings of the ACM SIGPLAN 1995 Conference on Programming Language Design and Implementation*, PLDI '95, pages 315–327, New York, NY, USA, 1995. ACM.

[58] Todd A. Proebsting. Optimizing an ansi c interpreter with superoperators. In *Proceedings of the 22Nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '95, pages 322–332, New York, NY, USA, 1995. ACM.

[59] Gilles Duboscq, Thomas Würthinger, Lukas Stadler, Christian Wimmer, Doug Simon, and Hanspeter Mössenböck. An intermediate representation for speculative optimizations in a dynamic compiler. In *Proceedings of the 7th ACM Workshop on Virtual Machines and Intermediate Languages*, VMIL '13, pages 1–10, New York, NY, USA, 2013. ACM.

[60] Kirk Kelsey, Tongxin Bai, Chen Ding, and Chengliang Zhang. Fast track: A software system for speculative program optimization. In *Proceedings of the 7th Annual IEEE/ACM International Symposium on Code Generation and Optimization*, CGO '09, pages 157–168, Washington, DC, USA, 2009. IEEE Computer Society.

[61] Matthew Arnold, Stephen Fink, David Grove, Michael Hind, and Peter F. Sweeney. Adaptive optimization in the jalapeno jvm. *SIGPLAN Not.*, 46(4):65–83, May 2011.

[62] MichałCierniak, Guei-Yuan Lueh, and James M. Stichnoth. Practicing judo: Java under dynamic optimizations. In *Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation*, PLDI '00, pages 13–26, New York, NY, USA, 2000. ACM.

[63] Chandra Ckrintz and Brad Calder. Using annotations to reduce dynamic optimization time. Technical report, La Jolla, CA, USA, 2000.

[64] Y. K. Lim, S. Parambil, C. G. Kim, and S. H. Lee. A selective ahead-of-time compiler on android device. In *Proceedings of the Information Science and Applications*, ICISA '12, pages 1–6, Suwon, South Korea, 2012. IEEE.

[65] Michael P. Plezbert and Ron K. Cytron. Does "just in time" = "better late than never"? In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '97, pages 120–131, New York, NY, USA, 1997. ACM.

[66] Timothy Harris. Controlling run-time compilation. In *In Procedings of the IEEE Workshop on Programming Languages for Real-Time Industrial Applications*, pages 75–84, 1998.

[67] Prasad A. Kulkarni. Jit compilation policy for modern machines. In *Proceedings of the 2011 ACM International Conference on Object Oriented Programming Systems Languages and Applications*, OOPSLA '11, pages 773–788, New York, NY, USA, 2011. ACM.

[68] Lukas Stadler, Gilles Duboscq, Hanspeter Mössenböck, and Thomas Würthinger. Compilation queuing and graph caching for dynamic compilers. In *Proceedings of the Sixth ACM Workshop on Virtual Machines and Intermediate Languages*, VMIL '12, pages 49–58, New York, NY, USA, 2012. ACM.

[69] Yufei Ding, Mingzhou Zhou, Zhijia Zhao, Sarah Eisenstat, and Xipeng Shen. Finding the limit: Examining the potential and complexity of compilation scheduling for jit-based runtime systems. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '14, pages 607–622, New York, NY, USA, 2014. ACM.

[70] Stefan Brunthaler. Inline caching meets quickening. In *Proceedings of the 24th European Conference on Object-oriented Programming*, ECOOP'10, pages 429–451, Berlin, Heidelberg, 2010. Springer-Verlag.

[71] Aibek Sarimbekov, Andrej Podzimek, Lubomir Bulej, Yudi Zheng, Nathan Ricci, and Walter Binder. Characteristics of dynamic jvm languages. In *Proceedings of the 7th ACM Workshop on Virtual Machines and Intermediate Languages*, VMIL '13, pages 11–20, New York, NY, USA, 2013. ACM.

[72] Kim Hazelwood and David Grove. Adaptive online context-sensitive inlining. In *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-directed and Runtime Optimization*, CGO '03, pages 253–264, Washington, DC, USA, 2003. IEEE Computer Society.

[73] Jeffrey Dean, David Grove, and Craig Chambers. Optimization of object-oriented programs using static class hierarchy analysis. In *Proceedings of the 9th European Conference on Object-Oriented Programming*, ECOOP '95, pages 77–101, London, UK, UK, 1995. Springer-Verlag.

[74] David Detlefs and Ole Agesen. Inlining of virtual methods. In *Proceedings of the 13th European Conference on Object-Oriented Programming*, ECOOP '99, pages 258–278, London, UK, UK, 1999. Springer-Verlag.

[75] Craig Chambers and David Ungar. Making pure object-oriented languages practical. In *Conference Proceedings on Object-oriented Programming Systems, Languages, and Applications*, OOPSLA '91, pages 1–15, New York, NY, USA, 1991. ACM.

[76] Jong-Deok Choi, Manish Gupta, Mauricio Serrano, Vugranam C. Sreedhar, and Sam Midkiff. Escape analysis for java. In *Proceedings of the 14th ACM SIGPLAN Conference on Object-oriented Programming, Systems, Languages, and Applications*, OOPSLA '99, pages 1–19, New York, NY, USA, 1999. ACM.

[77] Bruno Blanchet. Escape analysis for java™: Theory and practice. *ACM Trans. Program. Lang. Syst.*, 25(6):713–775, November 2003.

[78] Kyungwoo Lee, Xing Fang, and Samuel P. Midkiff. Practical escape analyses: How good are they? In *Proceedings of the 3rd International Conference on Virtual Execution Environments*, VEE '07, pages 180–190, New York, NY, USA, 2007. ACM.

[79] Jong-Deok Choi, Manish Gupta, Mauricio J. Serrano, Vugranam C. Sreedhar, and Samuel P. Midkiff. Stack allocation and synchronization optimizations for java using escape analysis. *ACM Trans. Program. Lang. Syst.*, 25(6):876–910, November 2003.

[80] Peter Molnar, Andreas Krall, and Florian Brandner. Stack allocation of objects in the cacao virtual machine. In *Proceedings of the 7th International Conference on Principles and Practice of Programming in Java*, PPPJ '09, pages 153–161, New York, NY, USA, 2009. ACM.

[81] Lei Wang and Xikun Sun. Escape analysis for synchronization removal. In *Proceedings of the 2006 ACM Symposium on Applied Computing*, SAC '06, pages 1419–1423, New York, NY, USA, 2006. ACM.

[82] Lukas Stadler, Thomas Würthinger, and Hanspeter Mössenböck. Partial escape analysis and scalar replacement for java. In *Proceedings of Annual IEEE/ACM International Symposium on Code Generation and Optimization*, CGO '14, pages 165:165–165:174, New York, NY, USA, 2014. ACM.

[83] 帕特森(Patterson D.A.) 亨尼西(Hennessy J.L.). 计算机体系结构——量化研究方法（第三版）. 电子工业出版社, 2004.

[84] Christoph Lameter. Numa (non-uniform memory access): An overview. *Queue*, 11(7):40:40–40:51, July 2013.

[85] Oracle. Java hotspot virtual machine performance enhancements. `http://docs.oracle.com/javase/7/docs/technotes/guides/vm/performance-enhancements-7.html`.

[86] Mustafa M. Tikir and Jeffery K. Hollingsworth. Numa-aware java heaps for server applications. In *Proceedings of the 19th IEEE International Parallel and Distributed Processing Symposium (IPDPS'05) - Papers - Volume 01*, IPDPS '05, pages 108.2–, Washington, DC, USA, 2005. IEEE Computer Society.

[87] Ming Liu and Tao Li. Optimizing virtual machine consolidation performance on numa server architecture for cloud workloads. In *Proceeding of the 41st Annual International Symposium on Computer Architecuture*, ISCA '14, pages 325–336, Piscataway, NJ, USA, 2014. IEEE Press.

[88] Sameer Kulkarni and John Cavazos. Mitigating the compiler optimization phase-ordering problem using machine learning. In *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications*, OOPSLA '12, pages 147–162, New York, NY, USA, 2012. ACM.

[89] Michael R. Jantz and Prasad A. Kulkarni. Performance potential of optimization phase selection during dynamic jit compilation. In *Proceedings of the 9th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, VEE '13, pages 131–142, New York, NY, USA, 2013. ACM.

[90] Douglas Simon, John Cavazos, Christian Wimmer, and Sameer Kulkarni. Automatic construction of inlining heuristics using machine learning. In *Proceedings of the 2013 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, CGO '13, pages 1–12, Washington, DC, USA, 2013. IEEE Computer Society.

[91] 蔡嵩松. 软硬件协同设计的*Java*虚拟机系统实现与性能优化. 博士学位论文, 中国科学院研究生院, 2011.

[92] Weiwu Hu, Liang Yang, Bao Xia Fan, Huandong Wang, and Yunji Chen. An 8-core mips-compatible processor in 32/28 nm bulk cmos. *J. Solid-State Circuits*, 49(1):41–49, 2014.

[93] Weiwu Hu, Yifu Zhang, Liang Yang, Bao Xia Fan, Yunji Chen, Shi Qiang Zhong, Huandong Wang, Zichu Qi, Pengyu Wang, Xiang Gao, Xu Yang, Bin Xiao, Hongsheng Wang, Zongren Yang, Liqiong Yang, and Shuai Chen. Godson-3b1500: A 32nm 1.35ghz 40w 172.8gflops 8-core processor. In *2013 IEEE International Solid-State Circuits Conference - Digest of Technical Papers, ISSCC 2013, San Francisco, CA, USA, February 17-21, 2013*, pages 54–55, 2013.

[94] Standard Performance Evaluation Corporation. Specjvm98. `http://www.spec.org/jvm98/`.

[95] Stephen M. Blackburn, Robin Garner, Chris Hoffmann, Asjad M. Khang, Kathryn S. McKinley, Rotem Bentzur, Amer Diwan, Daniel Feinberg, Daniel Frampton, Samuel Z. Guyer, Martin Hirzel, Antony Hosking, Maria Jump, Han Lee, J. Eliot B. Moss, Aashish Phansalkar, Darko Stefanović, Thomas VanDrunen, Daniel von Dincklage, and Ben Wiedermann. The dacapo benchmarks: Java benchmarking development and analysis. In *Proceedings of the 21st Annual ACM SIGPLAN Conference on Object-oriented Programming Systems, Languages, and Applications*, OOPSLA '06, pages 169–190, New York, NY, USA, 2006. ACM.

[96] Jonathan L. Schilling. The simplest heuristics may be the best in java jit compilers. *SIGPLAN Not.*, 38(2):36–46, February 2003.

[97] Yijun Liu and Anthony Shi-Sheung Fong. Heuristic optimisation algorithm for java dynamic compilation. *IET Software*, 6(4):307–312, 2012.

[98] Kevin P. Murphy. *Machine Learning: A Probabilistic Perspective*. The MIT Press, 2012.

[99] Giulio Concas, Michele Marchesi, Sandro Pinna, and Nicola Serra. Power-laws in a large object-oriented software system. *IEEE Trans. Softw. Eng.*, 33(10):687–708, October 2007.

[100] Panagiotis Louridas, Diomidis Spinellis, and Vasileios Vlachos. Power laws in software. *ACM Trans. Softw. Eng. Methodol.*, 18(1):2:1–2:26, October 2008.

[101] Simon F. Goldsmith, Alex S. Aiken, and Daniel S. Wilkerson. Measuring empirical computational complexity. In *Proceedings of the the 6th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on The Foundations of Software Engineering*, ESEC-FSE '07, pages 395–404, New York, NY, USA, 2007. ACM.

[102] Yu Qu, Qinghua Zheng, Ting Liu, Jian Li, and Xiaohong Guan. In-depth measurement and analysis on densification power law of software execution. In *Proceedings of the 5th International Workshop on Emerging Trends in Software Metrics*, WETSoM 2014, pages 55–58, New York, NY, USA, 2014. ACM.

[103] Yu Qu, Xiaohong Guan, Qinghua Zheng, Ting Liu, Jianliang Zhou, and Jian Li. Calling network: A new method for modeling software runtime behaviors. *SIGSOFT Softw. Eng. Notes*, 40(1):1–8, February 2015.

[104] Zhongpeng Lin and Jim Whitehead. Why power laws?: An explanation from fine-grained code changes. In *Proceedings of the 12th Working Conference on Mining Software Repositories*, MSR '15, pages 68–75, Piscataway, NJ, USA, 2015. IEEE Press.

[105] Nikhil Padhye. Evolutionary approaches for real world applications in 21st century. In *Proceedings of the 14th Annual Conference Companion on Genetic and Evolutionary Computation*, GECCO '12, pages 43–48, New York, NY, USA, 2012. ACM.

[106] Jonathan E. Rowe. Genetic algorithm theory. In *Proceedings of the 14th Annual Conference Companion on Genetic and Evolutionary Computation*, GECCO '12, pages 917–940, New York, NY, USA, 2012. ACM.

[107] Seong-Won Lee, Soo-Mook Moon, and Seong-Moo Kim. Enhanced hot spot detection heuristics for embedded java just-in-time compilers. In *Proceedings of the 2008 ACM SIGPLAN-SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems*, LCTES '08, pages 13–22, New York, NY, USA, 2008. ACM.

[108] Hiroshi Inoue, Hiroshige Hayashizaki, Peng Wu, and Toshio Nakatani. Adaptive multi-level compilation in a trace-based java jit compiler. In *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications*, OOPSLA '12, pages 179–194, New York, NY, USA, 2012. ACM.

[109] Stefan Brunthaler. Virtual-machine abstraction and optimization techniques. *Electron. Notes Theor. Comput. Sci.*, 253(5):3–14, December 2009.

[110] Gregor Richards, Sylvain Lebresne, Brian Burg, and Jan Vitek. An analysis of the dynamic behavior of javascript programs. In *Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '10, pages 1–12, New York, NY, USA, 2010. ACM.

[111] Jason Mccandless and David Gregg. Compiler techniques to improve dynamic branch prediction for indirect jump and call instructions. *ACM Trans. Archit. Code Optim.*, 8(4):24:1–24:20, January 2012.

[112] P. M. Kogge. An architectural trail to threaded-code systems. *Computer*, 15(3):22–32, March 1982.

[113] Ian Piumarta and Fabio Riccardi. Optimizing direct threaded code by selective inlining. In *Proceedings of the ACM SIGPLAN 1998 Conference on Programming Language Design and Implementation*, PLDI '98, pages 291–300, New York, NY, USA, 1998. ACM.

[114] M. Anton Ertl and David Gregg. Combining stack caching with dynamic superinstructions. In *Proceedings of the 2004 Workshop on Interpreters, Virtual Machines and Emulators*, IVME '04, pages 7–14, New York, NY, USA, 2004. ACM.

[115] Erven Rohou, Bharath Narasimha Swamy, and André Seznec. Branch prediction and the performance of interpreters: Don't trust folklore. In *Proceedings of the 13th Annual IEEE/ACM International Symposium on Code Generation and Optimization*, CGO '15, pages 103–114, Washington, DC, USA, 2015. IEEE Computer Society.

[116] Luis Mastrangelo, Luca Ponzanelli, Andrea Mocci, Michele Lanza, Matthias Hauswirth, and Nathaniel Nystrom. Use at your own risk: The java unsafe api in the wild. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA 2015, pages 695–710, New York, NY, USA, 2015. ACM.

致　　谢

值此论文即将完成之际，我心中充满了对良师益友的无限感激之情。

首先，我把最诚挚的谢意献给我敬爱的博士生导师王剑老师。王老师高尚的品德、渊博的学识、对学科前沿敏锐的洞察力以及严谨的治学态度，令我十分钦佩，为我今后的学习和工作树立了优秀的榜样。在博士生阶段，我十分有幸得到了王老师的悉心指导和谆谆教诲，使我在做人、做事和做学问方面逐步走向成熟。在此，谨向王老师表示最衷心的感谢!

我还要特别感谢我硕士阶段的导师胡伟武老师。胡老师崇高的人生理想、自主创新的科研态度以及为建设中国自主信息产业无私奉献的精神，深深地感染着我，使我受益匪浅。在此谨向胡老师表示崇高的敬意和衷心的感谢。

感谢我的组长靳国杰老师。靳老师对我倾注了大量心血，手把手将我从Java虚拟机的新手逐渐培养成独当一面的虚拟机开发者。同时也在工作和生活中激励着我不断前进。靳老师一直是我学习的好榜样。

感谢章隆兵老师、高翔老师和肖俊华老师对我无微不至的关怀，在学习、工作和生活上给了我许多帮助、鼓励和指导。

感谢中国科学院大学、中科院计算所和龙芯课题组为我们提供了完善的学习条件以及学以致用的科研环境。感谢研究生部的张平老师、冯刚老师、周世佳老师和李丹老师等对我们的关心和指导。

感谢UCASthesis论文模板的所有贡献者，他们让我的写作变得灵活自如。

感谢陈新科、孟晓甫、敖琪、段玮、张晓辉、苏文、王焕东、吴瑞阳、苏孟豪、邱吉、薛双百、廖彬、张爽爽、王昊、刘臻、孟时光、陈厦、赵鹏飞、范振亚、王一光、王振华、陈明、李森、毕研儒、王和俊、张良、黄小敏、丁璐璐、王韩月、黄如欢、张立志、夏飞、巍巍、王学辉、徐帆、臧大伟、苑鲁峰、姚彦斌和余钧等。他们勤奋好学、刻苦科研的精神给我留下了深刻的印象，也曾在生活上和学习上给了我许多帮助和指导性的意见。感谢龙芯的各位同事，包括王雪倩、任博文、徐成华、蒙天放、符兴建、李星、杨小娟、李雪峰、邹滨泽和江俊俊等。感谢他们对我工作的帮助和支持。感谢我的朋友们，感谢他们与我同甘共苦，感谢来自他们的陪伴、支持和鼓励。

感谢我的父母，感谢他们对我无私的爱，以及含辛茹苦的培养。他们永远是我最温暖、最强大的后盾。感谢我的妻子张雪萍。在学习和工作中，你是我最得力的帮手；在生活上，你是我永远的依靠。此生有你的陪伴是我最大的幸福!

最后诚挚地感谢在百忙中审阅本论文和参加本次答辩的各位专家学者!

作 者 简 历

基本情况

姓名：傅杰　　**性别：**男　　**出生日期：**1987.12　　**籍贯：**重庆九龙坡

【教育经历】

2011年9月— 至今，中国科学院计算技术研究所，计算机系统结构专业，硕博连读

2007年9月— 2011年6月，西安交通大学，计算机科学与技术专业，本科

【攻读博士学位期间发表的论文】

[1] Jie Fu, Guojie Jin, Longbin Zhang, Jian Wang. CAOS: Combined Analysis with Online Sifting for Dynamic Compilation Systems. In Proceedings of the 13th ACM International Conference on Computing Frontiers(CF 2016), 110-118, Como, Italy, May 16-18, 2016.

[2] 傅杰, 靳国杰, 章隆兵, 王剑. 软硬件协同设计的解释器指令分派方法[J]. 高技术通讯(中文), 已录用.

[3] Hu Weiwu, Zhang Yifu, Fu Jie. An introduction to CPU and DSP design in China. SCIENCE CHINA Information Sciences[J](《中国科学：信息科学》英文版), 2016, 59(1): 012101:1 – 012101:8.

[4] 傅杰, 廖彬, 陈新科, 靳国杰, 章隆兵, 王剑. 编译队列监视下的Size-Speed动态编译调度算法[J]. 高技术通讯(中文),2014, 24(12):1228-1237.

【攻读博士学位期间申请的专利】

[1] 傅杰，靳国杰，高翔。禁止已翻译指令序列执行的方法、装置和虚拟机。
发明专利，授权号：CN103744641B。

[2] 傅杰，靳国杰，高翔，王剑。动态编译方法及装置。
发明专利，申请号：201410727430.3。

[3] 傅杰，靳国杰，高翔，王剑。热点方法识别方法及装置。
发明专利，申请号：201410812093.8。

[4] 傅杰，靳国杰，高翔，王剑。解释器访存优化方法及装置。
发明专利，申请号：201510053110.9。

[5] 傅杰，靳国杰，高翔，王剑。虚拟机指令的解释执行方法和装置。
发明专利，申请号：201510368529.3。

[6] 傅杰，靳国杰，高翔，王剑。自适应动态编译调度方法及装置。
发明专利，申请号：201510334274.9。

[7] 傅杰，靳国杰，高翔，王剑。动态编译调度方法及装置。
发明专利，申请号：201510341035.6。

[8] 傅杰，靳国杰，高翔，王剑。指令分派方法和解释器。
发明专利，申请号：201610109699.4。

[9] 傅杰，靳国杰，高翔，王剑。待编译代码的过滤方法和装置。
发明专利，申请号：201610317585.9。

【攻读博士学位期间参加的科研项目】

[1] 国家“核高基”科技重大专项课题“高性能多核CPU研发与应用”（2009ZX01028-002-003）

[2] 国家863课题“面向多核/众核处理器的并行程序调试技术与工具”（2012AA010901）

【攻读博士学位期间获奖情况】

[1] 2015年中国科学院计算技术研究所华为博士生奖

[2] 2013年中国科学院计算技术研究所优秀志愿者

[3] 2013年中国科学院大学三好学生

[4] 2012年中国科学院研究生院三好学生